This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix. The content has been processed where content has been compressed (code blocks are separated by ⋯- delimiter).

File Summary

Purpose

This file contains a packed representation of a subset of the repository's contents that is considered the most important context. It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

File Format

The content is organized as follows: 1. This summary section 2. Repository information 3. Directory structure 4. Repository files (if enabled) 5. Multiple file entries, each consisting of: a. A header with the file path (## File: path/to/file) b. The full contents of the file in a code block

Usage Guidelines

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

Notes

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: vendor/, parsers/, **/*.svg
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Content has been compressed - code blocks are separated by ⋯- delimiter
- Files are sorted by Git change count (files with more changes are at the bottom)

Directory Structure

```
.iide/
  lsp.json
data/
  icons/
    meson.build
  Adwaita-dark.xml
  Adwaita.xml
  meson.build
  org.github.kai66673.iide.desktop.in
  org.github.kai66673.iide.gschema.xml
  org.github.kai66673.iide.metainfo.xml.in
  org.github.kai66673.iide.service.in
po/
  LINGUAS
  meson.build
  POTFILES.in
scripts/
  install-local-settings.sh.in
src/
  Services/
    Actions/
      Action.vala
      ActionManager.vala
      ShortcutSettings.vala
    LSP/
      config/
        LspConfig.vala
        LspRegistry.vala
        PythonLspConfig.vala
      DiagnosticsService.vala
      IdeLspManager.vala
      IdeLspService.vala
      LspClient.vala
      LspDiagnosticsMark.vala
      LspTypes.vala
    TreeSitter/
      cpp/
        CppTSHighlighter.vala
      model/
        AstItem.vala
        AstModel.vala
      python/
        higlights.scm
        PythonTSHighlighter.vala
```

```
    rust/
      RustHighlighter.vala
    vala/
      ValaTSHighlighter.vala
    BaseTreeSitterHighlighter.vala
    TreeSitterManager.vala
  DocumentManager.vala
  IconProvider.vala
  JsonUtils.vala
  LoggerService.vala
  PanelLayoutHelper.vala
  ProjectManager.vala
  SettingsService.vala
  StyleManager.vala
  Utils.vala
Widgets/
  Panels/
    BasePanel.vala
    DiagnosticsPanel.vala
    LogPanel.vala
    ProjectPanel.vala
    TerminalPanel.vala
  TextView/
    DiagnosticsPopover.vala
    EditorStatusBar.vala
    FontZoomer.vala
    GutterMarkRenderer.vala
    LspCompletionProvider.vala
    SourceView.vala
    TextView.vala
  ToolViews/
    LogView.vala
    ProjectView.vala
    TerminalView.vala
  FuzzyFinderDialog.vala
  PreferencesDialog.vala
  SearchInFilesDialog.vala
application.vala
config.vapi
iide.gresource.xml
main.vala
meson.build
shortcuts-dialog.ui
style.css
window.ui
window.vala
```

```
test_temp/
  test.c
vapi/
  libtreesitter.vapi
.gitmodules
COPYING
meson.build
org.github.kai66673.iide.json
README.md
repomix-output.pdf
```

Files

File: data/icons/meson.build

```
application_id = 'org.github.kai66673.iide'

scalable_dir = 'hicolor' / 'scalable' / 'apps'
install_data(
  scalable_dir / ('@0@.svg').format(application_id),
  install_dir: get_option('datadir') / 'icons' / scalable_dir
)

symbolic_dir = 'hicolor' / 'symbolic' / 'apps'
install_data(
  symbolic_dir / ('@0@-symbolic.svg').format(application_id),
  install_dir: get_option('datadir') / 'icons' / symbolic_dir
)
```

File: data/meson.build

```
desktop_file = i18n.merge_file(
        input: 'org.github.kai66673.iide.desktop.in',
       output: 'org.github.kai66673.iide.desktop',
         type: 'desktop',
       po_dir: '../po',
      install: true,
  install_dir: get_option('datadir') / 'applications'
)

desktop_utils = find_program('desktop-file-validate', required: false)
if desktop_utils.found()
  test('Validate desktop file', desktop_utils, args: [desktop_file])
endif
```

```
appstream_file = i18n.merge_file(
        input: 'org.github.kai66673.iide.metainfo.xml.in',
       output: 'org.github.kai66673.iide.metainfo.xml',
       po_dir: '../po',
      install: true,
  install_dir: get_option('datadir') / 'metainfo'
)

appstreamcli = find_program('appstreamcli', required: false, disabler: true)
test('Validate appstream file', appstreamcli,
     args: ['validate', '--no-net', '--explain', appstream_file])

install_data('org.github.kai66673.iide.gschema.xml',
  install_dir: get_option('datadir') / 'glib-2.0' / 'schemas'
)

compile_schemas = find_program('glib-compile-schemas', required: false, disabler: true)
test('Validate schema file',
     compile_schemas,
     args: ['--strict', '--dry-run', meson.current_source_dir()])


service_conf = configuration_data()
service_conf.set('bindir', get_option('prefix') / get_option('bindir'))
configure_file(
  input: 'org.github.kai66673.iide.service.in',
  output: 'org.github.kai66673.iide.service',
  configuration: service_conf,
  install_dir: get_option('datadir') / 'dbus-1' / 'services'
)

subdir('icons')
```

File: data/org.github.kai66673.iide.desktop.in

```
[Desktop Entry]
Name=iide
Exec=iide
Icon=org.github.kai66673.iide
Terminal=false
Type=Application
Categories=Utility;
Keywords=GTK;
StartupNotify=true
DBusActivatable=true
```

File: data/org.github.kai66673.iide.metainfo.xml.in

```
<?xml version="1.0" encoding="UTF-8"?>
<component type="desktop-application">
  <id>org.github.kai66673.iide</id>
  <metadata_license>CC0-1.0</metadata_license>
  <project_license>GPL-3.0-or-later</project_license>

  <name>Iide</name>
  <summary>Keep the summary shorter, between 10 and 35 characters</summary>
  <description>
    <p>No description</p>
  </description>

  <developer id="tld.vendor">
    <name>Developer name</name>
  </developer>

  <!-- Required: Should be a link to the upstream homepage for the component -->
  <url type="homepage">https://example.org/</url>
  <!-- Recommended: It is highly recommended for open-source projects to display the source
  <url type="vcs-browser">https://example.org/repository</url>
  <!-- Should point to the software's bug tracking system, for users to report new bugs -->
  <url type="bugtracker">https://example.org/issues</url>
  <!-- Should link a FAQ page for this software, to answer some of the most-asked questions
  <!-- URLs of this type should point to a webpage where users can submit or modify translat
  <url type="translate">https://example.org/translate</url>
  <url type="faq">https://example.org/faq</url>
  <!-- Should provide a web link to an online user's reference, a software manual or help pa
  <url type="help">https://example.org/help</url>
  <!-- URLs of this type should point to a webpage showing information on how to donate to t
  <url type="donation">https://example.org/donate</url>
  <!-- This could for example be an HTTPS URL to an online form or a page describing how to
  <url type="contact">https://example.org/contact</url>
  <!-- URLs of this type should point to a webpage showing information on how to contribute
  <url type="contribute">https://example.org/contribute</url>

  <translation type="gettext">iide</translation>
  <!-- All graphical applications having a desktop file must have this tag in the MetaInfo.
     If this is present, appstreamcli compose will pull icons, keywords and categories from
  <launchable type="desktop-id">org.github.kai66673.iide.desktop</launchable>
  <!-- Use the OARS website (https://hughsie.github.io/oars/generate.html) to generate these
  <content_rating type="oars-1.1" />

  <!-- Applications should set a brand color in both light and dark variants like so -->
  <branding>
```

```
    <color type="primary" scheme_preference="light">#ff00ff</color>
    <color type="primary" scheme_preference="dark">#993d3d</color>
  </branding>

  <screenshots>
    <screenshot type="default">
      <image>https://example.org/example1.png</image>
      <caption>A caption</caption>
    </screenshot>
    <screenshot>
      <image>https://example.org/example2.png</image>
      <caption>A caption</caption>
    </screenshot>
  </screenshots>

  <releases>
    <release version="1.0.1" date="2024-01-18">
      <url type="details">https://example.org/changelog.html#version_1.0.1</url>
      <description translate="no">
        <p>Release description</p>
        <ul>
          <li>List of changes</li>
          <li>List of changes</li>
        </ul>
      </description>
    </release>
  </releases>

</component>
```

File: data/org.github.kai66673.iide.service.in

```
[D-BUS Service]
Name=org.github.kai66673.iide
Exec=@bindir@/iide --gapplication-service
```

File: po/LINGUAS

```
# Please keep this file sorted alphabetically.
```

File: po/meson.build

```
i18n.gettext('iide', preset: 'glib')
```

File: po/POTFILES.in

```
# List of source files containing translatable strings.
# Please keep this file sorted alphabetically.
data/org.github.kai66673.iide.desktop.in
data/org.github.kai66673.iide.metainfo.xml.in
data/org.github.kai66673.iide.gschema.xml
src/main.vala
src/window.vala
src/window.ui
```

File: src/config.vapi

```
[CCode (cprefix = "", lower_case_cprefix = "", cheader_filename = "config.h")]
namespace Config {
    public const string GETTEXT_PACKAGE;
    public const string LOCALEDIR;
    public const string PACKAGE_VERSION;
}
```

File: src/main.vala

```
/* main.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

int main (string[] args) {
    Intl.bindtextdomain (Config.GETTEXT_PACKAGE, Config.LOCALEDIR);
    Intl.bind_textdomain_codeset (Config.GETTEXT_PACKAGE, "UTF-8");
    Intl.textdomain (Config.GETTEXT_PACKAGE);
```

```
    // GtkSource.init ();

    var app = new Iide.Application ();
    return app.run (args);
}
```

File: src/shortcuts-dialog.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <object class="AdwShortcutsDialog" id="shortcuts_dialog">
    <child>
      <object class="AdwShortcutsSection">
        <property name="title" translatable="yes">Shortcuts</property>
        <child>
          <object class="AdwShortcutsItem">
            <property name="title" translatable="yes" context="shortcut window">Show Shortcu
            <property name="action-name">app.shortcuts</property>
          </object>
        </child>
        <child>
          <object class="AdwShortcutsItem">
            <property name="title" translatable="yes" context="shortcut window">Quit</proper
            <property name="action-name">app.quit</property>
          </object>
        </child>
      </object>
    </child>
  </object>
</interface>
```

File: src/window.ui

```
<?xml version="1.0" encoding="UTF-8" ?>
<interface>
  <template class="IideWindow" parent="PanelDocumentWorkspace">
    <child type="titlebar">
      <object class="AdwHeaderBar">
        <child type="title">
          <object class="AdwWindowTitle" id="window_title">
            <property name="title">IIDe Application</property>
          </object>
        </child>
        <child type="start">
          <object class="PanelToggleButton">
```

```
        <property name="area">start</property>
        <property name="dock">dock</property>
      </object>
    </child>
    <child type="start">
      <object class="GtkButton">
        <property name="icon-name">list-add-symbolic</property>
        <property name="action-name">workspace.add-page</property>
        <property name="tooltip-text">Add Page</property>
      </object>
    </child>
    <child type="end">
      <object class="PanelToggleButton">
        <property name="area">end</property>
        <property name="dock">dock</property>
      </object>
    </child>
    <child type="end">
      <object class="GtkMenuButton">
        <property name="icon-name">open-menu-symbolic</property>
        <property name="menu-model">primary_menu</property>
        <property name="tooltip-text">Primary Menu</property>
      </object>
    </child>
  </object>
</child>
<child internal-child="dock">
  <object class="PanelDock" id="dock">
  </object>
</child>
<child internal-child="start_area">
  <object class="PanelPaned" id="start_area">
    <child>
      <object class="PanelFrame">
        <child>
          <object class="PanelWidget">
            <property name="title">XXX</property>
            <property name="icon-name">folder-documents-symbolic</property>
            <property name="vexpand">true</property>
            <child>
              <object class="GtkScrolledWindow">
                <property name="hscrollbar-policy">never</property>
                <child>
                  <object class="GtkListView">
                  </object>
                </child>
```

```
                </object>
              </child>
            </object>
          </child>
        </object>
      </child>
    </object>
    <object class="PanelPaned">
      <child>
        <object class="PanelFrame">
          <child>
            <object class="PanelWidget">
              <property name="title">ZZZ ZZZ</property>
              <property name="icon-name">folder-symbolic</property>
              <property name="vexpand">true</property>
              <child>
                <object class="GtkScrolledWindow">
                  <property name="hscrollbar-policy">never</property>
                  <child>
                    <object class="GtkListView">
                    </object>
                  </child>
                </object>
              </child>
            </object>
          </child>
        </object>
      </child>
    </object>
  </child>
  <child internal-child="bottom_area">
    <object class="PanelPaned">
      <child>
        <object class="PanelFrame">
          <child>
            <object class="PanelWidget">
              <property name="title">123</property>
              <property name="icon-name">folder-symbolic</property>
              <property name="vexpand">true</property>
              <child>
                <object class="GtkScrolledWindow">
                  <property name="hscrollbar-policy">never</property>
                  <child>
                    <object class="GtkListView">
                    </object>
                  </child>
```

```
            </object>
          </child>
        </object>
      </child>
    </object>
  </child>
</object>
<object class="PanelPaned">
  <child>
    <object class="PanelFrame">
      <child>
        <object class="PanelWidget">
          <property name="title">OOP</property>
          <property name="icon-name">folder-symbolic</property>
          <property name="vexpand">true</property>
          <child>
            <object class="GtkScrolledWindow">
              <property name="hscrollbar-policy">never</property>
              <child>
                <object class="GtkListView">
                </object>
              </child>
            </object>
          </child>
        </object>
      </child>
    </object>
  </child>
</object>
</child>
<child internal-child="end_area">
  <object class="PanelPaned" id="end_area">
  </object>
</child>
<child internal-child="statusbar">
  <object class="PanelStatusbar">
    <child type="suffix">
      <object class="PanelToggleButton">
        <property name="area">bottom</property>
        <property name="dock">dock</property>
      </object>
    </child>
  </object>
  <object class="PanelStatusbar">
    <child type="suffix">
      <object class="PanelToggleButton">
```

```
            <property name="area">top</property>
            <property name="dock">dock</property>
          </object>
        </child>
      </object>
    </child>
  </template>
  <menu id="primary_menu">
    <section>
      <item>
        <attribute name="label">Quit</attribute>
        <attribute name="action">app.quit</attribute>
      </item>
    </section>
  </menu>
</interface>
```

File: .gitmodules

```
[submodule "vendor/tree-sitter"]
    path = vendor/tree-sitter
    url = https://github.com/albfan/tree-sitter
[submodule "parsers/c"]
    path = parsers/c
    url = https://github.com/tree-sitter/tree-sitter-c
[submodule "parsers/bash"]
    path = parsers/bash
    url = https://github.com/tree-sitter/tree-sitter-bash
[submodule "parsers/json"]
    path = parsers/json
    url = https://github.com/tree-sitter/tree-sitter-json
[submodule "parsers/python"]
    path = parsers/python
    url = https://github.com/tree-sitter/tree-sitter-python
[submodule "parsers/php"]
    path = parsers/php
    url = https://github.com/tree-sitter/tree-sitter-php
[submodule "parsers/cpp"]
    path = parsers/cpp
    url = https://github.com/tree-sitter/tree-sitter-cpp
[submodule "parsers/go"]
    path = parsers/go
    url = https://github.com/tree-sitter/tree-sitter-go
[submodule "parsers/html"]
    path = parsers/html
    url = https://github.com/tree-sitter/tree-sitter-html
```

```
[submodule "parsers/javascript"]
    path = parsers/javascript
    url = https://github.com/tree-sitter/tree-sitter-javascript
[submodule "parsers/ruby"]
    path = parsers/ruby
    url = https://github.com/tree-sitter/tree-sitter-ruby
[submodule "parsers/rust"]
    path = parsers/rust
    url = https://github.com/tree-sitter/tree-sitter-rust
[submodule "parsers/typescript"]
    path = parsers/typescript
    url = https://github.com/tree-sitter/tree-sitter-typescript
[submodule "parsers/yaml"]
    path = parsers/yaml
    url = https://github.com/tree-sitter-grammars/tree-sitter-yaml
[submodule "parsers/xml"]
    path = parsers/xml
    url = https://github.com/tree-sitter-grammars/tree-sitter-xml
[submodule "parsers/vala"]
    path = parsers/vala
    url = https://github.com/vala-lang/tree-sitter-vala
```

File: COPYING

```
GNU GENERAL PUBLIC LICENSE
                       Version 3, 29 June 2007

 Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>
 Everyone is permitted to copy and distribute verbatim copies
 of this license document, but changing it is not allowed.

                            Preamble

  The GNU General Public License is a free, copyleft license for
software and other kinds of works.

  The licenses for most software and other practical works are designed
to take away your freedom to share and change the works.  By contrast,
the GNU General Public License is intended to guarantee your freedom to
share and change all versions of a program--to make sure it remains free
software for all its users.  We, the Free Software Foundation, use the
GNU General Public License for most of our software; it applies also to
any other work released this way by its authors.  You can apply it to
your programs, too.

  When we speak of free software, we are referring to freedom, not
```

price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and

modification follow.

TERMS AND CONDITIONS

0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

The "source code" for a work means the preferred form of the work

for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its

content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

a) The work must carry prominent notices stating that you modified it, and giving a relevant date.

b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".

c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.

d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

a) Convey the object code in, or embodied in, a physical product

(including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.

b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.

c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.

d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.

e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family,

or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or

b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or

c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or

d) Limiting the use for publicity purposes of names of licensors or authors of the material; or

e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or

f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is

governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For

purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the

parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software

Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

```
    <one line to give the program's name and a brief idea of what it does.>
    Copyright (C) <year>  <name of author>

    This program is free software: you can redistribute it and/or modify
    it under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    This program is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
    GNU General Public License for more details.

    You should have received a copy of the GNU General Public License
    along with this program.  If not, see <https://www.gnu.org/licenses/>.
```

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

```
    <program>  Copyright (C) <year>  <name of author>
    This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
    This is free software, and you are welcome to redistribute it
    under certain conditions; type `show c' for details.
```

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see

<https://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <https://www.gnu.org/philosophy/why-not-lgpl.html>.

File: org.github.kai66673.iide.json

```
{
    "id" : "org.github.kai66673.iide",
    "runtime" : "org.gnome.Platform",
    "runtime-version" : "master",
    "sdk" : "org.gnome.Sdk",
    "sdk-extensions" : [
        "org.freedesktop.Sdk.Extension.vala"
    ],
    "command" : "iide",
    "finish-args" : [
        "--share=network",
        "--share=ipc",
        "--socket=fallback-x11",
        "--device=dri",
        "--socket=wayland"
    ],
    "build-options" : {
        "append-path" : "/usr/lib/sdk/vala/bin",
        "prepend-ld-library-path" : "/usr/lib/sdk/vala/lib"
    },
    "cleanup" : [
        "/include",
        "/lib/pkgconfig",
        "/man",
        "/share/doc",
        "/share/gtk-doc",
        "/share/man",
        "/share/pkgconfig",
        "/share/vala",
        "*.la",
        "*.a"
    ],
    "modules" : [
        {
            "name" : "iide",
```

```
            "builddir" : true,
            "buildsystem" : "meson",
            "sources" : [
                {
                    "type" : "git",
                    "url" : "file:///home/kai/Projects"
                }
            ]
        }
    ]
}
```

File: README.md

```
# iide

A description of this project.
```

File: .iide/lsp.json

```
{"languages": [{"id": "cpp", "server": ["clangd", "--header-insertion=never"], "patterns": [
```

File: data/Adwaita-dark.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!--

  Copyright 2020 Christian Hergert <christian@hergert.me>

  GtkSourceView is free software; you can redistribute it and/or
  modify it under the terms of the GNU Lesser General Public
  License as published by the Free Software Foundation; either
  version 2.1 of the License, or (at your option) any later version.

  GtkSourceView is distributed in the hope that it will be useful,
  but WITHOUT ANY WARRANTY; without even the implied warranty of
  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the GNU
  Lesser General Public License for more details.

  You should have received a copy of the GNU Lesser General Public License
  along with this library; if not, see <http://www.gnu.org/licenses/>.

-->
<style-scheme id="Adwaita-dark" _name="Adwaita Dark" version="1.0"><author
    >Christian Hergert</author><_description
    >An style scheme for Adwaita</_description><metadata><property
```

```
        name="variant"
    >dark</property><property
        name="light-variant"
    >Adwaita</property></metadata><!-- Named Colors --><color
    name="blue_1"
    value="#99C1F1"
/><color name="blue_2" value="#62A0EA" /><color
    name="blue_3"
    value="#3584E4"
/><color name="blue_4" value="#1C71D8" /><color
    name="blue_5"
    value="#1A5FB4"
/><color name="blue_6" value="#1B497E" /><color
    name="blue_7"
    value="#193D66"
/><color name="brown_1" value="#CDAB8F" /><color
    name="brown_2"
    value="#B5835A"
/><color name="brown_3" value="#986A44" /><color
    name="brown_4"
    value="#865E3C"
/><color name="brown_5" value="#63452C" /><color
    name="chameleon_3"
    value="#4E9A06"
/><color name="dark_1" value="#777777" /><color
    name="dark_2"
    value="#5E5E5E"
/><color name="dark_3" value="#505050" /><color
    name="dark_4"
    value="#3D3D3D"
/><color name="dark_5" value="#242424" /><color
    name="dark_6"
    value="#121212"
/><color name="dark_7" value="#000000" /><color
    name="green_1"
    value="#8FF0A4"
/><color name="green_2" value="#57E389" /><color
    name="green_3"
    value="#33D17A"
/><color name="green_4" value="#2EC27E" /><color
    name="green_5"
    value="#26A269"
/><color name="green_6" value="#1F7F56" /><color
    name="green_7"
    value="#1C6849"
/><color name="libadwaita-dark" value="#1d1d20" /><color
```

```
    name="libadwaita-dark-alt"
    value="#242428"
/><color name="light_1" value="#FFFFFF" /><color
    name="light_2"
    value="#FCFCFC"
/><color name="light_3" value="#F6F5F4" /><color
    name="light_4"
    value="#DEDDDA"
/><color name="light_5" value="#C0BFBC" /><color
    name="light_6"
    value="#B0AFAC"
/><color name="light_7" value="#9A9996" /><color
    name="orange_1"
    value="#FFBE6F"
/><color name="orange_2" value="#FFA348" /><color
    name="orange_3"
    value="#FF7800"
/><color name="orange_4" value="#E66100" /><color
    name="orange_5"
    value="#C64600"
/><color name="purple_1" value="#DC8ADD" /><color
    name="purple_2"
    value="#C061CB"
/><color name="purple_3" value="#9141AC" /><color
    name="purple_4"
    value="#813D9C"
/><color name="purple_5" value="#613583" /><color
    name="red_1"
    value="#F66151"
/><color name="red_2" value="#ED333B" /><color
    name="red_3"
    value="#E01B24"
/><color name="red_4" value="#C01C28" /><color
    name="red_5"
    value="#A51D2D"
/><color name="teal_1" value="#93DDC2" /><color
    name="teal_2"
    value="#5BC8AF"
/><color name="teal_3" value="#33B2A4" /><color
    name="teal_4"
    value="#26A1A2"
/><color name="teal_5" value="#218787" /><color
    name="violet_2"
    value="#7D8AC7"
/><color name="violet_3" value="#6362C8" /><color
    name="violet_4"
```

```
    value="#4E57BA"
/><color name="yellow_1" value="#F9F06B" /><color
    name="yellow_2"
    value="#F8E45C"
/><color name="yellow_3" value="#F6D32D" /><color
    name="yellow_4"
    value="#F5C211"
/><color name="yellow_5" value="#E5A50A" /><color
    name="yellow_6"
    value="#D38B09"
/><!-- Global Styles --><style
    name="background-pattern"
    background="#141414"
/><style name="bracket-match" bold="true" /><style
    name="current-line"
    background="libadwaita-dark-alt"
/><style
    name="current-line-number"
    background="libadwaita-dark-alt"
    foreground="dark_1"
/><style name="cursor" foreground="light_5" /><style
    name="draw-spaces"
    foreground="dark_3"
/><style
    name="line-numbers"
    background="libadwaita-dark"
    foreground="dark_2"
/><style name="map-overlay" background="dark_1" /><style
    name="right-margin"
    background="dark_1"
    foreground="dark_1"
/><style
    name="search-match"
    background="#rgba(246,211,45,.5)"
    foreground="dark_5"
/><style
    name="text"
    background="libadwaita-dark"
    foreground="light_5"
/><!-- Defaults --><style
    name="def:base-n-integer"
    foreground="violet_2"
/><style name="def:boolean" foreground="violet_2" /><style
    name="def:comment"
    foreground="dark_1"
/><style name="def:constant" foreground="violet_2" /><style
```

```
    name="def:decimal"
    foreground="violet_2"
/><style name="def:deletion" strikethrough="true" /><style
    name="def:doc-comment-element"
    foreground="light_7"
/><style name="def:emphasis" italic="true" /><style
    name="def:error"
    underline="error"
    underline-color="red_4"
/><style name="def:floating-point" foreground="violet_2" /><style
    name="def:function"
    foreground="blue_2"
/><style name="def:heading" foreground="teal_3" bold="true" /><style
    name="def:identifier"
    foreground="chameleon_3"
/><style name="def:inline-code" foreground="violet_2" /><style
    name="def:link-destination"
    foreground="blue_2"
    italic="true"
    underline="low"
/><style name="def:link-text" foreground="red_2" /><style
    name="def:list-marker"
    foreground="orange_4"
    bold="true"
/><style name="def:net-address" foreground="blue_2" underline="low" /><style
    name="def:note"
    foreground="dark_4"
    background="yellow_4"
    bold="true"
/><style name="def:number" foreground="violet_2" /><style
    name="def:preformatted-section"
    foreground="violet_2"
/><style name="def:preprocessor" foreground="orange_4" /><style
    name="def:shebang"
    foreground="light_7"
    bold="true"
/><style name="def:special-char" foreground="red_1" bold="false" /><style
    name="def:statement"
    foreground="orange_2"
    bold="true"
/><style name="def:string" foreground="teal_2" /><style
    name="def:strong-emphasis"
    bold="true"
/><style name="def:type" foreground="teal_2" bold="true" /><style
    name="def:underlined"
    underline="single"
```

```
/><style
    name="def:warning"
    underline="error"
    underline-color="yellow_4"
/><!-- C# --><style name="c-sharp:format" foreground="violet_4" /><style
    name="c-sharp:preprocessor"
    foreground="dark_2"
/><!-- C --><style name="c:printf" foreground="violet_2" /><style
    name="c:signal-name"
    foreground="red_1"
/><style name="c:storage-class" foreground="teal_2" bold="true" /><style
    name="c:type-keyword"
    foreground="teal_2"
    bold="true"
/><!-- CSS --><style
    name="css:id-selector"
    foreground="teal_3"
    bold="true"
/><style name="css:property-name" foreground="orange_3" /><style
    name="css:pseudo-selector"
    foreground="violet_2"
    bold="true"
/><style
    name="css:selector-symbol"
    foreground="orange_3"
    bold="true"
/><style name="css:type-selector" foreground="teal_3" bold="true" /><style
    name="css:vendor-specific"
    foreground="yellow_5"
/><!-- Diff --><style name="diff:added-line" foreground="teal_3" /><style
    name="diff:changed-line"
    foreground="orange_3"
/><style name="diff:diff-file" foreground="violet_2" /><style
    name="diff:location"
    foreground="yellow_4"
/><style name="diff:removed-line" foreground="red_1" /><!-- Go --><style
    name="go:printf"
    foreground="violet_4"
/><!-- Python 2 --><style
    name="python:builtin-function"
    foreground="blue_2"
/><style name="python:class-name" foreground="teal_2" bold="true" /><style
    name="python:module-handler"
    foreground="red_1"
/><!-- Rust --><style name="rust:attribute" foreground="violet_2" /><style
    name="rust:lifetime"
```

```
        foreground="orange_2"
        bold="false"
        italic="false"
    /><style name="rust:macro" foreground="violet_2" bold="false" /><style
        name="rust:scope"
        foreground="orange_2"
    /><!-- Vala --><style
        name="vala:attributes"
        foreground="light_5"
        bold="false"
    /><!-- XML --><style
        name="xml:attribute-name"
        foreground="orange_3"
    /><style name="xml:attribute-value" foreground="violet_2" /><style
        name="xml:element-name"
        foreground="teal_3"
    /><style name="xml:namespace" foreground="yellow_4" /><style
        name="xml:processing-instruction"
        foreground="yellow_4"
        bold="true"
    /></style-scheme>
```

File: data/Adwaita.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!--

  Copyright 2020 Christian Hergert <christian@hergert.me>

  GtkSourceView is free software; you can redistribute it and/or
  modify it under the terms of the GNU Lesser General Public
  License as published by the Free Software Foundation; either
  version 2.1 of the License, or (at your option) any later version.

  GtkSourceView is distributed in the hope that it will be useful,
  but WITHOUT ANY WARRANTY; without even the implied warranty of
  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the GNU
  Lesser General Public License for more details.

  You should have received a copy of the GNU Lesser General Public License
  along with this library; if not, see <http://www.gnu.org/licenses/>.

-->
<style-scheme id="Adwaita" _name="Adwaita" version="1.0"><author
    >Christian Hergert</author><_description
    >An style scheme for Adwaita</_description><metadata><property
```

```
        name="variant"
    >light</property><property
        name="dark-variant"
    >Adwaita-dark</property></metadata><!-- Named Colors --><color
    name="blue_1"
    value="#99C1F1"
/><color name="blue_2" value="#62A0EA" /><color
    name="blue_3"
    value="#3584E4"
/><color name="blue_4" value="#1C71D8" /><color
    name="blue_5"
    value="#1A5FB4"
/><color name="blue_6" value="#1B497E" /><color
    name="blue_7"
    value="#193D66"
/><color name="brown_1" value="#CDAB8F" /><color
    name="brown_2"
    value="#B5835A"
/><color name="brown_3" value="#986A44" /><color
    name="brown_4"
    value="#865E3C"
/><color name="brown_5" value="#63452C" /><color
    name="chameleon_3"
    value="#4E9A06"
/><color name="dark_1" value="#77767B" /><color
    name="dark_2"
    value="#5E5C64"
/><color name="dark_3" value="#504E55" /><color
    name="dark_4"
    value="#3D3846"
/><color name="dark_5" value="#241F31" /><color
    name="dark_6"
    value="#000000"
/><color name="green_1" value="#8FF0A4" /><color
    name="green_2"
    value="#57E389"
/><color name="green_3" value="#33D17A" /><color
    name="green_4"
    value="#2EC27E"
/><color name="green_5" value="#26A269" /><color
    name="green_6"
    value="#1F7F56"
/><color name="green_7" value="#1C6849" /><color
    name="light_1"
    value="#FFFFFF"
/><color name="light_2" value="#FCFCFC" /><color
```

```
    name="light_3"
    value="#F6F5F4"
/><color name="light_4" value="#DEDDDA" /><color
    name="light_5"
    value="#C0BFBC"
/><color name="light_6" value="#B0AFAC" /><color
    name="light_7"
    value="#9A9996"
/><color name="orange_1" value="#FFBE6F" /><color
    name="orange_2"
    value="#FFA348"
/><color name="orange_3" value="#FF7800" /><color
    name="orange_4"
    value="#E66100"
/><color name="orange_5" value="#C64600" /><color
    name="purple_1"
    value="#DC8ADD"
/><color name="purple_2" value="#C061CB" /><color
    name="purple_3"
    value="#9141AC"
/><color name="purple_4" value="#813D9C" /><color
    name="purple_5"
    value="#613583"
/><color name="red_1" value="#F66151" /><color
    name="red_2"
    value="#ED333B"
/><color name="red_3" value="#E01B24" /><color
    name="red_4"
    value="#C01C28"
/><color name="red_5" value="#A51D2D" /><color
    name="teal_1"
    value="#93DDC2"
/><color name="teal_2" value="#5BC8AF" /><color
    name="teal_3"
    value="#33B2A4"
/><color name="teal_4" value="#26A1A2" /><color
    name="teal_5"
    value="#218787"
/><color name="violet_2" value="#7D8AC7" /><color
    name="violet_3"
    value="#6362C8"
/><color name="violet_4" value="#4E57BA" /><color
    name="yellow_1"
    value="#F9F06B"
/><color name="yellow_2" value="#F8E45C" /><color
    name="yellow_3"
```

```
    value="#F6D32D"
/><color name="yellow_4" value="#F5C211" /><color
    name="yellow_5"
    value="#E5A50A"
/><color name="yellow_6" value="#D38B09" /><!-- Global Styles --><style
    name="background-pattern"
    background="#FAFAFA"
/><style name="bracket-match" bold="true" /><style
    name="current-line"
    background="light_3"
/><style
    name="current-line-number"
    background="light_3"
    foreground="light_7"
/><style name="cursor" foreground="dark_1" /><style
    name="draw-spaces"
    foreground="light_6"
/><style
    name="line-numbers"
    background="light_1"
    foreground="light_6"
/><style name="map-overlay" background="dark_1" /><style
    name="right-margin"
    background="dark_1"
    foreground="dark_1"
/><style
    name="search-match"
    background="yellow_2"
    foreground="dark_4"
/><style
    name="text"
    background="light_1"
    foreground="dark_3"
/><!-- Defaults --><style
    name="def:base-n-integer"
    foreground="violet_4"
/><style name="def:boolean" foreground="violet_4" /><style
    name="def:comment"
    foreground="dark_1"
/><style name="def:constant" foreground="violet_4" /><style
    name="def:decimal"
    foreground="violet_4"
/><style name="def:deletion" strikethrough="true" /><style
    name="def:doc-comment-element"
    foreground="dark_3"
/><style name="def:emphasis" italic="true" /><style
```

```
    name="def:error"
    underline="error"
    underline-color="red_4"
/><style name="def:floating-point" foreground="violet_4" /><style
    name="def:function"
    foreground="blue_4"
/><style name="def:heading" foreground="teal_5" bold="true" /><style
    name="def:identifier"
    foreground="chameleon_3"
/><style name="def:inline-code" foreground="violet_4" /><style
    name="def:link-destination"
    foreground="blue_3"
    italic="true"
    underline="low"
/><style name="def:link-text" foreground="red_3" /><style
    name="def:list-marker"
    foreground="orange_5"
    bold="true"
/><style name="def:net-address" foreground="blue_3" underline="low" /><style
    name="def:note"
    foreground="dark_4"
    background="#FCF7B5"
    bold="true"
/><style name="def:number" foreground="violet_4" /><style
    name="def:preformatted-section"
    foreground="violet_4"
/><style name="def:preprocessor" foreground="orange_5" /><style
    name="def:shebang"
    foreground="dark_1"
    bold="true"
/><style name="def:special-char" foreground="red_2" bold="false" /><style
    name="def:statement"
    foreground="orange_5"
    bold="true"
/><style name="def:string" foreground="teal_5" /><style
    name="def:strong-emphasis"
    bold="true"
/><style name="def:type" foreground="teal_5" bold="true" /><style
    name="def:underlined"
    underline="single"
/><style
    name="def:warning"
    underline="error"
    underline-color="yellow_4"
/><!-- C# --><style name="c-sharp:format" foreground="violet_4" /><style
    name="c-sharp:preprocessor"
```

```
    foreground="dark_2"
/><!-- C --><style name="c:printf" foreground="violet_4" /><style
    name="c:signal-name"
    foreground="red_4"
/><style name="c:storage-class" foreground="teal_5" bold="true" /><style
    name="c:type-keyword"
    foreground="teal_5"
    bold="true"
/><!-- CSS --><style
    name="css:id-selector"
    foreground="teal_5"
    bold="true"
/><style name="css:property-name" foreground="orange_5" /><style
    name="css:pseudo-selector"
    foreground="violet_4"
    bold="true"
/><style
    name="css:selector-symbol"
    foreground="orange_5"
    bold="true"
/><style name="css:type-selector" foreground="teal_5" bold="true" /><style
    name="css:vendor-specific"
    foreground="yellow_6"
/><!-- Diff --><style name="diff:added-line" foreground="teal_4" /><style
    name="diff:changed-line"
    foreground="orange_4"
/><style name="diff:diff-file" foreground="violet_4" /><style
    name="diff:location"
    foreground="yellow_6"
/><style name="diff:removed-line" foreground="red_1" /><!-- Go --><style
    name="go:printf"
    foreground="violet_4"
/><!-- Python 2 --><style
    name="python:builtin-function"
    foreground="blue_4"
/><style name="python:class-name" foreground="teal_5" bold="true" /><style
    name="python:module-handler"
    foreground="red_3"
/><!-- Rust --><style name="rust:attribute" foreground="violet_4" /><style
    name="rust:lifetime"
    foreground="orange_5"
    bold="false"
    italic="false"
/><style name="rust:macro" foreground="violet_4" bold="false" /><style
    name="rust:scope"
    foreground="orange_5"
```

```
    /><!-- Vala --><style
        name="vala:attributes"
        foreground="dark_2"
        bold="false"
    /><!-- XML --><style
        name="xml:attribute-name"
        foreground="orange_5"
    /><style name="xml:attribute-value" foreground="violet_4" /><style
        name="xml:element-name"
        foreground="teal_5"
    /><style name="xml:namespace" foreground="yellow_6" /><style
        name="xml:processing-instruction"
        foreground="yellow_6"
        bold="true"
    /></style-scheme>
```

File: scripts/install-local-settings.sh.in

```
#!/bin/bash
set -e

PREFIX="@PREFIX@"
LOCAL_SCHEMA_DIR="$HOME/.local/share/glib-2.0/schemas"
LOCAL_ICON_DIR="$HOME/.local/share/icons/hicolor"
CURRENT_DIR="$(cd "$(dirname "$0")" && pwd)"

install_schemas() {
    mkdir -p "$LOCAL_SCHEMA_DIR"
    if [ -f "@PREFIX@/share/glib-2.0/schemas/org.github.kai66673.iide.gschema.xml" ]; then
        cp "@PREFIX@/share/glib-2.0/schemas/org.github.kai66673.iide.gschema.xml" "$LOCAL_SC
    elif [ -f "$CURRENT_DIR/data/org.github.kai66673.iide.gschema.xml" ]; then
        cp "$CURRENT_DIR/data/org.github.kai66673.iide.gschema.xml" "$LOCAL_SCHEMA_DIR/"
    fi
    glib-compile-schemas "$LOCAL_SCHEMA_DIR"
    echo "GSettings schemas installed to $LOCAL_SCHEMA_DIR"
}

install_icons() {
    if [ -d "$CURRENT_DIR/data/icons" ]; then
        mkdir -p "$LOCAL_ICON_DIR"
        cp -r "$CURRENT_DIR/data/icons/"* "$LOCAL_ICON_DIR/" 2>/dev/null || true
        gtk-update-icon-cache "$LOCAL_ICON_DIR" 2>/dev/null || true
        echo "Icons installed to $LOCAL_ICON_DIR"
    fi
}
```

```
echo "Installing iide settings locally for development..."
install_schemas
install_icons
echo "Done! If needed, set:"
echo "  export XDG_DATA_DIRS=\"\$HOME/.local/share:\$XDG_DATA_DIRS\""
```

File: src/Services/Actions/Action.vala

```
public abstract class Iide.Action : Object {
    public abstract string id { get; }
    public abstract string name { get; }
    public abstract string? description { get; }
    public abstract string? icon_name { get; }
    public string? shortcut { get; set; }

    public signal void shortcut_changed (string? new_shortcut);
    public signal void state_changed (bool new_state);

    public virtual bool is_toggle { get; default = false; }
    public virtual bool state { get; protected set; default = false; }

    public void update_state (bool new_state) {
        state = new_state;
        state_changed (new_state);
    }

    public abstract bool can_execute ();
    public abstract void execute ();

    public virtual string? category { get; default = null; }
}
```

File: src/Services/Actions/ActionManager.vala

```
public class Iide.ActionManager : Object {
    private static ActionManager? _instance;
    private Gee.HashMap<string, Iide.Action> actions;
    private Gee.HashMap<string, string> shortcuts;
    private Iide.ShortcutSettings shortcut_settings;

    public static ActionManager get_instance () {
        if (_instance == null) {
            _instance = new ActionManager ();
        }
        return _instance;
    }
```

```
private ActionManager () {
    actions = new Gee.HashMap<string, Iide.Action> ();
    shortcuts = new Gee.HashMap<string, string> ();
    shortcut_settings = Iide.ShortcutSettings.get_instance ();
    shortcut_settings.shortcut_changed.connect (on_shortcut_changed);
}

public void register_action (Iide.Action action) {
    actions.set (action.id, action);
    var saved_shortcut = shortcut_settings.get_shortcut (action.id);
    action.shortcut = saved_shortcut;
    shortcuts.set (action.id, saved_shortcut);

    if (action.is_toggle) {
        var saved_state = shortcut_settings.get_toggle_state (action.id);
        action.update_state (saved_state);
    }
}

public void unregister_action (string action_id) {
    actions.unset (action_id);
    shortcuts.unset (action_id);
}

public Iide.Action? get_action (string action_id) {
    return actions.get (action_id);
}

public Gee.Collection<Iide.Action> get_all_actions () {
    return actions.values;
}

public Gee.ArrayList<Iide.Action> get_actions_by_category (string category) {
    var result = new Gee.ArrayList<Iide.Action> ();
    foreach (var action in actions.values) {
        if (action.category == category) {
            result.add (action);
        }
    }
    return result;
}

public void execute (string action_id) {
    var action = actions.get (action_id);
    if (action != null && action.can_execute ()) {
```

```
        action.execute ();
    }
}

public bool can_execute (string action_id) {
    var action = actions.get (action_id);
    return action != null && action.can_execute ();
}

public string? get_shortcut (string action_id) {
    return shortcuts.get (action_id);
}

public void set_shortcut (string action_id, string? shortcut) {
    var action = actions.get (action_id);
    if (action != null) {
        action.shortcut = shortcut;
        shortcuts.set (action_id, shortcut);
        shortcut_settings.set_shortcut (action_id, shortcut);
        action.shortcut_changed (shortcut);
    }
}

public void set_toggle_state (string action_id, bool state) {
    var action = actions.get (action_id);
    if (action != null && action.is_toggle) {
        action.update_state (state);
        shortcut_settings.set_toggle_state (action_id, state);
    }
}

private void on_shortcut_changed (string action_id, string? new_shortcut) {
    var action = actions.get (action_id);
    if (action != null) {
        action.shortcut = new_shortcut;
        shortcuts.set (action_id, new_shortcut);
        action.shortcut_changed (new_shortcut);
    }
}

public void apply_shortcuts_to_application (Gtk.Application app) {
    foreach (var action in actions.values) {
        if (action.is_toggle) {
            var toggle_action = new SimpleAction.stateful (
                action.id,
                null,
```

```
                    new Variant.boolean (action.state)
                );
                toggle_action.activate.connect (() => {
                    execute (action.id);
                });
                action.state_changed.connect ((new_state) => {
                    toggle_action.set_state (new Variant.boolean (new_state));
                });
                app.add_action (toggle_action);
            } else {
                var simple_action = new SimpleAction (action.id, null);
                simple_action.activate.connect (() => {
                    execute (action.id);
                });
                app.add_action (simple_action);
            }

            if (action.shortcut != null && action.shortcut != "") {
                app.set_accels_for_action ("app." + action.id, { action.shortcut });
            }
        }
    }
}
```

File: src/Services/LSP/config/LspConfig.vala

```
public abstract class Iide.LspConfig {
    public abstract string command ();

    public abstract string[] args ();

    public abstract Json.Node initialize_params (string? workspace_root, string? initial_uri

    public abstract Json.Node? server_response_result (Json.Object response);
}
```

File: src/Services/LSP/config/LspRegistry.vala

```
public class Iide.LspRegistry {
    public static string ? get_lsp_id (string language) {
        switch (language) {
        case "python":
            return "basedpyright";
        default:
            return null;
        }
```

```
    }

    public static LspConfig ? get_config (string lsp_id) {
        switch (lsp_id) {
        case "basedpyright" :
            return new PythonLspConfig ();
        default:
            return null;
        }
    }
}
```

File: src/Services/LSP/config/PythonLspConfig.vala

```
public class Iide.PythonLspConfig : Iide.LspConfig {
    public override string command () {
        return "basedpyright-langserver";
    }

    public override string[] args () {
        return { "--stdio" };
    }

    public override Json.Node initialize_params (string? workspace_root, string? initial_uri
        var root_uri = workspace_root ?? "/";
        var params = """{
            "processId": null,
            "clientInfo": { "name": "iide", "version": "0.1.0" },
            "rootUri": "%s",
            "workspaceFolders": [
                { "uri": "%s", "name": "project" }
            ],
            "capabilities": {
                "textDocument": {
                "publishDiagnostics": { "relatedInformation": true }
                },
                "workspace": {
                "configuration": true
                },
                "window": { "workDoneProgress": true }
            },
            "initializationOptions": {
                "python": {},
                "settings": {
                "pyright": { "analysis": { "diagnosticMode": "workspace" } },
                "basedpyright": { "analysis": { "diagnosticMode": "workspace" } }
```

```
                }
            }
        }""".printf (root_uri, root_uri);
        var parser = new Json.Parser ();
        try {
            parser.load_from_data (params);
        } catch (GLib.Error e) {
            return new Json.Node (Json.NodeType.NULL);
        }
        return parser.get_root ();
    }

    public override Json.Node? server_response_result (Json.Object response) {
        string method = response.get_string_member ("method");
        if (method != "workspace/configuration") {
            return new Json.Node (Json.NodeType.NULL);
        }

        var items = response.get_object_member ("params").get_array_member ("items");
        var results = new Json.Array ();

        foreach (var item in items.get_elements ()) {
            var section = item.get_object ().get_string_member ("section");
            var settings = new Json.Object ();

            if (section == "basedpyright.analysis" || section == "python.analysis") {
                //            'analysis',
                settings.set_string_member ("diagnosticMode", "workspace");
            } else if (section == "basedpyright" || section == "pyright" || section == "pyth
                //          ,                 'analysis'
                var analysis_obj = new Json.Object ();
                analysis_obj.set_string_member ("diagnosticMode", "workspace");
                settings.set_object_member ("analysis", analysis_obj);
            } else {
                //                              ,
                settings = new Json.Object ();
            }

            results.add_object_element (settings);
        }

        var results_node = new Json.Node (Json.NodeType.ARRAY);
        results_node.set_array (results);
        return results_node;
    }
}
```

File: src/Services/LSP/DiagnosticsService.vala

```
public class Iide.DiagnosticsService : Object {
    private static DiagnosticsService? instance;

    //       : [ClientID] -> [FileURI] -> [         ]
    private Gee.HashMap<int, Gee.HashMap<string, Gee.List<IdeLspDiagnostic>>> server_map;
    private uint update_timeout_id = 0;

    public signal void diagnostics_updated (int client_id, string uri);
    public signal void total_count_changed (int total_errors, int total_warnings);

    public static DiagnosticsService get_instance () {
        if (instance == null)instance = new DiagnosticsService ();
        return instance;
    }

    private DiagnosticsService () {
        server_map = new Gee.HashMap<int, Gee.HashMap<string, Gee.List<IdeLspDiagnostic>>> (
    }

    /**
     *                    .
     *              Map,                  Entry (           ).
     */
    public Gee.Map<int, Gee.HashMap<string, Gee.List<IdeLspDiagnostic>>> get_server_map () {
        return server_map;
    }

    /**
     *                                ID.
     *                  "Clangd",     "14056232".
     */
    public string get_server_name (int client_id) {
        //                  LspService,
        var client = IdeLspService.get_instance ().get_client_by_hash (client_id);
        if (client != null) {
            return client.name (); //         ,     LspClient       name
        }
        return @"Unknown Server ($client_id)";
    }

    /**
     *                                      LSP-    .
     */
    public Gee.List<IdeLspDiagnostic>? get_diagnostics_for_file (int client_id, string uri)
```

```
    if (server_map.has_key (client_id)) {
        var client_files = server_map.get (client_id);
        if (client_files.has_key (uri)) {
            //            (  Vala Gee.List -           )
            return client_files.get (uri);
        }
    }
    return null;
}

public void update_diagnostics (int client_id, string uri, Gee.List<IdeLspDiagnostic> li
    if (!server_map.has_key (client_id)) {
        server_map.set (client_id, new Gee.HashMap<string, Gee.List<IdeLspDiagnostic>> (
    }

    var client_files = server_map.get (client_id);

    if (list.size == 0) {
        client_files.unset (uri);
    } else {
        client_files.set (uri, list);
    }

    diagnostics_updated (client_id, uri);

    //             emit_totals():
    if (update_timeout_id == 0) {
        update_timeout_id = Timeout.add (300, () => {
            emit_totals ();
            update_timeout_id = 0;
            return Source.REMOVE;
        });
    }
}

//
public void remove_client (int client_id) {
    if (server_map.unset (client_id)) {
        emit_totals ();
    }
}

private void emit_totals () {
    int e = 0; int w = 0;
    foreach (var client_files in server_map.values) {
        foreach (var list in client_files.values) {
```

```
                foreach (var d in list) {
                    if (d.severity == 1)e++;
                    else if (d.severity == 2)w++;
                }
            }
        }
        total_count_changed (e, w);
    }
}
```

File: src/Services/LSP/LspDiagnosticsMark.vala

```
public class Iide.LspDiagnosticsMark : GtkSource.Mark {
    public int severity { get; construct set; }
    public string diagnostic_message { get; construct set; }

    public LspDiagnosticsMark(string lsp_category, int lsp_severity, string lsp_message) {
        Object(name: null,
               category: lsp_category,
               severity: lsp_severity,
               diagnostic_message: lsp_message);
    }

    private static string category_from_severity(int severity) {
        switch (severity) {
        case 1:
            return "lsp_error";
        case 2:
            return "lsp_warning";
        case 3:
        case 4:
            return "lsp_info";
        }
        return "lsp_error";
    }

    public LspDiagnosticsMark.from_lsp_diagnostic(IdeLspDiagnostic diagnostics) {
        Object(name: null,
               category: category_from_severity(diagnostics.severity),
               severity: diagnostics.severity,
               diagnostic_message: diagnostics.message);
    }

    public static void set_mark_attributes(GtkSource.View text_view) {
        var error_mark_attrs = new GtkSource.MarkAttributes();
        var err_bg = Gdk.RGBA();
```

```
        err_bg.parse("#e01b2430");
        error_mark_attrs.set_background(err_bg);
        text_view.set_mark_attributes("lsp_error", error_mark_attrs, 100);

        var warning_mark_attrs = new GtkSource.MarkAttributes();
        var warn_bg = Gdk.RGBA();
        warn_bg.parse("#f5c21130");
        warning_mark_attrs.set_background(warn_bg);
        text_view.set_mark_attributes("lsp_warning", warning_mark_attrs, 90);

        var info_mark_attrs = new GtkSource.MarkAttributes();
        text_view.set_mark_attributes("lsp_info", info_mark_attrs, 80);
    }

    public static void clear_mark_attributes(GtkSource.View text_view) {
        var buffer = (GtkSource.Buffer) text_view.buffer;
        Gtk.TextIter start, end;
        buffer.get_start_iter(out start);
        buffer.get_end_iter(out end);

        buffer.remove_source_marks(start, end, "lsp_error");
        buffer.remove_source_marks(start, end, "lsp_warning");
        buffer.remove_source_marks(start, end, "lsp_info");
    }

    public string ? get_icon_name() {
        switch (severity) {
        case 1:
            return "dialog-error";
        case 2:
            return "dialog-warning";
        case 3:
        case 4:
            return "dialog-information";
        }
        return null;
    }
}
```

File: src/Services/LSP/LspTypes.vala

```
public enum Iide.CompletionTriggerKind {
    /**
     *                (     , Ctrl+Space)
     *               ,               .
     */
```

```
    INVOKED = 1,

    /**
     *                              -
     * (     , '.', ':', '->').
     */
    TRIGGER_CHARACTER = 2,

    /**
     *                       (     ,
     *               'incomplete').
     */
    TRIGGER_FOR_INCOMPLETE_COMPLETIONS = 3
}

public enum Iide.IdeLspCompletionKind {
    TEXT = 1,
    METHOD = 2,
    FUNCTION = 3,
    CONSTRUCTOR = 4,
    FIELD = 5,
    VARIABLE = 6,
    CLASS = 7,
    INTERFACE = 8,
    MODULE = 9,
    PROPERTY = 10,
    UNIT = 11,
    VALUE = 12,
    ENUM = 13,
    KEYWORD = 14,
    SNIPPET = 15,
    COLOR = 16,
    FILE = 17,
    REFERENCE = 18,
    FOLDER = 19,
    ENUM_MEMBER = 20,
    CONSTANT = 21,
    STRUCT = 22,
    EVENT = 23,
    OPERATOR = 24,
    TYPE_PARAMETER = 25
}

public class Iide.PendingChange : GLib.Object {
    public int start_offset;
    public int end_offset;
```

```
public string text;

//              LSP
public int start_line;
public int start_char;
public int end_line;
public int end_char;

public PendingChange (string t, Gtk.TextIter s_iter, Gtk.TextIter? e_iter = null) {
    this.text = t;
    this.start_offset = s_iter.get_offset ();
    this.end_offset = e_iter != null? e_iter.get_offset () : this.start_offset;

    //
    this.calculate_lsp_pos (s_iter, out this.start_line, out this.start_char);
    if (e_iter != null) {
        this.calculate_lsp_pos (e_iter, out this.end_line, out this.end_char);
    } else {
        this.end_line = this.start_line;
        this.end_char = this.start_char;
    }
}

private void calculate_lsp_pos (Gtk.TextIter iter, out int lsp_line, out int lsp_char) {
    lsp_line = iter.get_line ();

    //
    Gtk.TextIter line_start = iter;
    line_start.set_line_offset (0);

    //
    string line_text = line_start.get_text (iter);

    //      UTF-16 code units
    int utf16_count = 0;
    int i = 0;
    unichar c;
    while (line_text.get_next_char (ref i, out c)) {
        if (c <= 0xFFFF) {
            utf16_count += 1;
        } else {
            utf16_count += 2;
        }
    }
    lsp_char = utf16_count;
}
```

```
}

public class Iide.IdeLspCompletionItem : GLib.Object {
    public string label { get; set; default = ""; }
    public string? detail { get; set; }
    public string? documentation { get; set; }
    public int sort_text_priority { get; set; default = 0; }
    public int insert_text_priority { get; set; default = 0; }
    public string insert_text { get; set; default = ""; }
    public string? text_edit { get; set; }
    public int start_line { get; set; default = 0; }
    public int start_column { get; set; default = 0; }
    public int end_line { get; set; default = 0; }
    public int end_column { get; set; default = 0; }
    public IdeLspCompletionKind kind { get; set; default = IdeLspCompletionKind.TEXT; }
}

public class Iide.IdeLspDiagnostic : GLib.Object {
    public int severity { get; set; default = 1; }
    public string message { get; set; default = ""; }
    public int start_line { get; set; default = 0; }
    public int start_column { get; set; default = 0; }
    public int end_line { get; set; default = 0; }
    public int end_column { get; set; default = 0; }

    public string to_string () {
        return "Diagnostic: (%d:%d-%d:%d) %s".printf (start_line, start_column, end_line, en
    }
}

public class Iide.IdeLspLocation : GLib.Object {
    public string uri { get; set; }
    public int start_line { get; set; }
    public int start_column { get; set; }
    public int end_line { get; set; }
    public int end_column { get; set; }
}

public class Iide.IdeLspCompletionResult : GLib.Object {
    public Gee.ArrayList<IdeLspCompletionItem> items { get; set; }
    public bool is_incomplete { get; set; default = false; }
}
```

File: src/Services/TreeSitter/model/AstItem.vala

```
/*
```

```
 * Item for Abstract Syntax tree model
 *
 * Author: Alberto Fanjul <albfan@gnome.org>
 */
public class ASTItem : GLib.Object {
    public int start { get; set; }
    public int start_offset { get; set; }
    public int end { get; set; }
    public int end_offset { get; set; }
    public string node_type { get; set; }
    public bool fold { get; set; }
    public bool foldable { get; set; }

    public ASTItem(int start, int start_offset, int end, int end_offset,
        string node_type, bool fold) {
        this.start = start;
        this.start_offset = start_offset;
        this.end = end;
        this.end_offset = end_offset;
        this.node_type = node_type;
        this.fold = fold;
        this.foldable = false;
    }

    public bool contains_text_iter(Gtk.TextIter iter) {
        int line = iter.get_line();
        int offset = iter.get_line_offset();

        return (start < line || (start == line && start_offset <= offset))
               && (line < end || (line == end && offset <= end_offset));
    }
}
```

File: src/Services/TreeSitter/model/AstModel.vala

```
/*
 * Model for Abstract Syntax Tree
 *
 * Author: Alberto Fanjul <albfan@gnome.org>
 */
public class ASTModel : Gee.ArrayList<ASTItem> {

    public enum Style {
        INTELLIJ,
        ATOM;
```

```
        public string to_string() {
            switch (this) {
            case INTELLIJ:
                return "Intellij";
            case ATOM:
                return "Atom";
            default:
                assert_not_reached();
            }
        }
    }

    public Array<ASTItem> highlighted { get; set; default = new Array<ASTItem> (); }
    public bool show_symbols { get; set; }
    public string strategy { get; set; }
    public Style style { get; set; }
    public char whitespace { get; set; }
    public int whitespace_count { get; set; }

    public ASTModel() {
        whitespace = ' ';
        whitespace_count = 2;
    }

    public void add_item_from_iter(Gtk.TextIter start, Gtk.TextIter end,
                                   string type, bool fold) {
        add_item(start.get_line(), start.get_line_offset(), end.get_line(),
                 end.get_line_offset(), type, fold);
    }

    public void add_item(int start, int start_offset, int end, int end_offset,
                         string type, bool fold) {
        add(new ASTItem(start, start_offset, end, end_offset, type, fold));
    }

    public Array<ASTItem> get_folds(int line) {
        var result = new Array<ASTItem> ();
        foreach (ASTItem item in this) {
            if (line == item.start || (!item.fold && line == item.end)) {
                result.append_val(item);
            }
        }
        return result;
    }
}
```

File: src/Services/TreeSitter/python/higlights.scm

```
; Identifier naming conventions
(identifier) @variable

; Reset highlighting in f-string interpolations
(interpolation) @none

; Identifier naming conventions
((identifier) @type
  (#match? @type "^[A-Z].*[a-z]"))

((identifier) @constant
  (#match? @constant "^[A-Z][A-Z_0-9]*$"))

((identifier) @constant.builtin
  (#match? @constant.builtin "^__[a-zA-Z0-9_]*__$"))

((identifier) @constant.builtin
  (#any-of? @constant.builtin
    ; https://docs.python.org/3/library/constants.html
    "NotImplemented" "Ellipsis" "quit" "exit" "copyright" "credits" "license"))

"_" @character.special ; match wildcard

((assignment
  left: (identifier) @type.definition
  (type
    (identifier) @_annotation))
  (#eq? @_annotation "TypeAlias"))

((assignment
  left: (identifier) @type.definition
  right: (call
    function: (identifier) @_func))
  (#any-of? @_func "TypeVar" "NewType"))

; Function definitions
(function_definition
  name: (identifier) @function)

(type
  (identifier) @type)

(type
  (subscript
```

```
    (identifier) @type)) ; type subscript: Tuple[int]

((call
  function: (identifier) @_isinstance
  arguments: (argument_list
    (_)
    (identifier) @type))
  (#eq? @_isinstance "isinstance"))

; Literals
(none) @constant.builtin

[
  (true)
  (false)
] @boolean

(integer) @number

(float) @number.float

(comment) @comment @spell

((module
  .
  (comment) @keyword.directive @nospell)
  (#match? @keyword.directive "^#!/"))

(string) @string

[
  (escape_sequence)
  (escape_interpolation)
] @string.escape

; doc-strings
(expression_statement
  (string
    (string_content) @spell) @string.documentation)

; Tokens
[
  "-"
  "-="
  ":="
  "!="
```

```
  "*"
  "**"
  "**="
  "*="
  "/"
  "//"
  "//="
  "/="
  "&"
  "&="
  "%"
  "%="
  "^"
  "^="
  "+"
  "+="
  "<"
  "<<"
  "<<="
  "<="
  "<>"
  "="
  "=="
  ">"
  ">="
  ">>"
  ">>="
  "@"
  "@="
  "|"
  "|="
  "~"
  "->"
] @operator

; Keywords
[
  "and"
  "in"
  "is"
  "not"
  "or"
  "is not"
  "not in"
  "del"
] @keyword.operator
```

```
[
  "def"
  "lambda"
] @keyword.function

[
  "assert"
  "exec"
  "global"
  "nonlocal"
  "pass"
  "print"
  "with"
  "as"
] @keyword

[
  "type"
  "class"
] @keyword.type

[
  "async"
  "await"
] @keyword.coroutine

[
  "return"
  "yield"
] @keyword.return

(yield
  "from" @keyword.return)

(future_import_statement
  "from" @keyword.import
  "__future__" @module.builtin)

(import_from_statement
  "from" @keyword.import)

"import" @keyword.import

(aliased_import
  "as" @keyword.import)
```

```
(wildcard_import
  "*" @character.special)

(import_statement
  name: (dotted_name
    (identifier) @module))

(import_statement
  name: (aliased_import
    name: (dotted_name
      (identifier) @module)
    alias: (identifier) @module))

(import_from_statement
  module_name: (dotted_name
    (identifier) @module))

(import_from_statement
  module_name: (relative_import
    (dotted_name
      (identifier) @module)))

[
  "if"
  "elif"
  "else"
  "match"
  "case"
] @keyword.conditional

[
  "for"
  "while"
  "break"
  "continue"
] @keyword.repeat

[
  "try"
  "except"
  "raise"
  "finally"
] @keyword.exception

(raise_statement
```

```
  "from" @keyword.exception)

(try_statement
  (else_clause
    "else" @keyword.exception))

[
  "("
  ")"
  "["
  "]"
  "{"
  "}"
] @punctuation.bracket

(interpolation
  "{" @punctuation.special
  "}" @punctuation.special)

(type_conversion) @function.macro

[
  ","
  "."
  ":"
  ";"
  (ellipsis)
] @punctuation.delimiter

((identifier) @type.builtin
  (#any-of? @type.builtin
    ; https://docs.python.org/3/library/exceptions.html
    "BaseException" "Exception" "ArithmeticError" "BufferError" "LookupError" "AssertionErro
    "AttributeError" "EOFError" "FloatingPointError" "GeneratorExit" "ImportError"
    "ModuleNotFoundError" "IndexError" "KeyError" "KeyboardInterrupt" "MemoryError" "NameErr
    "NotImplementedError" "OSError" "OverflowError" "RecursionError" "ReferenceError" "Runti
    "StopIteration" "StopAsyncIteration" "SyntaxError" "IndentationError" "TabError" "System
    "SystemExit" "TypeError" "UnboundLocalError" "UnicodeError" "UnicodeEncodeError"
    "UnicodeDecodeError" "UnicodeTranslateError" "ValueError" "ZeroDivisionError" "Environme
    "IOError" "WindowsError" "BlockingIOError" "ChildProcessError" "ConnectionError"
    "BrokenPipeError" "ConnectionAbortedError" "ConnectionRefusedError" "ConnectionResetErro
    "FileExistsError" "FileNotFoundError" "InterruptedError" "IsADirectoryError"
    "NotADirectoryError" "PermissionError" "ProcessLookupError" "TimeoutError" "Warning"
    "UserWarning" "DeprecationWarning" "PendingDeprecationWarning" "SyntaxWarning" "RuntimeW
    "FutureWarning" "ImportWarning" "UnicodeWarning" "BytesWarning" "ResourceWarning"
    ; https://docs.python.org/3/library/stdtypes.html
```

```
    "bool" "int" "float" "complex" "list" "tuple" "range" "str" "bytes" "bytearray" "memoryv
    "set" "frozenset" "dict" "type" "object"))

; Normal parameters
(parameters
  (identifier) @variable.parameter)

; Lambda parameters
(lambda_parameters
  (identifier) @variable.parameter)

(lambda_parameters
  (tuple_pattern
    (identifier) @variable.parameter))

; Default parameters
(keyword_argument
  name: (identifier) @variable.parameter)

; Naming parameters on call-site
(default_parameter
  name: (identifier) @variable.parameter)

(typed_parameter
  (identifier) @variable.parameter)

(typed_default_parameter
  name: (identifier) @variable.parameter)

; Variadic parameters *args, **kwargs
(parameters
  (list_splat_pattern ; *args
    (identifier) @variable.parameter))

(parameters
  (dictionary_splat_pattern ; **kwargs
    (identifier) @variable.parameter))

; Typed variadic parameters
(parameters
  (typed_parameter
    (list_splat_pattern ; *args: type
      (identifier) @variable.parameter)))

(parameters
  (typed_parameter
```

```
    (dictionary_splat_pattern ; *kwargs: type
      (identifier) @variable.parameter)))

; Lambda parameters
(lambda_parameters
  (list_splat_pattern
    (identifier) @variable.parameter))

(lambda_parameters
  (dictionary_splat_pattern
    (identifier) @variable.parameter))

((identifier) @variable.builtin
  (#eq? @variable.builtin "self"))

((identifier) @variable.builtin
  (#eq? @variable.builtin "cls"))

; After @type.builtin bacause builtins (such as `type`) are valid as attribute name
((attribute
  attribute: (identifier) @variable.member)
  (#match? @variable.member "^[%l_].*$"))

; Class definitions
(class_definition
  name: (identifier) @type)

(class_definition
  body: (block
    (function_definition
      name: (identifier) @function.method)))

(class_definition
  superclasses: (argument_list
    (identifier) @type))

((class_definition
  body: (block
    (expression_statement
      (assignment
        left: (identifier) @variable.member))))
  (#match? @variable.member "^[%l_].*$"))

((class_definition
  body: (block
    (expression_statement
```

```
      (assignment
        left: (_
          (identifier) @variable.member)))))
  (#match? @variable.member "^[%l_].*$"))

((class_definition
  (block
    (function_definition
      name: (identifier) @constructor)))
  (#any-of? @constructor "__new__" "__init__"))

; Function calls
(call
  function: (identifier) @function.call)

(call
  function: (attribute
    attribute: (identifier) @function.method.call))

((call
  function: (identifier) @constructor)
  (#match? @constructor "^%u"))

((call
  function: (attribute
    attribute: (identifier) @constructor))
  (#match? @constructor "^%u"))

; Builtin functions
((call
  function: (identifier) @function.builtin)
  (#any-of? @function.builtin
    "abs" "all" "any" "ascii" "bin" "bool" "breakpoint" "bytearray" "bytes" "callable" "chr"
    "classmethod" "compile" "complex" "delattr" "dict" "dir" "divmod" "enumerate" "eval" "ex
    "filter" "float" "format" "frozenset" "getattr" "globals" "hasattr" "hash" "help" "hex"
    "input" "int" "isinstance" "issubclass" "iter" "len" "list" "locals" "map" "max" "memory
    "min" "next" "object" "oct" "open" "ord" "pow" "print" "property" "range" "repr" "revers
    "round" "set" "setattr" "slice" "sorted" "staticmethod" "str" "sum" "super" "tuple" "typ
    "vars" "zip" "__import__"))

; Regex from the `re` module
(call
  function: (attribute
    object: (identifier) @_re)
  arguments: (argument_list
    .
```

```
    (string
      (string_content) @string.regexp))
  (#eq? @_re "re"))

; Decorators
((decorator
  "@" @attribute)
  (#set! priority 101))

(decorator
  (identifier) @attribute)

(decorator
  (attribute
    attribute: (identifier) @attribute))

(decorator
  (call
    (identifier) @attribute))

(decorator
  (call
    (attribute
      attribute: (identifier) @attribute)))

((decorator
  (identifier) @attribute.builtin)
  (#any-of? @attribute.builtin "classmethod" "property" "staticmethod"))
```

File: src/Services/JsonUtils.vala

```
namespace Iide {
    public string json_node_to_string (Json.Node node) {
        var generator = new Json.Generator ();
        generator.set_root (node);
        return generator.to_data (null);
    }

    public string json_object_to_string (Json.Object? obj) {
        if (obj == null)return "<<null>>";
        var node = new Json.Node (Json.NodeType.OBJECT);
        node.set_object (obj);
        return json_node_to_string (node);
    }
}
```

File: src/Widgets/Panels/BasePanel.vala

```
public abstract class Iide.BasePanel : Panel.Widget {
    protected BasePanel (string title, string icon_name) {
        Object (title: title, icon_name: icon_name);
    }

    public abstract Panel.Position initial_pos ();
    public abstract string panel_id ();
}
```

File: src/Widgets/Panels/DiagnosticsPanel.vala

```
using Gee;
using Adw;

public class Iide.DiagnosticsRow : Adw.ActionRow {
    private IdeLspDiagnostic diagnostic;
    private string uri;

    public DiagnosticsRow (string uri, IdeLspDiagnostic diagnostic) {
        Object (
                title: diagnostic.message,
                subtitle: @"Line $(diagnostic.start_line + 1)",
                activatable: true
        );
        this.diagnostic = diagnostic;
        this.uri = uri;

        var icon = new Gtk.Image.from_icon_name (
                                                (diagnostic.severity == 1)
                                                ? "dialog-error-symbolic"
                                                : "dialog-warning-symbolic"
        );
        icon.add_css_class ((diagnostic.severity == 1) ? "error" : "warning");
        add_prefix (icon);
    }
}

public class Iide.FileRow : Adw.ExpanderRow {
    //     Box   ListBox
    private Gtk.ListBox content_list;
    private string uri;

    public FileRow (string uri) {
        Object (title: Path.get_basename (uri.replace ("file://", "")));
```

```
        this.uri = uri;

        //       ListBox              activated
        content_list = new Gtk.ListBox ();
        // content_list.add_css_class ("boxed-list"); //
        content_list.set_selection_mode (Gtk.SelectionMode.NONE);

        //       ListBox
        add_row (content_list);
    }

    public void update_rows (Gee.List<IdeLspDiagnostic>? diags) {
        //     ListBox
        Gtk.Widget? child;
        while ((child = content_list.get_first_child ()) != null) {
            content_list.remove (child);
        }

        subtitle = @"$(diags.size) issues";
        if (diags != null) {
            foreach (var diag in diags) {
                var row = new DiagnosticsRow (uri, diag);

                //       :
                row.activated.connect (() => {
                    //
                    var file = GLib.File.new_for_uri (uri);
                    DocumentManager.get_instance ().open_document_with_selection (file, diag
                });

                content_list.append (row);
            }
        }
    }
}

public class Iide.DiagnosticsPanel : BasePanel {
    private Gtk.ScrolledWindow scrolled;
    private Gtk.ListBox main_list;
    private Adw.StatusPage empty_page;

    //                          : [URI] -> ExpanderRow
    private HashMap<string, FileRow> file_rows = new HashMap<string, FileRow> ();
    //                 : [ClientID] -> Label
    private HashMap<int, Gtk.Widget> server_headers = new HashMap<int, Gtk.Widget> ();
```

```
public DiagnosticsPanel () {
    base ("Diagnostics", "dialog-error-symbolic");
    can_maximize = true;

    main_list = new Gtk.ListBox ();
    main_list.set_selection_mode (Gtk.SelectionMode.NONE);

    scrolled = new Gtk.ScrolledWindow () {
        vexpand = true,
        child = main_list
    };

    empty_page = new Adw.StatusPage () {
        title = "No issues found",
        icon_name = "emblem-ok-symbolic",
        vexpand = true
    };

    this.set_child (empty_page);

    var service = DiagnosticsService.get_instance ();
    //
    service.diagnostics_updated.connect (on_file_diagnostics_updated);
    service.total_count_changed.connect (on_total_changed);
}

public override Panel.Position initial_pos () {
    return new Panel.Position () { area = Panel.Area.BOTTOM };
}

public override string panel_id () {
    return "DiagnosticsPanel";
}

private void on_total_changed (int e, int w) {
    //        "         "
    if (e == 0 && w == 0) {
        if (this.get_child () != empty_page)this.set_child (empty_page);
    } else {
        if (this.get_child () != scrolled)this.set_child (scrolled);
    }
}

//
private void on_file_diagnostics_updated (int client_id, string uri) {
    var service = DiagnosticsService.get_instance ();
```

```
        var diags = service.get_diagnostics_for_file (client_id, uri);

        // 1.                      -
        if (diags == null || diags.size == 0) {
            if (file_rows.has_key (uri)) {
                var row = file_rows.get (uri);
                main_list.remove (row);
                file_rows.unset (uri);
            }
            return;
        }

        // 2.      ,
        ensure_server_header (client_id);

        // 3.
        FileRow file_row;
        if (file_rows.has_key (uri)) {
            file_row = file_rows.get (uri);
        } else {
            file_row = new FileRow (uri);
            main_list.append (file_row);
            file_rows.set (uri, file_row);
        }

        file_row.update_rows (diags);
    }

    private void ensure_server_header (int client_id) {
        if (!server_headers.has_key (client_id)) {
            var service = DiagnosticsService.get_instance ();
            var header = new Gtk.Label (service.get_server_name (client_id)) {
                xalign = 0, margin_start = 12, margin_top = 12, margin_bottom = 6
            };
            header.add_css_class ("heading");
            main_list.append (header);
            server_headers.set (client_id, header);
        }
    }
}
```

File: src/Widgets/Panels/LogPanel.vala

```
public class Iide.LogPanel : BasePanel {
    public LogPanel () {
        base ("Logs", "document-properties-symbolic");
```

```
        child = new LogView ();
        can_maximize = true;
    }

    public override Panel.Position initial_pos () {
        return new Panel.Position () { area = Panel.Area.BOTTOM };
    }

    public override string panel_id () {
        return "LogPanel";
    }
}
```

File: src/Widgets/Panels/TerminalPanel.vala

```
public class Iide.TerminalPanel : BasePanel {
    public TerminalPanel () {
        base ("Terminal", "utilities-terminal-symbolic");
        child = new Iide.Terminal ();
        can_maximize = true;
    }

    public override Panel.Position initial_pos () {
        return new Panel.Position () { area = Panel.Area.BOTTOM };
    }

    public override string panel_id () {
        return "TerminalPanel";
    }
}
```

File: src/Widgets/TextView/DiagnosticsPopover.vala

```
public class Iide.DiagnosticsPopover : Gtk.Popover {
    private Gtk.ListBox list_box;
    private GtkSource.View text_view;

    public DiagnosticsPopover (Gtk.Widget parent, GtkSource.View view) {
        this.set_parent (parent);
        this.text_view = view;

        //
        this.set_position (Gtk.PositionType.TOP); //                    -
        this.set_has_arrow (true);
        this.set_autohide (true);
```

```
var scroll = new Gtk.ScrolledWindow () {
    max_content_height = 400,
    propagate_natural_height = true,
    width_request = 400
};

list_box = new Gtk.ListBox ();
list_box.add_css_class ("boxed-list");
list_box.set_selection_mode (Gtk.SelectionMode.NONE); //              ,

scroll.set_child (list_box);
this.set_child (scroll);

list_box.row_activated.connect ((row) => {
    var diag_row = row as Adw.ActionRow;
    if (diag_row != null) {
        //                      ( .   )
        Gtk.TextIter start_line, end_line;

        // 1.
        var buffer = (GtkSource.Buffer) text_view.buffer;
        var mark = diag_row.get_data<LspDiagnosticsMark> ("mark");
        if (mark == null) {
            return;
        }
        buffer.get_iter_at_mark (out start_line, mark);

        // 2.
        start_line.set_line_offset (0);

        // 3.
        end_line = start_line;
        if (!end_line.ends_line ()) {
            end_line.forward_to_line_end ();
        }

        // 4.
        //  GTK select_range(   ,     )
        buffer.select_range (start_line, end_line);

        // 5.
        text_view.scroll_to_iter (start_line, 0.1, false, 0, 0.5);
        text_view.grab_focus ();

        //
        this.popdown ();
```

```
        }
    });
}

public void refresh () {
    // 1.
    Gtk.Widget? child;
    while ((child = list_box.get_first_child ()) != null) {
        list_box.remove (child);
    }

    var buffer = (GtkSource.Buffer) text_view.buffer;
    bool found_any = false;

    // 2.
    int line_count = buffer.get_line_count ();
    for (int i = 0; i < line_count; i++) {
        Gtk.TextIter iter;
        buffer.get_iter_at_line (out iter, i);

        var iter_marks = iter.get_marks ();

        //
        if (iter_marks != null) {
            foreach (var m in iter_marks) {
                if (m is LspDiagnosticsMark) {
                    add_diagnostic_row ((LspDiagnosticsMark) m);
                    found_any = true;
                }
            }
        }
    }

    // 3.           "     "
    if (!found_any) {
        var status_page = new Adw.StatusPage () {
            title = "No issues found",
            icon_name = "emblem-ok-symbolic",
            description = "LSP servers haven't reported any problems."
        };
        status_page.add_css_class ("compact"); //           libadwaita 1.2+
        list_box.append (status_page);
    }
}

private void add_diagnostic_row (LspDiagnosticsMark mark) {
```

```
Gtk.TextIter iter;
var buffer = (GtkSource.Buffer) text_view.buffer;
buffer.get_iter_at_mark (out iter, mark);
int line = iter.get_line () + 1;

var row = new Adw.ActionRow () {
    title = mark.diagnostic_message,
    subtitle = @"Line $line",
    activatable = true, //
    selectable = false
};
row.set_data ("mark", mark);

//                 (    )
var icon_name = mark.get_icon_name ();
if (icon_name != null) {
    var img = new Gtk.Image.from_icon_name (icon_name + "-symbolic");
    img.add_css_class (mark.category); //                   CSS
    row.add_prefix (img);
}

//             -
row.activated.connect (() => {
    Gtk.TextIter start_line, end_line;

    // 1.
    buffer.get_iter_at_mark (out start_line, mark);

    // 2.
    start_line.set_line_offset (0);

    // 3.
    end_line = start_line;
    if (!end_line.ends_line ()) {
        end_line.forward_to_line_end ();
    }

    // 4.
    //  GTK select_range(    ,     )
    buffer.select_range (start_line, end_line);

    // 5.
    text_view.scroll_to_iter (start_line, 0.1, false, 0, 0.5);
    text_view.grab_focus ();

    //
```

```
            this.popdown ();
        });

        list_box.append (row);
    }
}
```

File: src/Widgets/ToolViews/TerminalView.vala

```
/*
 * SPDX-License-Identifier: LGPL-3.0-or-later
 * SPDX-FileCopyrightText: 2022 elementary, Inc. (https://elementary.io)
 *                         2011-2013 Mario Guerriero <mario@elementaryos.org>
 */

public class Iide.Terminal : Gtk.Box {
    private const string ACTION_GROUP = "term";
    private const string ACTION_PREFIX = ACTION_GROUP + ".";
    private const string ACTION_COPY = "action-copy";
    private const string ACTION_PASTE = "action-paste";
    private const string ACTION_INCREASE_FONT = "increase-font";
    private const string ACTION_DECREASE_FONT = "decrease-font";
    private const string ACTION_RESET_FONT = "reset-font";

    private const double MAX_SCALE = 5.0;
    private const double MIN_SCALE = 0.2;
    private const string LEGACY_SETTINGS_SCHEMA = "org.pantheon.terminal.settings";
    private const string SETTINGS_SCHEMA = "io.elementary.terminal.settings";
    private const string GNOME_DESKTOP_INTERFACE_SCHEMA = "org.gnome.desktop.interface";
    private const string GNOME_DESKTOP_WM_PREFERENCES_SCHEMA = "org.gnome.desktop.wm.prefere
    private const string TERMINAL_FONT_KEY = "font";
    private const string TERMINAL_BELL_KEY = "audible-bell";
    private const string TERMINAL_CURSOR_KEY = "cursor-shape";
    private const string TERMINAL_FOREGROUND_KEY = "foreground";
    private const string TERMINAL_BACKGROUND_KEY = "background";
    private const string TERMINAL_PALETTE_KEY = "palette";
    private const string GNOME_FONT_KEY = "monospace-font-name";
    private const string GNOME_BELL_KEY = "audible-bell";

    public Vte.Terminal terminal { get; construct; }
    private Gtk.EventControllerKey key_controller;
    private Gtk.GestureClick gesture_click;
    private Gtk.EventControllerScroll scroll_controller;
    private Gdk.Clipboard current_clipboard;
    private Gtk.ScrolledWindow scrolled_window;
```

```
private const double FONT_SCALE_STEP = 0.1;

private Settings? terminal_settings = null;
private Settings? gnome_interface_settings = null;
private Settings? gnome_wm_settings = null;

public SimpleActionGroup actions { get; construct; }

private GLib.Pid child_pid;
// private Gtk.Clipboard current_clipboard;

construct {
    terminal = new Vte.Terminal () {
        hexpand = true,
        vexpand = true,
        scrollback_lines = -1,
        cursor_blink_mode = SYSTEM // There is no Terminal setting so follow Gnome
    };

    // Set font, allow-bold, audible-bell, background, foreground, and palette of panthe
    var schema_source = SettingsSchemaSource.get_default ();
    var terminal_schema = schema_source.lookup (SETTINGS_SCHEMA, true);
    if (terminal_schema == null) {
        terminal_schema = schema_source.lookup (LEGACY_SETTINGS_SCHEMA, true);
    }

    if (terminal_schema != null) {
        terminal_settings = new Settings.full (terminal_schema, null, null);
        terminal_settings.changed.connect ((key) => {
            switch (key) {
                case TERMINAL_FONT_KEY :
                    update_font ();
                    break;
                case TERMINAL_BELL_KEY :
                    update_audible_bell ();
                    break;
                case TERMINAL_CURSOR_KEY :
                    update_cursor ();
                    break;
                case TERMINAL_FOREGROUND_KEY:
                case TERMINAL_BACKGROUND_KEY:
                case TERMINAL_PALETTE_KEY:
                    update_colors ();
                    break;
                default:
                    // TODO Handle other relevant terminal settings?
```

```
                // "theme"
                // "prefer-dark-style"
                break;
        }
    });
} else {
    var gnome_wm_settings_schema = schema_source.lookup (GNOME_DESKTOP_WM_PREFERENCE
    if (gnome_wm_settings_schema != null) {
        gnome_wm_settings = new Settings.full (gnome_wm_settings_schema, null, null)
        gnome_wm_settings.changed.connect ((key) => {
            switch (key) {
                case GNOME_BELL_KEY:
                    update_audible_bell ();
                    break;
                default:
                    break;
            }
        });
    }
    // TODO monitor changes in relevant system settings?
    // "org.gnome.desktop.interface.color-scheme"
}

// Always monitor changes in default font as that is what Terminal usually follows
var gnome_interface_settings_schema = schema_source.lookup (GNOME_DESKTOP_INTERFACE_
if (gnome_interface_settings_schema != null) {
    gnome_interface_settings = new Settings.full (gnome_interface_settings_schema, n
    gnome_interface_settings.changed.connect ((key) => {
        switch (key) {
            case GNOME_FONT_KEY:
                update_font ();
                break;
            default:
                break;
        }
    });
}

update_font ();
update_audible_bell ();
update_cursor ();
update_colors ();

var adw_style_manager = Adw.StyleManager.get_default ();
adw_style_manager.notify["color-scheme"].connect (() => {
    update_colors ();
```

```
});

terminal.child_exited.connect (() => {
    // Hide the exited terminal
    // var win_group = get_action_group (Scratch.MainWindow.ACTION_GROUP);
    // win_group.activate_action (Scratch.MainWindow.ACTION_TOGGLE_TERMINAL, null);
    //// Get ready to resume at last saved location
    // spawn_shell (Scratch.saved_state.get_string ("last-opened-path"));
    //// Clear screen of new shell
    terminal.feed_child ("clear -x\n".data);
});

var copy_action = new SimpleAction (ACTION_COPY, null);
copy_action.set_enabled (false);
copy_action.activate.connect (() => terminal.copy_clipboard ());

var paste_action = new SimpleAction (ACTION_PASTE, null);
paste_action.activate.connect (() => terminal.paste_clipboard ());

var increase_font_action = new SimpleAction (ACTION_INCREASE_FONT, null);
increase_font_action.activate.connect (() => increment_size ());

var decrease_font_action = new SimpleAction (ACTION_DECREASE_FONT, null);
decrease_font_action.activate.connect (() => decrement_size ());

var reset_font_action = new SimpleAction (ACTION_RESET_FONT, null);
reset_font_action.activate.connect (() => set_default_font_size ());

actions = new SimpleActionGroup ();
actions.add_action (copy_action);
actions.add_action (paste_action);
actions.add_action (increase_font_action);
actions.add_action (decrease_font_action);
actions.add_action (reset_font_action);

var menu_model = new GLib.Menu ();
var clipboard_section = new GLib.Menu ();
clipboard_section.append (_("Copy"), ACTION_PREFIX + ACTION_COPY);
clipboard_section.append (_("Paste"), ACTION_PREFIX + ACTION_PASTE);
menu_model.append_section (null, clipboard_section);
var zoom_section = new GLib.Menu ();
zoom_section.append (_("Zoom In"), ACTION_PREFIX + ACTION_INCREASE_FONT);
zoom_section.append (_("Zoom Out"), ACTION_PREFIX + ACTION_DECREASE_FONT);
zoom_section.append (_("Reset Zoom"), ACTION_PREFIX + ACTION_RESET_FONT);
menu_model.append_section (null, zoom_section);
```

```
scrolled_window = new Gtk.ScrolledWindow ();
scrolled_window.hexpand = true;
scrolled_window.set_child (terminal);

var popover_menu = new Gtk.PopoverMenu.from_model (menu_model);
popover_menu.insert_action_group (ACTION_GROUP, actions);
popover_menu.set_parent (scrolled_window);
// menu.show_all ();

key_controller = new Gtk.EventControllerKey ();
key_controller.propagation_phase = BUBBLE;
terminal.add_controller (key_controller);
key_controller.key_pressed.connect (key_pressed);

gesture_click = new Gtk.GestureClick ();
gesture_click.button = 3; // Right click
scrolled_window.add_controller (gesture_click);
gesture_click.pressed.connect ((n_press, x, y) => {
    if (n_press == 1) {
        paste_action.set_enabled (current_clipboard.content != null);
        var rect = Gdk.Rectangle () { x = (int) x, y = (int) y, width = 1, height =
        popover_menu.set_pointing_to (rect);
        popover_menu.popup ();
    }
});

scroll_controller = new Gtk.EventControllerScroll (Gtk.EventControllerScrollFlags.VE
scroll_controller.propagation_phase = CAPTURE;
scrolled_window.add_controller (scroll_controller);
scroll_controller.scroll.connect ((dx, dy) => {
    var state = scroll_controller.get_current_event_state ();
    if (Gdk.ModifierType.CONTROL_MASK in state) {
        if (dy > 0) {
            decrement_size ();
        } else {
            increment_size ();
        }
        return Gdk.EVENT_STOP;
    }
    return Gdk.EVENT_PROPAGATE;
});

// terminal.button_press_event.connect ((event) => {
// if (event.button == 3) {
// paste_action.set_enabled (current_clipboard.wait_is_text_available ());
// menu.select_first (false);
```

```
    // menu.popup_at_pointer (event);
    // }
    // return false;
    // });

    // realize.connect (() => {
    // current_clipboard = terminal.get_clipboard (Gdk.SELECTION_CLIPBOARD);
    // copy_action.set_enabled (terminal.get_has_selection ());
    // });

    spawn_shell ();

    terminal.selection_changed.connect (() => {
        copy_action.set_enabled (terminal.get_has_selection ());
    });

    realize.connect (() => {
        current_clipboard = scrolled_window.get_clipboard ();
        copy_action.set_enabled (terminal.get_has_selection ());
    });

    append (scrolled_window);
}

private void spawn_shell (string dir = GLib.Environment.get_current_dir ()) {
    terminal.spawn_async (
                          Vte.PtyFlags.DEFAULT,
                          dir,
                          { Vte.get_user_shell () },
                          null,
                          SpawnFlags.SEARCH_PATH,
                          null,
                          100,
                          null,
                          (source, pid, error) => {
        if (error != null) {
            stderr.printf ("           : %s\n", error.message);
        } else {
            this.child_pid = pid;
            stdout.printf ("              . PID: %d\n", pid);
        }
    });
}

// public void change_location (string dir) {
// Posix.kill (child_pid, Posix.Signal.TERM);
```

```
// terminal.reset (true, true);
// spawn_shell (dir);
// Scratch.saved_state.set_string ("last-opened-path", dir);
// }

// private string get_shell_location () {
// int pid = (!) (this.child_pid);
// try {
// return GLib.FileUtils.read_link ("/proc/%d/cwd".printf (pid));
// } catch (GLib.FileError error) {
// warning ("An error occurred while fetching the current dir of shell: %s", error.messa
// return "";
// }
// }

private void update_font () {
    var font_name = "";
    if (terminal_settings != null) {
        font_name = terminal_settings.get_string (TERMINAL_FONT_KEY);
    }

    if (font_name == "" && gnome_interface_settings != null) {
        font_name = gnome_interface_settings.get_string (GNOME_FONT_KEY);
    }

    var fd = Pango.FontDescription.from_string (font_name);
    terminal.set_font (fd);
}

private void update_audible_bell () {
    var audible_bell = false;
    if (terminal_settings != null) {
        audible_bell = terminal_settings.get_boolean (TERMINAL_BELL_KEY);
    } else if (gnome_wm_settings != null) {
        audible_bell = gnome_wm_settings.get_boolean (GNOME_BELL_KEY);
    }

    terminal.set_audible_bell (audible_bell);
}

private void update_cursor () {
    if (terminal_settings != null) {
        var cursor_shape_setting = terminal_settings.get_string (TERMINAL_CURSOR_KEY);
        switch (cursor_shape_setting) {
        case "Block":
            terminal.cursor_shape = Vte.CursorShape.BLOCK;
```

```
            break;
        case "I-Beam":
            terminal.cursor_shape = Vte.CursorShape.IBEAM;
            break;
        case "Underline":
            terminal.cursor_shape = Vte.CursorShape.UNDERLINE;
            break;
        }
    } // No corresponding system keymap
}

private void update_colors () {
    if (terminal_settings != null) {
        string background_setting = terminal_settings.get_string (TERMINAL_BACKGROUND_KE
        Gdk.RGBA background_color = Gdk.RGBA ();
        background_color.parse (background_setting);

        string foreground_setting = terminal_settings.get_string (TERMINAL_FOREGROUND_KE
        Gdk.RGBA foreground_color = Gdk.RGBA ();
        foreground_color.parse (foreground_setting);

        string palette_setting = terminal_settings.get_string (TERMINAL_PALETTE_KEY);

        string[] hex_palette = { "#000000", "#FF6C60", "#A8FF60", "#FFFFCC", "#96CBFE",
                                 "#FF73FE", "#C6C5FE", "#EEEEEE", "#000000", "#FF6C60",
                                 "#A8FF60", "#FFFFB6", "#96CBFE", "#FF73FE", "#C6C5FE",
                                 "#EEEEEE" };

        string current_string = "";
        int current_color = 0;
        for (var i = 0; i < palette_setting.length; i++) {
            if (palette_setting[i] == ':') {
                hex_palette[current_color] = current_string;
                current_string = "";
                current_color++;
            } else {
                current_string += palette_setting[i].to_string ();
            }
        }

        Gdk.RGBA[] palette = new Gdk.RGBA[16];

        for (int i = 0; i < hex_palette.length; i++) {
            Gdk.RGBA new_color = Gdk.RGBA ();
            new_color.parse (hex_palette[i]);
            palette[i] = new_color;
```

```
        }

        terminal.set_colors (foreground_color, background_color, palette);
    } else {
        update_colors_for_theme (Adw.StyleManager.get_default ().color_scheme);
    }
}

private void update_colors_for_theme (Adw.ColorScheme scheme) {
    Gdk.RGBA foreground_color;
    Gdk.RGBA background_color;
    string[] hex_palette = { "#000000", "#cc0000", "#4e9a06", "#c4a000", "#3465a4", "#75

    Gdk.RGBA[] palette = new Gdk.RGBA[16];

    for (int i = 0; i < hex_palette.length; i++) {
        Gdk.RGBA new_color = Gdk.RGBA ();
        new_color.parse (hex_palette[i]);
        palette[i] = new_color;
    }

    if (scheme == Adw.ColorScheme.FORCE_LIGHT) {
        foreground_color = { 0.0f, 0.0f, 0.0f, 1.0f };
        background_color = { 1.0f, 1.0f, 1.0f, 1.0f };
    } else {
        foreground_color = { 1.0f, 1.0f, 1.0f, 1.0f };
        background_color = { 0.0f, 0.0f, 0.0f, 1.0f };
    }

    terminal.set_colors (foreground_color, background_color, palette);
}

public void save_settings () {
    // Scratch.saved_state.set_string ("last-opened-path", get_shell_location ());
}

private void increment_size () {
    terminal.font_scale = (terminal.font_scale + FONT_SCALE_STEP).clamp (MIN_SCALE, MAX_
}

private void decrement_size () {
    terminal.font_scale = (terminal.font_scale - FONT_SCALE_STEP).clamp (MIN_SCALE, MAX_
}

public void set_default_font_size () {
    terminal.font_scale = 1.0;
```

```
    }

    private bool key_pressed (uint keyval, uint keycode, Gdk.ModifierType modifiers) {
        if (Gdk.ModifierType.CONTROL_MASK in modifiers && keyval == Gdk.Key.d) {
            terminal.reset (true, true);
            return Gdk.EVENT_STOP;
        }

        if (Gdk.ModifierType.CONTROL_MASK in modifiers &&
            (Gdk.ModifierType.SHIFT_MASK in modifiers ||
             terminal_settings != null && terminal_settings.get_boolean ("natural-copy-paste

            if (keyval == Gdk.Key.c && terminal.get_has_selection ()) {
                actions.activate_action (ACTION_COPY, null);
                return Gdk.EVENT_STOP;
                // } else if (keyval == Gdk.Key.v && current_clipboard.wait_is_text_availabl
                // actions.activate_action (ACTION_PASTE, null);
                // return Gdk.EVENT_STOP;
            }
        }

        return Gdk.EVENT_PROPAGATE;
    }
}
```

File: test_temp/test.c

```
int main() {
```

File: src/Widgets/Panels/ProjectPanel.vala

```
public class Iide.ProjectPanel : BasePanel {
    public FileTreeView folder_view;

    public ProjectPanel () {
        base ("Project Tree", "folder-symbolic");
        folder_view = new FileTreeView ();
        child = folder_view;
        can_maximize = true;

        //
        var project_manager = ProjectManager.get_instance ();
        project_manager.project_opened.connect ((project_root) => {
            folder_view.set_root_file (project_root);
        });
```

```
        project_manager.project_closed.connect (() => {
            folder_view.set_root_file (null);
        });

        // Handle file activation to open documents
        var document_manager = DocumentManager.get_instance ();
        folder_view.file_activated.connect ((item) => {
            if (!item.is_directory) {
                document_manager.open_document (item.file, null);
            }
        });
    }

    public override Panel.Position initial_pos () {
        return new Panel.Position () { area = Panel.Area.START };
    }

    public override string panel_id () {
        return "ProjectPanel";
    }
}
```

File: src/Widgets/ToolViews/LogView.vala

```
/*
 * logview.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

public class Iide.LogView : Gtk.Box {
```

```
private Gtk.TextView text_view;
private Gtk.TextBuffer buffer;
private Gtk.ScrolledWindow scrolled_window;
private Gtk.Button clear_button;
private Gtk.ToggleButton wrap_button;
private Gtk.DropDown level_filter;
private Gtk.SearchEntry search_entry;
private Iide.LoggerService logger;

private Gtk.TextTag tag_debug;
private Gtk.TextTag tag_info;
private Gtk.TextTag tag_warning;
private Gtk.TextTag tag_error;
private Gtk.TextTag tag_critical;
private Gtk.TextTag tag_bold;
private Gtk.TextTag tag_dim;

private GLib.Mutex buffer_lock;
private bool auto_scroll = true;
private const int MAX_VISIBLE_LINES = 5000;
private int line_count = 0;

public LogView () {
    Object (orientation: Gtk.Orientation.VERTICAL, spacing: 0);
}

construct {
    logger = Iide.LoggerService.get_instance ();

    var toolbar = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
    toolbar.add_css_class ("toolbar");
    toolbar.margin_start = 6;
    toolbar.margin_end = 6;
    toolbar.margin_top = 4;
    toolbar.margin_bottom = 4;

    clear_button = new Gtk.Button () {
        icon_name = "edit-clear-symbolic",
        tooltip_text = _("Clear logs")
    };
    clear_button.clicked.connect (on_clear_clicked);

    wrap_button = new Gtk.ToggleButton () {
        icon_name = "preferences-desktop-text-wrap-symbolic",
        tooltip_text = _("Word wrap")
    };
```

```
wrap_button.active = true;
wrap_button.toggled.connect (on_wrap_toggled);

var level_label = new Gtk.Label (_("Level:")) {
    margin_start = 12
};

var level_list = new Gtk.StringList ({
    _("All"),
    "DEBUG",
    "INFO",
    "WARN",
    "ERROR",
    "CRITICAL"
});
level_filter = new Gtk.DropDown (level_list, null);
level_filter.selected = 0;
level_filter.notify["selected"].connect (on_level_changed);

search_entry = new Gtk.SearchEntry () {
    placeholder_text = _("Search logs..."),
    hexpand = true
};
search_entry.search_changed.connect (on_search_changed);

toolbar.append (clear_button);
toolbar.append (wrap_button);
toolbar.append (new Gtk.Separator (Gtk.Orientation.VERTICAL));
toolbar.append (level_label);
toolbar.append (level_filter);
toolbar.append (search_entry);

buffer = new Gtk.TextBuffer (null);
tag_debug = buffer.create_tag (null, foreground: "#888888");
tag_info = buffer.create_tag (null, foreground: "#4a9eff", weight: Pango.Weight.BOLD
tag_warning = buffer.create_tag (null, foreground: "#f5a623", weight: Pango.Weight.B
tag_error = buffer.create_tag (null, foreground: "#e74c3c", weight: Pango.Weight.BOL
tag_critical = buffer.create_tag (null, foreground: "#ff4757", weight: Pango.Weight.
tag_bold = buffer.create_tag (null, weight: Pango.Weight.BOLD);
tag_dim = buffer.create_tag (null, foreground: "#666666", style: Pango.Style.ITALIC)

text_view = new Gtk.TextView () {
    buffer = buffer,
    editable = false,
    monospace = true,
    wrap_mode = Gtk.WrapMode.WORD_CHAR,
```

```
        hscroll_policy = Gtk.ScrollablePolicy.NATURAL,
        vscroll_policy = Gtk.ScrollablePolicy.NATURAL
    };
    text_view.add_css_class ("log-view");

    scrolled_window = new Gtk.ScrolledWindow () {
        child = text_view,
        hexpand = true,
        vexpand = true
    };

    append (toolbar);
    append (scrolled_window);

    logger.log_added.connect (on_log_added);
    logger.log_cleared.connect (on_log_cleared);

    foreach (var entry in logger.get_entries ()) {
        append_entry (entry);
    }
}

private void on_log_added (Iide.LogEntry entry) {
    append_entry (entry);
}

private void append_entry (Iide.LogEntry entry) {
    buffer_lock.lock ();
    try {
        var timestamp = entry.get_timestamp_string ();
        var level_str = entry.level.to_string ();
        var domain = entry.domain;
        var message = entry.message;

        Gtk.TextTag tag;
        switch (entry.level) {
        case Iide.LogLevel.DEBUG:
            tag = tag_debug;
            break;
        case Iide.LogLevel.INFO:
            tag = tag_info;
            break;
        case Iide.LogLevel.WARNING:
            tag = tag_warning;
            break;
        case Iide.LogLevel.ERROR:
```

```
    tag = tag_error;
    break;
case Iide.LogLevel.CRITICAL:
    tag = tag_critical;
    break;
default:
    tag = tag_dim;
    break;
}

var ts_text = "[%s] ".printf (timestamp);
var level_text = "[%s] ".printf (level_str);
var domain_text = "[%s] ".printf (domain);

int offset = buffer.get_char_count ();

Gtk.TextIter iter;
buffer.get_end_iter (out iter);
buffer.insert (ref iter, ts_text, ts_text.length);
buffer.get_end_iter (out iter);
buffer.insert (ref iter, level_text, level_text.length);
buffer.get_end_iter (out iter);
buffer.insert (ref iter, domain_text, domain_text.length);
buffer.get_end_iter (out iter);
buffer.insert (ref iter, message, message.length);

if (entry.details != null) {
    buffer.get_end_iter (out iter);
    buffer.insert (ref iter, "\n  ", 3);
    buffer.get_end_iter (out iter);
    buffer.insert (ref iter, entry.details, entry.details.length);
}

buffer.get_end_iter (out iter);
buffer.insert (ref iter, "\n", 1);

Gtk.TextIter start, end;
buffer.get_iter_at_offset (out start, offset);
buffer.get_iter_at_offset (out end, offset + (int) ts_text.length);
buffer.apply_tag (tag_dim, start, end);

buffer.get_iter_at_offset (out start, offset + (int) ts_text.length);
buffer.get_iter_at_offset (out end, offset + (int) ts_text.length + (int) level_
buffer.apply_tag (tag, start, end);

buffer.get_iter_at_offset (out start, offset + (int) ts_text.length + (int) leve
```

```
        buffer.get_iter_at_offset (out end, offset + (int) ts_text.length + (int) level_
        buffer.apply_tag (tag_bold, start, end);

        if (entry.details != null) {
            int detail_start = offset + (int) ts_text.length + (int) level_text.length +
            buffer.get_iter_at_offset (out start, detail_start);
            buffer.get_iter_at_offset (out end, detail_start + 3);
            buffer.apply_tag (tag_dim, start, end);
        }

        line_count++;
        if (auto_scroll) {
            scroll_to_end ();
        }

        trim_buffer ();
    } finally {
        buffer_lock.unlock ();
    }
}

private void scroll_to_end () {
    Gtk.TextIter end;
    buffer.get_end_iter (out end);
    text_view.scroll_to_iter (end, 0.0, true, 0.0, 1.0);
}

private void trim_buffer () {
    if (line_count > MAX_VISIBLE_LINES) {
        buffer_lock.lock ();
        try {
            Gtk.TextIter start;
            buffer.get_iter_at_line (out start, line_count - MAX_VISIBLE_LINES);
            Gtk.TextIter end = start;
            buffer.get_iter_at_line (out end, line_count - MAX_VISIBLE_LINES + 100);
            buffer.delete (ref start, ref end);
            line_count -= 100;
        } finally {
            buffer_lock.unlock ();
        }
    }
}

private void on_clear_clicked () {
    logger.clear ();
}
```

```
private void on_log_cleared () {
    buffer.set_text ("");
    line_count = 0;
}

private void on_wrap_toggled () {
    text_view.wrap_mode = wrap_button.active ? Gtk.WrapMode.WORD_CHAR : Gtk.WrapMode.NON
}

private void on_level_changed () {
    rebuild_log_view ();
}

private void on_search_changed () {
    rebuild_log_view ();
}

private bool matches_search (Iide.LogEntry entry, string search_text) {
    if (search_text == "") {
        return true;
    }
    var lower_search = search_text.down ();
    return entry.message.down ().contains (lower_search) ||
           entry.domain.down ().contains (lower_search) ||
           (entry.details != null && entry.details.down ().contains (lower_search));
}

private void rebuild_log_view () {
    buffer_lock.lock ();
    try {
        buffer.set_text ("");
        line_count = 0;

        var filter_level = (int) level_filter.selected;
        var search_text = search_entry.text;

        foreach (var entry in logger.get_entries ()) {
            if (filter_level > 0) {
                var entry_level = (int) entry.level + 1;
                if (entry_level != filter_level) {
                    continue;
                }
            }
            if (!matches_search (entry, search_text)) {
                continue;
```

```
            }
            append_entry_unlocked (entry);
        }
    } finally {
        buffer_lock.unlock ();
    }
}

private void append_entry_unlocked (Iide.LogEntry entry) {
    Gtk.TextIter end;
    buffer.get_end_iter (out end);

    var timestamp = entry.get_timestamp_string ();
    var level_str = entry.level.to_string ();
    var domain = entry.domain;
    var message = entry.message;

    Gtk.TextTag tag;
    switch (entry.level) {
    case Iide.LogLevel.DEBUG:
        tag = tag_debug;
        break;
    case Iide.LogLevel.INFO:
        tag = tag_info;
        break;
    case Iide.LogLevel.WARNING:
        tag = tag_warning;
        break;
    case Iide.LogLevel.ERROR:
        tag = tag_error;
        break;
    case Iide.LogLevel.CRITICAL:
        tag = tag_critical;
        break;
    default:
        tag = tag_dim;
        break;
    }

    var ts_text = "[%s] ".printf (timestamp);
    var level_text = "[%s] ".printf (level_str);
    var domain_text = "[%s] ".printf (domain);

    var ts_start = end;
    buffer.insert (ref end, ts_text, ts_text.length);
    var ts_end = end;
```

```
        buffer.apply_tag (tag_dim, ts_start, ts_end);

        var level_start = end;
        buffer.insert (ref end, level_text, level_text.length);
        var level_end = end;
        buffer.apply_tag (tag, level_start, level_end);

        var domain_start = end;
        buffer.insert (ref end, domain_text, domain_text.length);
        var domain_end = end;
        buffer.apply_tag (tag_bold, domain_start, domain_end);

        buffer.insert (ref end, message, message.length);

        if (entry.details != null) {
            var detail_start = end;
            buffer.insert (ref end, "\n  ", 3);
            var detail_end = end;
            buffer.apply_tag (tag_dim, detail_start, detail_end);

            buffer.insert (ref end, entry.details, entry.details.length);
        }

        buffer.insert (ref end, "\n", 1);

        line_count++;
    }
}
```

File: src/iide.gresource.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<gresources>
  <gresource prefix="/org/github/kai66673/iide">
    <file preprocess="xml-stripblanks">window.ui</file>
    <file preprocess="xml-stripblanks">shortcuts-dialog.ui</file>
    <file>style.css</file>
  </gresource>
  <gresource prefix="/org/github/kai66673/iide/icons">
     <file
            compressed="true"
            alias="scalable/mimetypes/text-x-python-symbolic.svg"
        >icons/mimetypes/text-x-python-symbolic.svg</file>
    <file
            compressed="true"
            alias="scalable/mimetypes/text-x-vala-symbolic.svg"
```

```
        >icons/mimetypes/text-x-vala-symbolic.svg</file>
    <file
            compressed="true"
            alias="scalable/mimetypes/text-x-meson-symbolic.svg"
        >icons/mimetypes/text-x-meson-symbolic.svg</file>
    <file
            compressed="true"
            alias="scalable/mimetypes/text-markdown-symbolic.svg"
        >icons/mimetypes/text-markdown-symbolic.svg</file>
    <file
            compressed="true"
            alias="scalable/mimetypes/text-x-javascript-symbolic.svg"
        >icons/mimetypes/text-x-javascript-symbolic.svg</file>
    <file
            compressed="true"
            alias="scalable/mimetypes/text-xml-symbolic.svg"
        >icons/mimetypes/text-xml-symbolic.svg</file>
  </gresource>
</gresources>
```

File: src/Services/TreeSitter/cpp/CppTSHighlighter.vala

```
[CCode (cname = "tree_sitter_cpp")]
extern unowned TreeSitter.Language ? get_lang_cpp ();

public class Iide.CppHighlighter : BaseTreeSitterHighlighter {
    public CppHighlighter (SourceView view) {
        base (view);
    }

    protected override unowned TreeSitter.Language get_ts_language () {
        return get_lang_cpp ();
    }

    protected override string get_query_filename () {
        return "cpp/highlights.scm";
    }

    protected override string query_source () {
        return """
        ; Functions

        (call_expression
          function: (qualified_identifier
            name: (identifier) @function))
```

```
(template_function
  name: (identifier) @function)

(template_method
  name: (field_identifier) @function)

(template_function
  name: (identifier) @function)

(function_declarator
  declarator: (qualified_identifier
    name: (identifier) @function))

(function_declarator
  declarator: (field_identifier) @function)

; Types

((namespace_identifier) @type
 (#match? @type "^[A-Z]"))

(auto) @type

; Constants

(this) @variable.builtin
(null "nullptr" @constant)

; Modules
(module_name
  (identifier) @module)

; Keywords

[
 "catch"
 "class"
 "co_await"
 "co_return"
 "co_yield"
 "constexpr"
 "constinit"
 "consteval"
 "delete"
 "explicit"
 "final"
```

```
        "friend"
        "mutable"
        "namespace"
        "noexcept"
        "new"
        "override"
        "private"
        "protected"
        "public"
        "template"
        "throw"
        "try"
        "typename"
        "using"
        "concept"
        "requires"
        "virtual"
        "import"
        "export"
        "module"
       ] @keyword

       ; Strings

       (raw_string_literal) @string
       """;
    }

    protected override bool is_container_node (string node_type) {
        return node_type in new string[] {
                   "class_specifier", "struct_specifier", "function_definition", "namespace_
        };
    }
}
```

File: src/Services/TreeSitter/rust/RustHighlighter.vala

```
[CCode (cname = "tree_sitter_rust")]
extern unowned TreeSitter.Language ? get_language_rust ();

public class Iide.RustHighlighter : BaseTreeSitterHighlighter {
    public RustHighlighter (SourceView view) {
        base (view);
    }

    protected override unowned TreeSitter.Language get_ts_language () {
```

```
    return get_language_rust ();
}

protected override string get_query_filename () {
    return "rust/highlights.scm";
}

protected override string query_source () {
    return """
    ; Identifiers

    (type_identifier) @type
    (primitive_type) @type.builtin
    (field_identifier) @property

    ; Identifier conventions

    ; Assume all-caps names are constants
    ((identifier) @constant
     (#match? @constant "^[A-Z][A-Z\\d_]+$'"))

    ; Assume uppercase names are enum constructors
    ((identifier) @constructor
     (#match? @constructor "^[A-Z]"))

    ; Assume that uppercase names in paths are types
    ((scoped_identifier
      path: (identifier) @type)
     (#match? @type "^[A-Z]"))
    ((scoped_identifier
      path: (scoped_identifier
        name: (identifier) @type))
     (#match? @type "^[A-Z]"))
    ((scoped_type_identifier
      path: (identifier) @type)
     (#match? @type "^[A-Z]"))
    ((scoped_type_identifier
      path: (scoped_identifier
        name: (identifier) @type))
     (#match? @type "^[A-Z]"))

    ; Assume all qualified names in struct patterns are enum constructors. (They're
    ; either that, or struct names; highlighting both as constructors seems to be
    ; the less glaring choice of error, visually.)
    (struct_pattern
      type: (scoped_type_identifier
```

```
    name: (type_identifier) @constructor))

; Function calls

(call_expression
  function: (identifier) @function)
(call_expression
  function: (field_expression
    field: (field_identifier) @function.method))
(call_expression
  function: (scoped_identifier
    "::"
    name: (identifier) @function))

(generic_function
  function: (identifier) @function)
(generic_function
  function: (scoped_identifier
    name: (identifier) @function))
(generic_function
  function: (field_expression
    field: (field_identifier) @function.method))

(macro_invocation
  macro: (identifier) @function.macro
  "!" @function.macro)

; Function definitions

(function_item (identifier) @function)
(function_signature_item (identifier) @function)

(line_comment) @comment
(block_comment) @comment

(line_comment (doc_comment)) @comment.documentation
(block_comment (doc_comment)) @comment.documentation

"(" @punctuation.bracket
")" @punctuation.bracket
"[" @punctuation.bracket
"]" @punctuation.bracket
"{" @punctuation.bracket
"}" @punctuation.bracket

(type_arguments
```

```
  "<" @punctuation.bracket
  ">" @punctuation.bracket)
(type_parameters
  "<" @punctuation.bracket
  ">" @punctuation.bracket)

"::" @punctuation.delimiter
":" @punctuation.delimiter
"." @punctuation.delimiter
"," @punctuation.delimiter
";" @punctuation.delimiter

(parameter (identifier) @variable.parameter)

(lifetime (identifier) @label)

"as" @keyword
"async" @keyword
"await" @keyword
"break" @keyword
"const" @keyword
"continue" @keyword
"default" @keyword
"dyn" @keyword
"else" @keyword
"enum" @keyword
"extern" @keyword
"fn" @keyword
"for" @keyword
"gen" @keyword
"if" @keyword
"impl" @keyword
"in" @keyword
"let" @keyword
"loop" @keyword
"macro_rules!" @keyword
"match" @keyword
"mod" @keyword
"move" @keyword
"pub" @keyword
"raw" @keyword
"ref" @keyword
"return" @keyword
"static" @keyword
"struct" @keyword
"trait" @keyword
```

```
        "type" @keyword
        "union" @keyword
        "unsafe" @keyword
        "use" @keyword
        "where" @keyword
        "while" @keyword
        "yield" @keyword
        (crate) @keyword
        (mutable_specifier) @keyword
        (use_list (self) @keyword)
        (scoped_use_list (self) @keyword)
        (scoped_identifier (self) @keyword)
        (super) @keyword

        (self) @variable.builtin

        (char_literal) @string
        (string_literal) @string
        (raw_string_literal) @string

        (boolean_literal) @constant.builtin
        (integer_literal) @constant.builtin
        (float_literal) @constant.builtin

        (escape_sequence) @escape

        (attribute_item) @attribute
        (inner_attribute_item) @attribute

        "*" @operator
        "&" @operator
        "'" @operator
        """;
    }

    protected override bool is_container_node (string node_type) {
        return node_type in new string[] {
                   "function_item", "impl_item", "struct_item", "enum_item", "trait_item", '
        };
    }
}
```

File: src/Services/TreeSitter/vala/ValaTSHighlighter.vala

```
[CCode (cname = "tree_sitter_vala")]
extern unowned TreeSitter.Language ? get_lang_vala ();
```

```
public class Iide.ValaHighlighter : BaseTreeSitterHighlighter {
    public ValaHighlighter (SourceView view) {
        base (view);
    }

    protected override unowned TreeSitter.Language get_ts_language () {
        return get_lang_vala ();
    }

    protected override string get_query_filename () {
        return "cpp/highlights.scm";
    }

    protected override string query_source () {
        return """
        ; highlights.scm

        (comment) @comment
        (type) @type
        (unqualified_type) @type
        (attribute) @attribute
        (method_declaration (symbol (symbol) @type (identifier) @function.method))
        (method_declaration (symbol (identifier) @function.method))
        (local_function_declaration (identifier) @function)
        (destructor_declaration (identifier) @function)
        (creation_method_declaration (symbol (symbol) @type (identifier) @constructor))
        (creation_method_declaration (symbol (identifier) @constructor))
        (enum_declaration (symbol) @type)
        (enum_value (identifier) @constant)
        (errordomain_declaration (symbol) @type)
        (errorcode (identifier) @constant)
        (constant_declaration (identifier) @constant)
        (method_call_expression (member_access_expression (identifier) @function))
        (lambda_expression (identifier) @variable.parameter)
        (parameter (identifier) @variable.parameter)
        (property_declaration (symbol (identifier) @property))
        [
         (this_access)
         (base_access)
         (value_access)
        ] @variable.builtin
        (boolean) @constant.builtin
        (character) @constant
        (integer) @number
        (null) @constant.builtin
```

```
(real) @number
(regex) @constant
(string) @string
[
 (escape_sequence)
 (string_formatter)
] @string.special
(template_string) @string
(template_string_expression) @string.special
(verbatim_string) @string
[
 "var"
 "void"
] @type.builtin

[
 "abstract"
 "and"
 "as"
 "async"
 "break"
 "case"
 "catch"
 "class"
 "const"
 "construct"
 "continue"
 "default"
 "delegate"
 "delete"
 "do"
 "dynamic"
 "else"
 "enum"
 "errordomain"
 "extern"
 "finally"
 "for"
 "foreach"
 "get"
 "if"
 "in"
 "inline"
 "interface"
 "internal"
 "is"
```

```
 "lock"
 "namespace"
 "new"
 "not"
 "or"
 "out"
 "override"
 "owned"
 "partial"
 "private"
 "protected"
 "public"
 "ref"
 "return"
 "set"
 "signal"
 "sizeof"
 "static"
 "struct"
 "switch"
 "throw"
 "throws"
 "try"
 "typeof"
 "unowned"
 "using"
 "virtual"
 "weak"
 "while"
 "with"
 "yield"
] @keyword

[
 "="
 "=="
 "+"
 "+="
 "-"
 "-="
 "|"
 "|="
 "&"
 "&="
 "^"
 "^="
```

```
     "/"
     "/="
     "*"
     "*="
     "%"
     "%="
     "<<"
     "<<="
     ">>"
     ">>="
     "."
     "?."
     "->"
     "!"
     "~"
     "??"
     "?"
     ":"
     "<"
     "<="
     ">"
     ">="
     "||"
     "&&"
     "=>"
    ] @operator

    [
     ","
     ";"
    ] @punctuation.delimiter

    [
     "("
     ")"
     "{"
     "}"
     "["
     "]"
    ] @punctuation.bracket
    """;
}

protected override bool is_container_node (string node_type) {
    return node_type in new string[] { "class_definition", "function_definition" };
}
```

```
}
```

File: src/Services/IconProvider.vala

```
public class Iide.IconProvider {
    private static IconProvider? instance;
    private IconProvider() {}

    public static IconProvider get_instance() {
        if (instance == null) {
            instance = new IconProvider();
        }
        return instance;
    }

    public static string ? get_mime_type_icon_name(string mime_type) {
        switch (mime_type) {
        case "text/x-vala" :
            return "text-x-vala";
        case "text/x-meson":
            return "text-x-meson";
        case "text/markdown":
            return "text-markdown";
        case "application/json":
            return "text-x-javascript";
        case "application/x-gtk-builder":
            return "text-xml";
        }
        return "text-x-generic";
    }
}
```

File: src/Services/LoggerService.vala

```
/*
 * loggerservice.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
```

```
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

using GLib;

public enum Iide.LogLevel {
    DEBUG,
    INFO,
    WARNING,
    ERROR,
    CRITICAL;

    public string to_string () {
        switch (this) {
        case DEBUG:    return "DEBUG";
        case INFO:     return "INFO";
        case WARNING:  return "WARN";
        case ERROR:    return "ERROR";
        case CRITICAL: return "CRITICAL";
        default:       return "UNKNOWN";
        }
    }

    public string get_color () {
        switch (this) {
        case DEBUG:    return "#888888";
        case INFO:     return "#4a9eff";
        case WARNING:  return "#f5a623";
        case ERROR:    return "#e74c3c";
        case CRITICAL: return "#ff4757";
        default:       return "#ffffff";
        }
    }
}

public class Iide.LogEntry : Object {
    public int64 timestamp { get; set; }
    public Iide.LogLevel level { get; set; }
    public string domain { get; set; }
    public string message { get; set; }
```

```
    public string? details { get; set; }

    public string get_timestamp_string () {
        var time = new DateTime.from_unix_utc (timestamp / 1000000);
        var local = time.to_timezone (new TimeZone.local ());
        var msec = (int) ((timestamp / 1000) % 1000);
        return "%s.%03d".printf (local.format ("%H:%M:%S"), msec);
    }
}

public class Iide.LoggerService : Object {
    private static LoggerService? _instance;
    private Gee.ArrayList<Iide.LogEntry> entries;
    private int max_entries = 1000;
    private bool enable_logging = true;
    private File log_file;

    public signal void log_added (Iide.LogEntry entry);
    public signal void log_cleared ();

    public static LoggerService get_instance () {
        if (_instance == null) {
            _instance = new LoggerService ();
        }
        return _instance;
    }

    private LoggerService () {
        entries = new Gee.ArrayList<Iide.LogEntry> ();
        log_file = File.new_for_path (
                                    Path.build_filename (Environment.get_user_data_dir ()
        );
        try {
            var dir = log_file.get_parent ();
            if (dir != null && !dir.query_exists (null)) {
                dir.make_directory_with_parents (null);
            }
        } catch (Error e) {
            stderr.printf ("Failed to create log directory: %s\n", e.message);
        }
    }

    public void log (LogLevel level, string domain, string message, string? details = null)
        if (!enable_logging)return;

        var entry = new Iide.LogEntry () {
```

```
        timestamp = (int64) get_real_time (),
        level = level,
        domain = domain,
        message = message,
        details = details
    };

    entries.add (entry);

    if (entries.size > max_entries) {
        entries.remove_at (0);
    }

    log_to_file (entry);
    log_added (entry);
}

public void debug (string domain, string message, string? details = null) {
    log (LogLevel.DEBUG, domain, message, details);
}

public void info (string domain, string message, string? details = null) {
    log (LogLevel.INFO, domain, message, details);
}

public void warning (string domain, string message, string? details = null) {
    log (LogLevel.WARNING, domain, message, details);
}

public void error (string domain, string message, string? details = null) {
    log (LogLevel.ERROR, domain, message, details);
}

public void critical (string domain, string message, string? details = null) {
    log (LogLevel.CRITICAL, domain, message, details);
}

private void log_to_file (LogEntry entry) {
    try {
        var time = new DateTime.from_unix_utc (entry.timestamp / 1000000);
        var local = time.to_timezone (new TimeZone.local ());
        var timestamp = local.format ("%Y-%m-%d %H:%M:%S");
        var details_str = entry.details != null ? " | " + entry.details : "";
        var line = "[%s] [%s] [%s] %s%s\n".printf (timestamp, entry.level.to_string (),

        if (log_file.query_exists (null)) {
```

```
                log_file.append_to (FileCreateFlags.NONE, null).write (line.data);
            } else {
                log_file.create (FileCreateFlags.REPLACE_DESTINATION, null).write (line.data
            }
        } catch (Error e) {
            stderr.printf ("Failed to write to log file: %s\n", e.message);
        }
    }

    public Gee.ArrayList<Iide.LogEntry> get_entries () {
        return entries;
    }

    public Gee.ArrayList<Iide.LogEntry> get_entries_by_level (LogLevel level) {
        var result = new Gee.ArrayList<Iide.LogEntry> ();
        foreach (var entry in entries) {
            if (entry.level == level) {
                result.add (entry);
            }
        }
        return result;
    }

    public void clear () {
        entries.clear ();
        log_cleared ();
    }

    public void set_max_entries (int max) {
        max_entries = max;
        while (entries.size > max_entries) {
            entries.remove_at (0);
        }
    }

    public void set_logging_enabled (bool enabled) {
        enable_logging = enabled;
    }
}
```

File: src/Services/Utils.vala

```
using GLib;

[CCode(cheader_filename = "unistd.h")]
extern int getpid();
```

```
namespace Iide {
    public string mime_type_for_file(File file) {
        string mime_type = "text-x";
        try {
            var info = file.query_info("standard::*", FileQueryInfoFlags.NONE, null);
            mime_type = ContentType.get_mime_type(info.get_attribute_as_string(FileAttribute
        } catch (Error e) {
        }
        return mime_type;
    }

    public int application_pid() {
        return getpid();
    }

    public void copy_resource_to_file(string resource_path, string local_path) {
        // 1.          File        (               resource:///)
        var resource_file = File.new_for_uri(resource_path);

        // 2.          File
        var local_file = File.new_for_path(local_path);

        try {
            // 3.             .    OVERWRITE          ,            .
            resource_file.copy(local_file, FileCopyFlags.OVERWRITE, null, null);
            print("               : %s\n", local_path);
        } catch (Error e) {
            stderr.printf("            : %s\n", e.message);
        }
    }
}
```

File: src/Widgets/ToolViews/ProjectView.vala

```
using Gtk;

public class Iide.FileTreeView : Box {
    private ColumnView column_view;
    private SingleSelection selection;
    private GLib.File? _root_directory = null;
    private CustomSorter sorter;

    public signal void file_activated (FileItem item);

    //              null
```

```
public GLib.File? root_directory {
    get { return _root_directory; }
    set {
        _root_directory = value;

        if (_root_directory == null) {
            //         null,                View
            column_view.model = null;
            selection = null;
            return;
        }

        //
        var root_store = create_file_model (_root_directory);
        var sorted_root = new SortListModel (root_store, sorter);

        //           ,
        var tree_model = new TreeListModel (sorted_root, false, false, (item) => {
            var file_item = item as FileItem;
            if (file_item != null && file_item.is_directory) {
                var child_store = create_file_model (file_item.file);
                return new SortListModel (child_store, sorter);
            }
            return null;
        });

        selection = new SingleSelection (tree_model);
        column_view.model = selection;
    }
}

public FileTreeView (GLib.File? root_dir = null) {
    Object (orientation : Orientation.VERTICAL);

    //          View
    column_view = new ColumnView (null);
    //
    column_view.get_first_child ().set_visible (false);

    //                  ,
    if (root_dir != null) {
        root_directory = root_dir;
    }

    setup_view ();
}
```

```
public void set_root_file (GLib.File? root_dir) {
    root_directory = root_dir;
}

private void setup_view () {
    var column = new ColumnViewColumn (null, create_factory ());
    column.expand = true;
    sorter = new CustomSorter ((a, b) => {
        var fi1 = a as FileItem;
        var fi2 = b as FileItem;
        if (fi1 == null || fi2 == null)return 0;

        //              :
        if (fi1.is_directory != fi2.is_directory) {
            return fi1.is_directory ? -1 : 1;
        }
        //
        return strcmp (fi1.name.down (), fi2.name.down ());
    });
    sorter.set_sort_func ((a, b) => {
        var fi1 = a as FileItem;
        var fi2 = b as FileItem;
        if (fi1 == null || fi2 == null)return 0;

        //              :
        if (fi1.is_directory != fi2.is_directory) {
            return fi1.is_directory ? -1 : 1;
        }
        //
        return strcmp (fi1.name.down (), fi2.name.down ());
    });
    column.sorter = sorter;
    column_view.append_column (column);

    var scroll = new ScrolledWindow ();
    scroll.vexpand = true;
    scroll.set_child (column_view);
    this.append (scroll);

    column_view.activate.connect ((pos) => {
        var item = selection.get_item (pos) as TreeListRow;
        if (item != null) {
            var file_item = item.get_item () as FileItem;
            if (file_item != null) {
                file_activated (file_item);
```

```
            }
        }
    });
}

private GLib.ListStore create_file_model (GLib.File dir) {
    var store = new GLib.ListStore (typeof (FileItem));
    try {
        //
        var enumerator = dir.enumerate_children ("standard::display-name,standard::type,
        GLib.FileInfo info;
        while ((info = enumerator.next_file (null)) != null) {
            store.append (new FileItem (dir.get_child (info.get_name ()), info));
        }
    } catch (Error e) {
        //
        debug ("Cannot read directory: %s", e.message);
    }
    return store;
}

private ListItemFactory create_factory () {
    var factory = new SignalListItemFactory ();

    factory.setup.connect ((list_item) => {
        var item = (Gtk.ListItem) list_item;
        var expander = new TreeExpander ();
        var box = new Box (Orientation.HORIZONTAL, 6);
        var icon = new Image ();
        var label = new Label ("");

        box.append (icon);
        box.append (label);
        expander.set_child (box);
        item.set_child (expander);
    });

    factory.bind.connect ((list_item) => {
        var item = (Gtk.ListItem) list_item;
        var tree_row = item.get_item () as TreeListRow;
        if (tree_row == null)return;

        var file_item = tree_row.get_item () as FileItem;
        var expander = item.get_child () as TreeExpander;
        var box = expander.get_child () as Box;
        var icon = box.get_first_child () as Image;
```

```
            var label = icon.get_next_sibling () as Label;

            expander.list_row = tree_row;
            if (file_item != null) {
                label.label = file_item.name;
                icon.icon_name = file_item.is_directory ? "folder-symbolic" : IconProvider.g

                if (!file_item.is_directory) {
                    var click = new Gtk.GestureClick ();
                    click.released.connect (() => {
                        file_activated (file_item);
                    });
                    box.add_controller (click);
                }
            }
        });

        return factory;
    }
}

//
public class Iide.FileItem : Object {
    public GLib.File file { get; construct; }
    public string name { get; construct; }
    public bool is_directory { get; construct; }

    public FileItem (GLib.File file, GLib.FileInfo info) {
        Object (
                file : file,
                name : info.get_display_name (),
                is_directory: info.get_file_type () == GLib.FileType.DIRECTORY
        );
    }
}
```

File: src/Widgets/FuzzyFinderDialog.vala

```
public class Iide.FuzzyFinderDialog : Adw.Window {
    private Gtk.Entry search_entry;
    private Gtk.ListView list_view;
    private Gtk.SingleSelection selection;
    private Gtk.StringList string_list;
    private Gee.List<FileEntry> all_files;
    private Gee.List<FileEntry> filtered_files;
    private Iide.ProjectManager project_manager;
```

```
private Iide.DocumentManager document_manager;
private Window? parent_window;

private const int MAX_RESULTS = 50;

private class FileEntry : Object {
    public string path { get; construct; }
    public string name { get; construct; }
    public string relative_path { get; construct; }
    public string display_name { get; construct; }

    public FileEntry (string path, string name, string relative_path) {
        Object (path: path, name: name, relative_path: relative_path, display_name: "%s
    }
}

public FuzzyFinderDialog (Window parent_window, Iide.DocumentManager document_manager) {
    Object (
            title: _("Quick Open"),
            modal: true,
            destroy_with_parent: true,
            default_width: 600,
            default_height: 400
    );

    this.parent_window = parent_window;
    this.document_manager = document_manager;
    this.project_manager = Iide.ProjectManager.get_instance ();
    this.all_files = new Gee.ArrayList<FileEntry> ();
    this.filtered_files = new Gee.ArrayList<FileEntry> ();

    setup_ui ();
    load_project_files ();
}

private bool on_key_pressed (Gtk.EventControllerKey controller, uint keyval, uint keycod
    if (keyval == Gdk.Key.Escape) {
        this.close ();
        return true;
    } else if (keyval == Gdk.Key.Return || keyval == Gdk.Key.KP_Enter) {
        open_selected ();
        return true;
    } else if (keyval == Gdk.Key.Up || keyval == Gdk.Key.KP_Up) {
        if (selection.selected > 0) {
            selection.selected -= 1;
            list_view.scroll_to (selection.selected, Gtk.ListScrollFlags.NONE, null);
```

```
        }
        return true;
    } else if (keyval == Gdk.Key.Down || keyval == Gdk.Key.KP_Down) {
        if (selection.selected < (int) string_list.get_n_items () - 1) {
            selection.selected += 1;
            list_view.scroll_to (selection.selected, Gtk.ListScrollFlags.NONE, null);
        }
        return true;
    }
    return false;
}

private void setup_ui () {
    var vbox = new Gtk.Box (Gtk.Orientation.VERTICAL, 8);
    vbox.margin_top = 12;
    vbox.margin_bottom = 12;
    vbox.margin_start = 12;
    vbox.margin_end = 12;

    search_entry = new Gtk.Entry () {
        placeholder_text = _("Search files..."),
        hexpand = true
    };
    vbox.append (search_entry);

    string_list = new Gtk.StringList (new string[0]);
    selection = new Gtk.SingleSelection (string_list);
    list_view = new Gtk.ListView (selection, null);
    list_view.hexpand = true;
    list_view.vexpand = true;

    var factory = new Gtk.SignalListItemFactory ();
    factory.setup.connect ((item) => {
        var list_item = item as Gtk.ListItem;
        var item_box = new Gtk.Box (Gtk.Orientation.VERTICAL, 2);
        item_box.margin_start = 6;
        item_box.margin_end = 6;
        item_box.margin_top = 6;
        item_box.margin_bottom = 6;

        var name_label = new Gtk.Label (null);
        name_label.xalign = 0;
        name_label.add_css_class ("title-5");
        name_label.hexpand = true;

        var path_label = new Gtk.Label (null);
```

```
        path_label.xalign = 0;
        path_label.add_css_class ("dim-label");
        path_label.hexpand = true;

        item_box.append (name_label);
        item_box.append (path_label);
        list_item.set_child (item_box);
    });
    factory.bind.connect ((item) => {
        var list_item = item as Gtk.ListItem;
        var item_box = list_item.get_child () as Gtk.Box;
        var name_label = item_box.get_first_child () as Gtk.Label;
        var path_label = name_label.get_next_sibling () as Gtk.Label;

        var index = list_item.get_position ();
        if (index >= 0 && index < filtered_files.size) {
            var entry = filtered_files[(int) index];
            name_label.set_label (entry.name);
            path_label.set_label (entry.relative_path);
        }
    });
    list_view.factory = factory;

    var scrolled = new Gtk.ScrolledWindow ();
    scrolled.child = list_view;
    scrolled.hexpand = true;
    scrolled.vexpand = true;
    vbox.append (scrolled);

    set_content (vbox);

    search_entry.changed.connect (on_search_changed);

    var key_controller = new Gtk.EventControllerKey ();
    key_controller.key_pressed.connect (on_key_pressed);
    search_entry.add_controller (key_controller);

    list_view.activate.connect (() => {
        open_selected ();
    });

    search_entry.activate.connect (() => {
        open_selected ();
    });
}
```

```
private void load_project_files () {
    var project_root = project_manager.get_current_project_root ();
    if (project_root == null) {
        return;
    }

    var root_path = project_root.get_path ();
    if (root_path == null) {
        return;
    }

    try {
        scan_directory (project_root, root_path);
    } catch (Error e) {
        warning ("Error scanning directory: %s", e.message);
    }

    all_files.sort ((a, b) => a.name.collate (b.name));
    on_search_changed ();
}

private void scan_directory (GLib.File dir, string base_path) throws Error {
    var enumerator = dir.enumerate_children (
                                            "standard::name,standard::type",
                                            FileQueryInfoFlags.NONE,
                                            null
    );

    FileInfo? info;
    while ((info = enumerator.next_file (null)) != null) {
        var name = info.get_name ();
        if (name.has_prefix (".")) {
            continue;
        }

        var file_type = info.get_file_type ();
        if (file_type == FileType.DIRECTORY) {
            if (name == "node_modules" || name == "target" || name == "build" || name ==
                continue;
            }
            scan_directory (dir.get_child (name), base_path);
        } else if (file_type == FileType.REGULAR) {
            var path = dir.get_child (name).get_path ();
            if (path != null) {
                var relative = path.substring (base_path.length + 1);
                all_files.add (new FileEntry (path, name, relative));
```

```
            }
        }
    }
}

private void on_search_changed () {
    var query = search_entry.get_text ().down ();
    filtered_files.clear ();

    if (query == "") {
        foreach (var f in all_files) {
            if (filtered_files.size >= MAX_RESULTS)break;
            filtered_files.add (f);
        }
    } else {
        foreach (var f in all_files) {
            if (fuzzy_match (f.name.down (), query)) {
                filtered_files.add (f);
                if (filtered_files.size >= MAX_RESULTS)break;
            }
        }
    }

    update_results ();
}

private bool fuzzy_match (string text, string query) {
    int qi = 0;
    for (int i = 0; i < text.length && qi < query.length; i++) {
        if (text[i] == query[qi]) {
            qi++;
        }
    }
    return qi == query.length;
}

private void update_results () {
    var strings = new string[filtered_files.size];
    for (int i = 0; i < filtered_files.size; i++) {
        strings[i] = filtered_files[i].display_name;
    }

    string_list.splice (0, string_list.get_n_items (), strings);

    if (filtered_files.size > 0) {
        selection.selected = 0;
```

```
        }
    }

    private void open_selected () {
        var index = (int) selection.selected;
        if (index >= 0 && index < filtered_files.size) {
            var entry = filtered_files[index];
            var file = GLib.File.new_for_path (entry.path);
            document_manager.open_document (file, null);
            this.close ();
        }
    }

    public override void show () {
        base.show ();
        search_entry.grab_focus ();
    }
}
```

File: src/Widgets/PreferencesDialog.vala

```
/*
 * preferencesdialog.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

public class Iide.PreferencesDialog : Adw.PreferencesWindow {
    private Iide.SettingsService settings;
    private Adw.StyleManager style_manager;
```

```
private Adw.ComboRow color_scheme_row;
private Adw.SwitchRow show_minimap_row;
private Adw.SwitchRow show_line_numbers_row;
private Adw.SwitchRow highlight_current_line_row;
private Adw.SwitchRow auto_indent_row;

public PreferencesDialog () {
    Object (modal: true, destroy_with_parent: true);
    settings = Iide.SettingsService.get_instance ();
    style_manager = Adw.StyleManager.get_default ();

    title = _("Preferences");
    set_default_size (500, 450);

    build_ui ();
}

private void build_ui () {
    var appearance_page = new Adw.PreferencesPage () {
        title = _("Appearance"),
        icon_name = "preferences-desktop-appearance-symbolic"
    };

    var appearance_group = new Adw.PreferencesGroup () {
        title = _("Appearance")
    };

    color_scheme_row = new Adw.ComboRow () {
        title = _("Color Scheme"),
        model = new Gtk.StringList ({ "System", "Light", "Dark" })
    };
    var scheme_index = settings.color_scheme;
    color_scheme_row.selected = (uint) scheme_index;
    color_scheme_row.notify["selected"].connect (() => {
        var scheme = (ColorScheme) color_scheme_row.selected;
        settings.color_scheme = scheme;
        style_manager.color_scheme = scheme.to_adw_color_scheme ();
    });
    appearance_group.add (color_scheme_row);
    appearance_page.add (appearance_group);

    var editor_page = new Adw.PreferencesPage () {
        title = _("Editor"),
        icon_name = "accessories-text-editor-symbolic"
    };
```

```
var editor_group = new Adw.PreferencesGroup () {
    title = _("Editor")
};

show_minimap_row = new Adw.SwitchRow () {
    title = _("Show Minimap"),
    subtitle = _("Display a minimap showing the document overview")
};
show_minimap_row.active = settings.show_minimap;
show_minimap_row.notify["active"].connect (() => {
    settings.show_minimap = show_minimap_row.active;
});
editor_group.add (show_minimap_row);

show_line_numbers_row = new Adw.SwitchRow () {
    title = _("Show Line Numbers"),
    subtitle = _("Display line numbers on the editor gutter")
};
show_line_numbers_row.active = settings.show_line_numbers;
show_line_numbers_row.notify["active"].connect (() => {
    settings.show_line_numbers = show_line_numbers_row.active;
});
editor_group.add (show_line_numbers_row);

highlight_current_line_row = new Adw.SwitchRow () {
    title = _("Highlight Current Line"),
    subtitle = _("Highlight the line where the cursor is positioned")
};
highlight_current_line_row.active = settings.highlight_current_line;
highlight_current_line_row.notify["active"].connect (() => {
    settings.highlight_current_line = highlight_current_line_row.active;
});
editor_group.add (highlight_current_line_row);

auto_indent_row = new Adw.SwitchRow () {
    title = _("Auto Indent"),
    subtitle = _("Automatically indent new lines based on the previous line")
};
auto_indent_row.active = settings.auto_indent;
auto_indent_row.notify["active"].connect (() => {
    settings.auto_indent = auto_indent_row.active;
});
editor_group.add (auto_indent_row);

var sizes = FontSizeHelper.get_available_sizes ();
var size_strings = new string[sizes.length];
```

```
for (int i = 0; i < sizes.length; i++) {
    size_strings[i] = "%d px".printf (sizes[i]);
}
var font_size_model = new Gtk.StringList (size_strings);
var current_level = settings.editor_font_size;
if (current_level < FontSizeHelper.MIN_ZOOM_LEVEL || current_level > FontSizeHelper.
    current_level = FontSizeHelper.DEFAULT_ZOOM_LEVEL;
}

var font_size_row = new Adw.ComboRow () {
    title = _("Font Size"),
    model = font_size_model,
    selected = (uint) (current_level - 1)
};
font_size_row.notify["selected"].connect (() => {
    settings.editor_font_size = (int) font_size_row.selected + 1;
});
editor_group.add (font_size_row);
editor_page.add (editor_group);

var projects_page = new Adw.PreferencesPage () {
    title = _("Projects"),
    icon_name = "folder-open-symbolic"
};

var projects_group = new Adw.PreferencesGroup () {
    title = _("Projects")
};

var recent_projects_row = new Adw.ActionRow () {
    title = _("Recent Projects")
};
var recent_projects = settings.recent_projects;
if (recent_projects.length > 0) {
    var subtitle = string.joinv ("\n", recent_projects);
    recent_projects_row.subtitle = subtitle;
} else {
    recent_projects_row.subtitle = _("No recent projects");
}
var clear_button = new Gtk.Button () {
    icon_name = "edit-clear-symbolic",
    tooltip_text = _("Clear Recent Projects")
};
clear_button.clicked.connect (() => {
    settings.clear_recent_projects ();
    recent_projects_row.subtitle = _("No recent projects");
```

```
});
recent_projects_row.add_suffix (clear_button);
recent_projects_row.activatable_widget = clear_button;
projects_group.add (recent_projects_row);
projects_page.add (projects_group);

var shortcuts_page = new Adw.PreferencesPage () {
    title = _("Shortcuts"),
    icon_name = "input-keyboard-symbolic"
};

var shortcuts_group = new Adw.PreferencesGroup () {
    title = _("Keyboard Shortcuts")
};

var action_manager = Iide.ActionManager.get_instance ();
var actions = action_manager.get_all_actions ();

var list_box = new Gtk.ListBox () {
    selection_mode = Gtk.SelectionMode.NONE,
    show_separators = true
};

var sorted_actions = new Gee.ArrayList<Iide.Action> ();
foreach (var action in actions) {
    sorted_actions.add (action);
}
sorted_actions.sort ((a, b) => {
    var cat_cmp = (a.category ?? "").collate (b.category ?? "");
    if (cat_cmp != 0)return cat_cmp;
    return a.name.collate (b.name);
});

string? current_category = null;
foreach (var action in sorted_actions) {
    if (action.category != current_category) {
        current_category = action.category;
        var header_row = new Gtk.ListBoxRow () {
            selectable = false,
            activatable = false
        };
        var header_label = new Gtk.Label (action.category ?? _("Other")) {
            halign = Gtk.Align.START,
            margin_top = 8,
            margin_bottom = 4
        };
```

```
                header_label.add_css_class ("title");
                header_label.add_css_class ("dim-label");
                header_row.child = header_label;
                list_box.append (header_row);
            }
            var row = new ShortcutRow (action);
            list_box.append (row);
        }

        var scrolled = new Gtk.ScrolledWindow () {
            child = list_box,
            hexpand = true,
            vexpand = true
        };
        scrolled.set_size_request (-1, 300);
        shortcuts_group.add (scrolled);
        shortcuts_page.add (shortcuts_group);

        add (appearance_page);
        add (editor_page);
        add (projects_page);
        add (shortcuts_page);
    }
}

private class ShortcutRow : Gtk.ListBoxRow {
    private Iide.Action action;
    private Gtk.Label shortcut_label;

    public ShortcutRow (Iide.Action action) {
        this.action = action;

        var box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 12) {
            margin_start = 12,
            margin_end = 12,
            margin_top = 8,
            margin_bottom = 8
        };

        var info_box = new Gtk.Box (Gtk.Orientation.VERTICAL, 2);
        var name_label = new Gtk.Label (action.name) {
            halign = Gtk.Align.START,
            hexpand = true
        };
        name_label.add_css_class ("title");
        var desc_label = new Gtk.Label (action.description ?? "") {
```

```
        halign = Gtk.Align.START,
        hexpand = true
    };
    desc_label.add_css_class ("caption");
    desc_label.add_css_class ("dim-label");
    info_box.append (name_label);
    info_box.append (desc_label);
    box.append (info_box);

    shortcut_label = new Gtk.Label (action.shortcut != null ? action.shortcut : _("None'
        halign = Gtk.Align.END,
        margin_start = 12
    };
    shortcut_label.add_css_class ("caption");
    shortcut_label.add_css_class ("dim-label");
    box.append (shortcut_label);

    var clear_button = new Gtk.Button () {
        icon_name = "edit-clear-symbolic",
        tooltip_text = _("Clear shortcut"),
        valign = Gtk.Align.CENTER
    };
    clear_button.clicked.connect (on_clear_clicked);
    box.append (clear_button);

    var capture_button = new Gtk.Button () {
        label = _("Change"),
        valign = Gtk.Align.CENTER
    };
    capture_button.clicked.connect (on_capture_clicked);
    box.append (capture_button);

    child = box;

    action.shortcut_changed.connect ((new_shortcut) => {
        shortcut_label.set_label (new_shortcut != null ? new_shortcut : _("None"));
    });
}

private void on_clear_clicked () {
    Iide.ActionManager.get_instance ().set_shortcut (action.id, null);
}

private void on_capture_clicked () {
    var window = this.get_ancestor (typeof (Gtk.Window)) as Gtk.Window;
    var dialog = new ShortcutCaptureWindow (action, window);
```

```
        dialog.show ();
    }
}

private class ShortcutCaptureWindow : Gtk.Window {
    private Iide.Action action;
    private Gtk.Label label;
    private uint keyval = 0;
    private Gdk.ModifierType modifiers;

    public ShortcutCaptureWindow (Iide.Action action, Gtk.Window? parent) {
        this.action = action;
        title = _("Set Shortcut for %s").printf (action.name);
        modal = true;
        if (parent != null) {
            set_transient_for (parent);
        }

        var content_box = new Gtk.Box (Gtk.Orientation.VERTICAL, 12) {
            margin_top = 20,
            margin_bottom = 20,
            margin_start = 20,
            margin_end = 20
        };

        label = new Gtk.Label (_("Press a key combination...")) {
            halign = Gtk.Align.CENTER
        };
        content_box.append (label);

        var current = action.shortcut;
        if (current != null && current != "") {
            var hint = new Gtk.Label (_("Current: %s").printf (current)) {
                halign = Gtk.Align.CENTER
            };
            hint.add_css_class ("caption");
            hint.add_css_class ("dim-label");
            content_box.append (hint);
        }

        var event_controller = new Gtk.EventControllerKey ();
        event_controller.key_pressed.connect (on_key_pressed);
        content_box.add_controller (event_controller);

        var cancel_button = new Gtk.Button () {
            label = _("Cancel")
```

```
        };
        cancel_button.clicked.connect (() => destroy ());

        var clear_button = new Gtk.Button () {
            label = _("Clear")
        };
        clear_button.clicked.connect (() => {
            Iide.ActionManager.get_instance ().set_shortcut (action.id, null);
            destroy ();
        });

        var save_button = new Gtk.Button () {
            label = _("Save")
        };
        save_button.add_css_class ("suggested-action");
        save_button.clicked.connect (on_save);

        var button_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
        button_box.append (cancel_button);
        button_box.append (clear_button);
        button_box.append (save_button);
        button_box.halign = Gtk.Align.END;
        content_box.append (button_box);

        child = content_box;
        set_size_request (350, 120);
    }

    private bool on_key_pressed (uint keyval, uint keycode, Gdk.ModifierType state) {
        var modifiers = state & (Gdk.ModifierType.SHIFT_MASK | Gdk.ModifierType.CONTROL_MASK

        if (keyval == Gdk.Key.Escape) {
            destroy ();
            return true;
        }

        if (keyval == Gdk.Key.BackSpace && modifiers == 0) {
            Iide.ActionManager.get_instance ().set_shortcut (action.id, null);
            destroy ();
            return true;
        }

        if (modifiers == 0 || (modifiers & Gdk.ModifierType.CONTROL_MASK) != 0 ||
            (modifiers & Gdk.ModifierType.SHIFT_MASK) != 0 ||
            (modifiers & Gdk.ModifierType.ALT_MASK) != 0 ||
            (modifiers & Gdk.ModifierType.SUPER_MASK) != 0) {
```

```
            if (modifiers != 0) {
                this.keyval = keyval;
                this.modifiers = modifiers;
                label.set_label (format_shortcut (keyval, modifiers));
            }
            return true;
        }
        return false;
    }

    private string format_shortcut (uint keyval, Gdk.ModifierType modifiers) {
        var accel_string = "";

        if ((modifiers & Gdk.ModifierType.CONTROL_MASK) != 0)accel_string += "<primary>";
        if ((modifiers & Gdk.ModifierType.ALT_MASK) != 0)accel_string += "<Alt>";
        if ((modifiers & Gdk.ModifierType.SHIFT_MASK) != 0)accel_string += "<Shift>";
        if ((modifiers & Gdk.ModifierType.SUPER_MASK) != 0)accel_string += "<Super>";

        var key_name = Gdk.keyval_name (keyval);
        if (key_name != null) {
            accel_string += key_name;
        }

        return accel_string;
    }

    private void on_save () {
        if (keyval != 0) {
            var shortcut = format_shortcut (keyval, modifiers);
            Iide.ActionManager.get_instance ().set_shortcut (action.id, shortcut);
        }
        destroy ();
    }
}
```

File: src/Widgets/SearchInFilesDialog.vala

```
public class Iide.SearchInFilesDialog : Adw.Window {
    private Gtk.Entry search_entry;
    private Gtk.ListView results_view;
    private Gtk.SingleSelection selection;
    private Iide.ProjectManager project_manager;
    private Iide.DocumentManager document_manager;
    private Window? parent_window;
    private string project_root_path;
```

```
private string current_query;

private const int MAX_RESULTS = 200;

private interface ListItem : Object {
    public abstract string get_display_text ();
    public abstract bool is_header ();
}

private class SearchResult : Object, ListItem {
    public string file_path { get; construct; }
    public string file_name { get; construct; }
    public string relative_path { get; construct; }
    public int line_number { get; construct; }
    public string line_content { get; construct; }
    public int match_start { get; construct; }
    public int match_end { get; construct; }

    public SearchResult (string file_path, string file_name, string relative_path, int l
        Object (
                file_path: file_path,
                file_name: file_name,
                relative_path: relative_path,
                line_number: line_number,
                line_content: line_content,
                match_start: match_start,
                match_end: match_end
        );
    }

    public string get_display_text () {
        return "%d: %s".printf (line_number + 1, line_content);
    }

    public bool is_header () {
        return false;
    }
}

private class ResultGroup : Object, ListItem {
    public string file_path { get; construct; }
    public string file_name { get; construct; }
    public string relative_path { get; construct; }
    public int result_count { get; construct; }

    public ResultGroup (string file_path, string file_name, string relative_path, int re
```

```
        Object (
                file_path: file_path,
                file_name: file_name,
                relative_path: relative_path,
                result_count: result_count
        );
    }

    public string get_display_text () {
        return relative_path;
    }

    public bool is_header () {
        return true;
    }
}

private Gee.List<ListItem> all_items;
private Gee.List<ListItem> filtered_items;
private Gtk.StringList string_list;

public SearchInFilesDialog (Window parent_window, Iide.DocumentManager document_manager)
    Object (
            title: _("Search in Files"),
            modal: true,
            destroy_with_parent: true,
            default_width: 800,
            default_height: 500
    );

    this.parent_window = parent_window;
    this.document_manager = document_manager;
    this.project_manager = Iide.ProjectManager.get_instance ();
    this.all_items = new Gee.ArrayList<ListItem> ();
    this.filtered_items = new Gee.ArrayList<ListItem> ();

    var project_root = project_manager.get_current_project_root ();
    if (project_root != null) {
        this.project_root_path = project_root.get_path () ?? "";
    } else {
        this.project_root_path = "";
    }

    setup_ui ();
}
```

```
private void setup_ui () {
    var vbox = new Gtk.Box (Gtk.Orientation.VERTICAL, 8);
    vbox.margin_top = 12;
    vbox.margin_bottom = 12;
    vbox.margin_start = 12;
    vbox.margin_end = 12;

    search_entry = new Gtk.Entry () {
        placeholder_text = _("Search text..."),
        hexpand = true
    };
    vbox.append (search_entry);

    var list_model = new Gtk.StringList (new string[0]);
    string_list = list_model;
    selection = new Gtk.SingleSelection (list_model);
    results_view = new Gtk.ListView (selection, null);
    results_view.hexpand = true;
    results_view.vexpand = true;
    results_view.show_separators = true;

    var factory = new Gtk.SignalListItemFactory ();
    factory.setup.connect ((item) => {
        var list_item = item as Gtk.ListItem;
        list_item.set_child (new Gtk.Label (null));
    });
    factory.bind.connect ((item) => {
        var list_item = item as Gtk.ListItem;
        var label = list_item.get_child () as Gtk.Label;
        var index = list_item.get_position ();

        if (index >= 0 && index < filtered_items.size) {
            var list_item_obj = filtered_items[(int) index];
            if (list_item_obj.is_header ()) {
                var group = list_item_obj as ResultGroup;
                label.set_label ("%s (%d)".printf (group.relative_path, group.result_cou
                label.add_css_class ("title-5");
                label.add_css_class ("dim-label");
                label.xalign = 0;
            } else {
                label.set_label (list_item_obj.get_display_text ());
                label.add_css_class ("monospace");
                label.add_css_class ("body");
                label.xalign = 0;
            }
        }
```

```
        });
        results_view.factory = factory;

        var scrolled = new Gtk.ScrolledWindow ();
        scrolled.child = results_view;
        scrolled.hexpand = true;
        scrolled.vexpand = true;
        vbox.append (scrolled);

        set_content (vbox);

        search_entry.changed.connect (on_search_changed);

        var key_controller = new Gtk.EventControllerKey ();
        key_controller.key_pressed.connect (on_key_pressed);
        search_entry.add_controller (key_controller);

        results_view.activate.connect (() => {
            open_selected ();
        });

        search_entry.activate.connect (() => {
            open_selected ();
        });

        search_entry.focus_on_click = false;

        this.set_default_widget (search_entry);
    }

    private bool on_key_pressed (Gtk.EventControllerKey controller, uint keyval, uint keyco
        if (keyval == Gdk.Key.Escape) {
            this.close ();
            return true;
        } else if (keyval == Gdk.Key.Return || keyval == Gdk.Key.KP_Enter) {
            open_selected ();
            return true;
        } else if (keyval == Gdk.Key.Up || keyval == Gdk.Key.KP_Up) {
            navigate_up ();
            return true;
        } else if (keyval == Gdk.Key.Down || keyval == Gdk.Key.KP_Down) {
            navigate_down ();
            return true;
        }
        return false;
    }
```

```
private void navigate_up () {
    var current = (int) selection.selected;
    if (current <= 0)return;

    for (int i = current - 1; i >= 0; i--) {
        if (filtered_items[i] is SearchResult) {
            selection.selected = i;
            results_view.scroll_to (i, Gtk.ListScrollFlags.NONE, null);
            return;
        }
    }
}

private void navigate_down () {
    var current = (int) selection.selected;
    var max = (int) string_list.get_n_items () - 1;
    if (current >= max)return;

    for (int i = current + 1; i < filtered_items.size; i++) {
        if (filtered_items[i] is SearchResult) {
            selection.selected = i;
            results_view.scroll_to (i, Gtk.ListScrollFlags.NONE, null);
            return;
        }
    }
}

private void on_search_changed () {
    current_query = search_entry.get_text ();

    if (current_query == "" || current_query.length < 2) {
        filtered_items = new Gee.ArrayList<ListItem> ();
        update_results ();
        return;
    }

    var query_lower = current_query.down ();
    var results_by_file = new Gee.HashMap<string, Gee.List<SearchResult>> ();

    foreach (var item in all_items) {
        if (item is ResultGroup) {
            continue;
        }
        var result = item as SearchResult;
        if (result.line_content.down ().contains (query_lower)) {
```

```
            if (!results_by_file.has_key (result.file_path)) {
                results_by_file[result.file_path] = new Gee.ArrayList<SearchResult> ();
            }
            results_by_file[result.file_path].add (result);
        }
    }

    filtered_items = new Gee.ArrayList<ListItem> ();
    foreach (var entry in results_by_file) {
        if (filtered_items.size >= MAX_RESULTS)break;

        var group = new ResultGroup (
                                    entry.key,
                                    entry.value[0].file_name,
                                    entry.value[0].relative_path,
                                    entry.value.size
        );
        filtered_items.add (group);

        foreach (var result in entry.value) {
            if (filtered_items.size >= MAX_RESULTS)break;
            filtered_items.add (result);
        }
    }

    update_results ();
}

private void update_results () {
    var strings = new string[filtered_items.size];
    for (int i = 0; i < filtered_items.size; i++) {
        strings[i] = filtered_items[i].get_display_text ();
    }

    string_list.splice (0, string_list.get_n_items (), strings);

    if (filtered_items.size > 0) {
        select_first_result ();
    }
}

private void select_first_result () {
    for (int i = 0; i < filtered_items.size; i++) {
        if (filtered_items[i] is SearchResult) {
            selection.selected = i;
            return;
```

```
        }
    }
}

private void open_selected () {
    var index = (int) selection.selected;
    if (index >= 0 && index < filtered_items.size) {
        var item = filtered_items[index];
        if (item is SearchResult) {
            var result = item as SearchResult;
            var file = GLib.File.new_for_path (result.file_path);

            var query_lower = current_query.down ();
            var line_lower = result.line_content.down ();
            var pos = line_lower.index_of (query_lower);
            var start_col = pos >= 0 ? pos : 0;
            var end_col = pos >= 0 ? pos + (int) current_query.length : start_col;

            document_manager.open_document_with_selection (file, result.line_number, sta
            this.close ();
        }
    }
}

public void start_search (string query) {
    search_entry.set_text (query);
}

private void scan_files_for_search () {
    if (project_root_path == "") {
        return;
    }

    try {
        scan_directory (GLib.File.new_for_path (project_root_path));
    } catch (Error e) {
        warning ("Error scanning for search: %s", e.message);
    }

    on_search_changed ();
}

private void scan_directory (GLib.File dir) throws Error {
    var enumerator = dir.enumerate_children (
                                             "standard::name,standard::type",
                                             FileQueryInfoFlags.NONE,
```

```
                                            null
    );

    FileInfo? info;
    while ((info = enumerator.next_file (null)) != null) {
        var name = info.get_name ();
        if (name.has_prefix (".")) {
            continue;
        }

        var file_type = info.get_file_type ();
        if (file_type == FileType.DIRECTORY) {
            if (name == "node_modules" || name == "target" || name == "build" ||
                name == "__pycache__" || name == ".git") {
                continue;
            }
            scan_directory (dir.get_child (name));
        } else if (file_type == FileType.REGULAR) {
            var path = dir.get_child (name).get_path ();
            if (path != null) {
                search_file (path);
            }
        }
    }
}

private void search_file (string file_path) {
    try {
        var file = GLib.File.new_for_path (file_path);
        var dis = new DataInputStream (file.read ());
        string line;
        int line_num = 0;

        string relative_path = file_path;
        if (file_path.has_prefix (project_root_path)) {
            relative_path = file_path.substring (project_root_path.length);
            if (relative_path.has_prefix ("/")) {
                relative_path = relative_path.substring (1);
            }
        }

        while ((line = dis.read_line ()) != null) {
            all_items.add (new SearchResult (
                                            file_path,
                                            file.get_basename (),
                                            relative_path,
```

```
                                                    line_num,
                                                    line.strip (),
                                                    0, 0
                ));
                line_num++;
            }
            dis.close ();
        } catch (Error e) {
        }
    }

    public override void show () {
        base.show ();
        if (all_items.size == 0) {
            scan_files_for_search ();
        }
        search_entry.grab_focus ();
    }
}
```

File: src/Services/LSP/IdeLspManager.vala

```
using GLib;
using Gee;
using GtkSource;

namespace Iide {

    public class IdeLspManager : GLib.Object {
        private static IdeLspManager? _instance;
        private IdeLspService lsp_service;

        public static unowned IdeLspManager get_instance () {
            if (_instance == null) {
                _instance = new IdeLspManager ();
            }
            return _instance;
        }

        construct {
            lsp_service = IdeLspService.get_instance ();
        }

        public async void open_document (string uri, string language_id, string content, str
            yield lsp_service.open_document (uri, language_id, content, workspace_root, view
        }
```

```
public async void change_document (string uri, string content, int? change_start = n
    yield lsp_service.change_document (uri, content, change_start, change_end);
}

public async void close_document (string uri) {
    yield lsp_service.close_document (uri);
}

public LspClient ? get_client_for_uri (string uri) {
    return lsp_service.get_client_for_uri (uri);
}

public void connect_diagnostics (DiagnosticsCallback diagnostics_callback) {
    lsp_service.diagnostics_updated.connect ((uri, diagnostics) => {
        diagnostics_callback (uri, diagnostics);
    });
}

public string ? get_language_id_for_file (GLib.File file) {
    string filename = file.get_basename () ?? "";

    switch (filename) {
    case "CMakeLists.txt" :
        return "cmake";
    case ".gitignore" :
        return "git-config";
    case "meson.build":
        return "meson";
    case "PKGBUILD":
        return "bash";
    default:
        break;
    }

    string path = file.get_path () ?? "";
    int dot_pos = path.last_index_of (".");
    if (dot_pos >= 0 && dot_pos < path.length - 1) {
        string ext = path[dot_pos + 1 : path.length].down ();
        switch (ext) {
        case "py":
            return "python";
        case "c":
        case "h":
            return "c";
        case "cpp":
```

```
                case "cc":
                case "cxx":
                case "hpp":
                case "hxx":
                    return "cpp";
                case "vala":
                case "vapi":
                    return "vala";
                case "rs":
                    return "rust";
                case "go":
                    return "go";
                case "js":
                case "ts":
                    return "javascript";
                case "json":
                    return "json";
                case "xml":
                    return "xml";
                case "html":
                case "htm":
                    return "html";
                case "css":
                    return "css";
                case "md":
                case "markdown":
                    return "markdown";
                case "sh":
                case "bash":
                case "zsh":
                    return "bash";
                case "yaml":
                case "yml":
                    return "yaml";
                }
            }

            return null;
        }

        public delegate void DiagnosticsCallback (string uri, Gee.ArrayList<IdeLspDiagnostic
    }
}
```

File: src/Services/PanelLayoutHelper.vala

```
/*
 * panellayouthelper.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

using Adw;

public class Iide.PanelLayoutHelper : Object {

    public class WidgetInfo {
        public string panel_id { get; set; }
        public int area { get; set; }
        public uint column { get; set; }
        public uint row { get; set; }
        public uint depth { get; set; }

        public Panel.Position to_pos () {
            var pos = new Panel.Position ();
            pos.area = (Panel.Area) this.area;
            pos.column = this.column;
            pos.row = this.row;
            pos.depth = this.depth;
            return pos;
        }
    }

    public class DocumentInfo {
        public string uri { get; set; }
```

```
    public uint column { get; set; }
    public uint row { get; set; }
}

public static string serialize_dock (Panel.Dock dock) {
    var builder = new Json.Builder ();
    builder.begin_object ();

    builder.set_member_name ("reveal_start");
    builder.add_boolean_value (dock.reveal_start);
    builder.set_member_name ("reveal_end");
    builder.add_boolean_value (dock.reveal_end);
    builder.set_member_name ("reveal_bottom");
    builder.add_boolean_value (dock.reveal_bottom);
    builder.set_member_name ("start_width");
    builder.add_int_value (dock.start_width);
    builder.set_member_name ("end_width");
    builder.add_int_value (dock.end_width);
    builder.set_member_name ("bottom_height");
    builder.add_int_value (dock.bottom_height);

    builder.set_member_name ("widgets");
    builder.begin_array ();

    dock.foreach_frame ((frame) => {
        var pages = frame.get_pages ();
        var position = frame.get_position ();

        for (uint i = 0; i < pages.get_n_items (); i++) {
            var item = pages.get_item (i) as Adw.TabPage;
            if (item == null) {
                continue;
            }
            var child = item.get_child ();
            if (child == null) {
                continue;
            }
            var widget = child as BasePanel;
            if (widget == null) {
                continue;
            }

            builder.begin_object ();
            builder.set_member_name ("panel_id");
            builder.add_string_value (widget.panel_id ());
```

```
            if (position != null) {
                builder.set_member_name ("area");
                builder.add_int_value ((int) position.get_area ());
                builder.set_member_name ("column");
                builder.add_int_value ((int) position.get_column ());
                builder.set_member_name ("row");
                builder.add_int_value ((int) position.get_row ());
                builder.set_member_name ("depth");
                builder.add_int_value ((int) position.get_depth ());
            }

            builder.end_object ();
        }
    });

    builder.end_array ();
    builder.end_object ();

    var generator = new Json.Generator ();
    generator.root = builder.get_root ();
    return generator.to_data (null);
}

public static Gee.HashMap<string, WidgetInfo> parse_widgets (string data) {
    var result = new Gee.HashMap<string, WidgetInfo> ();

    if (data == null || data == "") {
        return result;
    }

    try {
        var parser = new Json.Parser ();
        parser.load_from_data (data);
        var root = parser.get_root ().get_object ();

        if (root.has_member ("widgets")) {
            var widgets_array = root.get_array_member ("widgets");
            foreach (var node in widgets_array.get_elements ()) {
                var obj = node.get_object ();
                var info = new WidgetInfo ();
                info.panel_id = obj.has_member ("panel_id") ? obj.get_string_member ("pa
                info.area = obj.has_member ("area") ? (int) obj.get_int_member ("area")
                info.column = obj.has_member ("column") ? (uint) obj.get_int_member ("co
                info.row = obj.has_member ("row") ? (uint) obj.get_int_member ("row") :
                info.depth = obj.has_member ("depth") ? (uint) obj.get_int_member ("dept
                result.set (info.panel_id, info);
```

```
            }
        }
    } catch (Error e) {
        warning ("Failed to parse widgets: %s", e.message);
    }

    return result;
}

public static void deserialize_dock (string data, Panel.Dock dock) {
    if (data == null || data == "") {
        return;
    }

    try {
        var parser = new Json.Parser ();
        parser.load_from_data (data);
        var root = parser.get_root ().get_object ();

        if (root.has_member ("reveal_start")) {
            dock.reveal_start = root.get_boolean_member ("reveal_start");
        }
        if (root.has_member ("reveal_end")) {
            dock.reveal_end = root.get_boolean_member ("reveal_end");
        }
        if (root.has_member ("reveal_bottom")) {
            dock.reveal_bottom = root.get_boolean_member ("reveal_bottom");
        }
        if (root.has_member ("start_width")) {
            dock.start_width = (int) root.get_int_member ("start_width");
        }
        if (root.has_member ("end_width")) {
            dock.end_width = (int) root.get_int_member ("end_width");
        }
        if (root.has_member ("bottom_height")) {
            dock.bottom_height = (int) root.get_int_member ("bottom_height");
        }
    } catch (Error e) {
        warning ("Failed to parse dock layout: %s", e.message);
    }
}

public static string serialize_grid (Panel.Grid grid) {
    var builder = new Json.Builder ();
    builder.begin_object ();
```

```
builder.set_member_name ("n_columns");
builder.add_int_value ((int) grid.get_n_columns ());

builder.set_member_name ("documents");
builder.begin_array ();

grid.foreach_frame ((frame) => {
    var pages = frame.get_pages ();
    var position = frame.get_position ();

    for (uint i = 0; i < pages.get_n_items (); i++) {
        var item = pages.get_item (i) as Adw.TabPage;
        if (item == null) {
            continue;
        }
        var child = item.get_child ();
        if (child == null) {
            continue;
        }
        var widget = child as Panel.Widget;
        if (widget == null) {
            continue;
        }
        var text_view = widget as Iide.TextView;
        if (text_view == null) {
            continue;
        }

        builder.begin_object ();
        builder.set_member_name ("uri");
        builder.add_string_value (text_view.uri);
        if (position != null) {
            builder.set_member_name ("column");
            builder.add_int_value ((int) position.get_column ());
            builder.set_member_name ("row");
            builder.add_int_value ((int) position.get_row ());
        }
        builder.end_object ();
    }
});

builder.end_array ();
builder.end_object ();

var generator = new Json.Generator ();
generator.root = builder.get_root ();
```

```
        return generator.to_data (null);
    }

    public static Gee.ArrayList<DocumentInfo> parse_grid_documents (string data) {
        var result = new Gee.ArrayList<DocumentInfo> ();

        if (data == null || data == "") {
            return result;
        }

        try {
            var parser = new Json.Parser ();
            parser.load_from_data (data);
            var root = parser.get_root ().get_object ();

            if (root.has_member ("documents")) {
                var docs_array = root.get_array_member ("documents");
                foreach (var node in docs_array.get_elements ()) {
                    var obj = node.get_object ();
                    var info = new DocumentInfo ();
                    info.uri = obj.get_string_member ("uri");
                    info.column = obj.has_member ("column") ? (uint) obj.get_int_member ("co
                    info.row = obj.has_member ("row") ? (uint) obj.get_int_member ("row") :
                    result.add (info);
                }
            }
        } catch (Error e) {
            warning ("Failed to parse grid documents: %s", e.message);
        }

        return result;
    }
}
```

File: src/Services/StyleManager.vala

```
public class Iide.TagPair : Object {
    public Gtk.TextTag light { get; construct; }
    public Gtk.TextTag dark { get; construct; }

    public TagPair (Gtk.TextTag l, Gtk.TextTag d) {
        Object (light: l, dark: d);
    }

    public Gtk.TextTag get_by_index (int index) {
        return (index == 0) ? light : dark;
```

```
    }
}

public class Iide.StyleService : Object {
    private static StyleService? instance;
    public Gtk.TextTagTable shared_table { get; private set; }

    //     Gee.HashMap          TagPair
    private Gee.HashMap<string, TagPair> registry;

    public static StyleService get_instance () {
        if (instance == null)instance = new StyleService ();
        return instance;
    }

    private StyleService () {
        shared_table = new Gtk.TextTagTable ();
        registry = new Gee.HashMap<string, TagPair> ();

        // ---     ---
        setup_tag ("function", "#1a5fb4", "#62a0ea", true);
        setup_tag ("keyword", "#a51d2d", "#ff7b72", true);
        setup_tag ("string", "#26a269", "#8ff0a4", false);
        setup_tag ("comment", "#5e5c64", "#94989b", false, true); // Italic
        setup_tag ("type", "#422d5b", "#f9c06b", true);
        setup_tag ("constant", "#986a44", "#d0b5f3", false);

        // ---               ---
        setup_tag ("variable", "#241f31", "#ffffff", false);
        setup_tag ("variable.parameter", "#3584e4", "#78aeed", false);
        setup_tag ("variable.builtin", "#1a5fb4", "#62a0ea", false, true); // self, cls

        // ---           ---
        setup_tag ("number", "#3071db", "#78aeed", false);
        setup_tag ("boolean", "#986a44", "#d0b5f3", true);

        // ---             (Rust     , Vala     ) ---
        setup_tag ("function.macro", "#63452c", "#c061cb", false);
        setup_tag ("attribute", "#63452c", "#c061cb", false);
        setup_tag ("label", "#422d5b", "#f9c06b", false); // Lifetimes   Rust ('a)

        // ---               ---
        setup_tag ("operator", "#241f31", "#ffa348", false);
        setup_tag ("punctuation.bracket", "#241f31", "#d3d3d7", false);
        setup_tag ("punctuation.delimiter", "#241f31", "#d3d3d7", false);
```

```
    // ---               (#include, #define,       C/C++) ---
    setup_tag ("keyword.directive", "#63452c", "#c061cb", false);
    setup_tag ("keyword.control.import", "#63452c", "#c061cb", true);

    // ---            C++ (Namespace, Qualifier) ---
    setup_tag ("namespace", "#422d5b", "#f9c06b", false);
    setup_tag ("type.qualifier", "#a51d2d", "#ff7b72", false); // const, static, volatil

    // ---                        ---
    setup_tag ("function.call", "#1c71d8", "#78aeed", false);
    setup_tag ("function.method", "#1a5fb4", "#62a0ea", true);

    //         StyleService
    setup_tag ("bracket.lvl1", "#3584e4", "#78aeed", false); //
    setup_tag ("bracket.lvl2", "#26a269", "#8ff0a4", false); //
    setup_tag ("bracket.lvl3", "#e66100", "#ffa348", false); //
    setup_tag ("bracket.lvl4", "#e01b24", "#ff7b72", false); //
    setup_tag ("bracket.lvl5", "#9141ac", "#c061cb", false); //

    //          (     query    @none)
    setup_tag ("none", null, null, false);
}

private void setup_tag (string name, string? light_color, string? dark_color, bool bold,
    //
    var tag_light = new Gtk.TextTag (name + ":light");
    tag_light.foreground = light_color;
    if (bold)tag_light.weight = Pango.Weight.BOLD;
    if (italic)tag_light.style = Pango.Style.ITALIC;
    shared_table.add (tag_light);

    //
    var tag_dark = new Gtk.TextTag (name + ":dark");
    tag_dark.foreground = dark_color;
    if (bold)tag_dark.weight = Pango.Weight.BOLD;
    if (italic)tag_dark.style = Pango.Style.ITALIC;
    shared_table.add (tag_dark);

    //
    var pair = new TagPair (tag_light, tag_dark);
    registry.set (name, pair);
}

public Gtk.TextTag? get_tag (string name, int theme_index) {
    var pair = registry.get (name);
    if (pair != null) {
```

```
            return pair.get_by_index (theme_index);
        }
        return null;
    }
}
```

File: src/Widgets/TextView/EditorStatusBar.vala

```
public class Iide.EditorStatusBar : Gtk.Box {
    private SourceView source_view;
    private Gtk.Box breadcrumbs_container;
    private Gtk.Label pos_label;
    private Gtk.Label mode_label;

    private Gtk.Label error_label;
    private Gtk.Label warn_label;
    private Gtk.Box diagnostic_box;

    public signal void breadcrumb_clicked (uint line, uint column);

    private Iide.DiagnosticsPopover diag_popover = null;

    public EditorStatusBar (SourceView source_view) {
        Object (orientation: Gtk.Orientation.HORIZONTAL, spacing: 12);
        this.source_view = source_view;
        this.add_css_class ("editor-status-bar");

        //        : Breadcrumbs
        breadcrumbs_container = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
        breadcrumbs_container.set_hexpand (true);
        this.append (breadcrumbs_container);
        breadcrumbs_container.height_request = 24;

        //        :
        var info_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 12);

        mode_label = new Gtk.Label ("INS");
        mode_label.add_css_class ("dim-label");
        mode_label.height_request = 24;

        pos_label = new Gtk.Label ("1:1");

        info_box.append (mode_label);
        info_box.append (pos_label);
        this.append (info_box);
```

```
    diagnostic_box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 8);

    //     (    )
    error_label = new Gtk.Label ("0");
    error_label.add_css_class ("error-label"); //           CSS

    //         (   )
    warn_label = new Gtk.Label ("0");
    warn_label.add_css_class ("warning-label");

    diagnostic_box.append (new Gtk.Image.from_icon_name ("dialog-error-symbolic"));
    diagnostic_box.append (error_label);
    diagnostic_box.append (new Gtk.Image.from_icon_name ("dialog-warning-symbolic"));
    diagnostic_box.append (warn_label);

    //          -
    info_box.prepend (diagnostic_box);
    diagnostic_box.hide (); //     ,

    init_diagnostics_interaction ();
    this.diagnostic_box.add_css_class ("diagnostic-box");
}

private void init_diagnostics_interaction () {
    //
    var click_gesture = new Gtk.GestureClick ();

    //                    (pressed)
    click_gesture.pressed.connect ((n_press, x, y) => {
        //           ,                 Popover
        show_diagnostics_popup ();
    });

    //
    this.diagnostic_box.add_controller (click_gesture);

    // (      )               :
    this.diagnostic_box.set_cursor (new Gdk.Cursor.from_name ("pointer", null));

    // --- 2.             (Hover) ---
    var motion_controller = new Gtk.EventControllerMotion ();

    //
    motion_controller.enter.connect ((x, y) => {
        this.diagnostic_box.add_css_class ("hover");
        //            "  "
```

```
        this.diagnostic_box.set_cursor (new Gdk.Cursor.from_name ("pointer", null));
    });

    //
    motion_controller.leave.connect (() => {
        this.diagnostic_box.remove_css_class ("hover");
        //
        this.diagnostic_box.set_cursor (null);
    });

    this.diagnostic_box.add_controller (motion_controller);
}

private void show_diagnostics_popup () {
    if (this.diag_popover == null) {
        //        ,            diagnostic_box
        this.diag_popover = new Iide.DiagnosticsPopover (this.diagnostic_box, this.sourc
    }

    //
    this.diag_popover.refresh ();
    this.diag_popover.popup ();
}

public void update_diagnostics (int errors, int warnings, int infos) {
    if (errors == 0 && warnings == 0 && infos == 0) {
        diagnostic_box.hide ();
        return;
    }

    diagnostic_box.show ();
    error_label.label = errors.to_string ();
    warn_label.label = (warnings + infos).to_string ();
}

public void update_breadcrumbs (Gee.List<BreadcrumbItem?> crumbs) {
    //
    var child = breadcrumbs_container.get_first_child ();
    while (child != null) {
        var next = child.get_next_sibling ();
        breadcrumbs_container.remove (child);
        child = next;
    }

    foreach (var crumb in crumbs) {
        var btn = new Gtk.Button.with_label (crumb.name + " >");
```

```
            btn.add_css_class ("flat");
            btn.add_css_class ("breadcrumps-btn");
            btn.vexpand = false;
            btn.valign = Gtk.Align.CENTER;
            breadcrumbs_container.append (btn);

            btn.clicked.connect (() => {
                //
                this.breadcrumb_clicked (crumb.start_point.row, crumb.start_point.column);
            });
        }
    }

    public void update_position (int line, int col, int selection_len = 0) {
        if (selection_len > 0) {
            pos_label.label = "%d:%d (%d selected)".printf (line + 1, col + 1, selection_len
        } else {
            pos_label.label = "%d:%d".printf (line + 1, col + 1);
        }
    }

    public void update_mode (bool overwrite) {
        mode_label.label = overwrite ? "OVR" : "INS";
    }
}
```

File: src/Widgets/TextView/GutterMarkRenderer.vala

```
using Gtk;
using GtkSource;
using GLib;

public class Iide.GutterMarkRenderer : GutterRenderer {
    private int current_icon_size = 16;

    public void set_icons_size (int size) {
        if (current_icon_size == size) {
            return;
        }
        current_icon_size = size;
        queue_resize ();

        var gutter = (Gutter) get_parent ();
        if (gutter != null) {
            gutter.queue_allocate ();
        }
```

```
}

public override void measure (Gtk.Orientation orientation, int for_size, out int minimum
    minimum_baseline = natural_baseline = -1;
    if (orientation == Gtk.Orientation.HORIZONTAL) {
        minimum = natural = current_icon_size;
    } else {
        minimum = natural = 0;
    }
}

public override void snapshot_line (Gtk.Snapshot snapshot, GutterLines lines, uint line)
    var buffer = get_buffer ();
    if (buffer == null) {
        return;
    }

    Gtk.TextIter iter;
    lines.get_iter_at_line (out iter, line);
    var marks = iter.get_marks ();

    string? icon_name_to_draw = null;

    foreach (var text_mark in marks) {
        var lsp_mark = text_mark as LspDiagnosticsMark;
        if (lsp_mark != null) {
            icon_name_to_draw = lsp_mark.get_icon_name ();
        }
    }

    if (icon_name_to_draw == null) {
        return;
    }

    var display = Gdk.Display.get_default ();
    if (display == null) {
        return;
    }

    var theme = Gtk.IconTheme.get_for_display (display);
    var gicon = new GLib.ThemedIcon (icon_name_to_draw);
    var paintable = theme.lookup_by_gicon (gicon, current_icon_size, 1, Gtk.TextDirectio
    if (paintable == null) {
        return;
    }
```

```
        int y, height;
        lines.get_line_yrange (line, GutterRendererAlignmentMode.CELL, out y, out height);

        var snapshot_size = (float) current_icon_size;
        var x = (float) xpad;
        var y_pos = (float) y + ((float) height - snapshot_size) / 2.0f;

        var point = Graphene.Point () { x = x, y = y_pos };
        snapshot.translate (point);
        paintable.snapshot (snapshot, snapshot_size, snapshot_size);
        snapshot.translate (Graphene.Point () { x = -x, y = -y_pos });
    }
}
```

File: src/Services/Actions/ShortcutSettings.vala

```
public class Iide.ShortcutSettings : Object {
    private static ShortcutSettings? _instance;
    private GLib.Settings settings;
    private Gee.HashMap<string, string> shortcuts_cache;
    private Gee.HashMap<string, bool> toggle_states_cache;

    public static ShortcutSettings get_instance () {
        if (_instance == null) {
            _instance = new ShortcutSettings ();
        }
        return _instance;
    }

    private ShortcutSettings () {
        settings = new GLib.Settings ("org.github.kai66673.iide");
        shortcuts_cache = new Gee.HashMap<string, string> ();
        toggle_states_cache = new Gee.HashMap<string, bool> ();
        set_default_shortcuts ();
        set_default_toggle_states ();
        load_from_gsettings ();
    }

    private void set_default_shortcuts () {
        shortcuts_cache.set ("save", "<primary>s");
        shortcuts_cache.set ("open_project", "<primary>o");
        shortcuts_cache.set ("preferences", "<primary>comma");
        shortcuts_cache.set ("toggle_minimap", "<primary>m");
        shortcuts_cache.set ("fuzzy_finder", "<primary>p");
        shortcuts_cache.set ("search_in_files", "<primary><shift>f");
        shortcuts_cache.set ("zoom_in", "<primary>plus");
```

```
    shortcuts_cache.set ("zoom_out", "<primary>minus");
    shortcuts_cache.set ("zoom_reset", "<primary>0");
    shortcuts_cache.set ("expand_selection", "<primary>w");
    shortcuts_cache.set ("shrink_selection", "<primary><shift>w");
    shortcuts_cache.set ("quit", "<primary>q");
}

private void set_default_toggle_states () {
    toggle_states_cache.set ("toggle_minimap", true);
}

private void load_from_gsettings () {
    var json_str = settings.get_string ("shortcuts");
    if (json_str == null || json_str == "") {
        save_to_gsettings ();
        return;
    }

    try {
        var parser = new Json.Parser ();
        parser.load_from_data (json_str);
        var root = parser.get_root ();
        if (root.get_node_type () == Json.NodeType.OBJECT) {
            var obj = root.get_object ();
            obj.foreach_member ((obj, name, node) => {
                shortcuts_cache.set (name, node.get_string ());
            });
        }
    } catch (Error e) {
        warning ("Failed to parse shortcuts: %s", e.message);
    }
}

private void save_to_gsettings () {
    var root = new Json.Node (Json.NodeType.OBJECT);
    var obj = new Json.Object ();
    foreach (var entry in shortcuts_cache.entries) {
        obj.set_string_member (entry.key, entry.value);
    }
    root.set_object (obj);

    var generator = new Json.Generator ();
    generator.set_root (root);
    var json_str = generator.to_data (null);
    settings.set_string ("shortcuts", json_str);
}
```

```
public signal void shortcut_changed (string action_id, string? new_shortcut);

public string ? get_shortcut (string action_id) {
    return shortcuts_cache.get (action_id);
}

public void set_shortcut (string action_id, string? shortcut) {
    if (shortcut == null || shortcut == "") {
        shortcuts_cache.unset (action_id);
    } else {
        shortcuts_cache.set (action_id, shortcut);
    }
    save_to_gsettings ();
    shortcut_changed (action_id, shortcut);
}

public Gee.Map<string, string?> get_all_shortcuts () {
    var result = new Gee.HashMap<string, string?> ();
    foreach (var entry in shortcuts_cache.entries) {
        result.set (entry.key, entry.value);
    }
    return result;
}

public void reset_shortcut (string action_id) {
    shortcuts_cache.unset (action_id);
    save_to_gsettings ();
    shortcut_changed (action_id, null);
}

public void reset_all_shortcuts () {
    shortcuts_cache.clear ();
    save_to_gsettings ();
    foreach (var action_id in shortcuts_cache.keys) {
        shortcut_changed (action_id, null);
    }
}

public bool get_toggle_state (string action_id) {
    return toggle_states_cache.get (action_id);
}

public void set_toggle_state (string action_id, bool state) {
    toggle_states_cache.set (action_id, state);
}
```

```
}
```

File: src/Services/LSP/LspClient.vala

```
using GLib;
using Gee;

public class Iide.LspPromise : Object {
    public SourceFunc callback;
    //
    public int64 creation_time;

    public LspPromise (owned SourceFunc cb) {
        this.callback = (owned) cb;
        this.creation_time = get_monotonic_time ();
    }
}

public enum Iide.TextDocumentSyncKind {
    NONE = 0,
    FULL = 1,
    INCREMENTAL = 2
}

public class Iide.ServerCapabilities : Object {
    public TextDocumentSyncKind sync_kind = TextDocumentSyncKind.FULL;
    public HashSet<string> completion_triggers = new HashSet<string> ();
    public bool hover_provider = false;
    public bool definition_provider = false;

    // To features...
    public bool references_provider = false;
    public bool rename_provider = false;
}

public struct Iide.LspTaskInfo {
    public string server_name;
    public string message;
    public int percentage; // -1
}

public class Iide.LspClient : Object {
    private int next_id = 0;
    private Map<int, LspPromise> pending_requests = new HashMap<int, LspPromise> ();
    private Map<int, Json.Object?> responses = new HashMap<int, Json.Object?> ();
    private LspConfig config;
```

```
private DiagnosticsService diagnostics_service;

//      LSP
private GLib.Subprocess process;

//            (               )
private GLib.DataInputStream input_stream;

//            (               )
private bool is_writing = false;
private Deque<string> write_queue = new LinkedList<string> ();
private OutputStream output_stream;

//                       UI
public signal void diagnostics_received (string uri, Gee.ArrayList<IdeLspDiagnostic> dia

//
public signal void log_message (int type, string message);

//
public ServerCapabilities capabilities { get; private set; }

//    ,       ,
public signal void initialized_with_capabilities (ServerCapabilities caps);

public bool is_initialized { get; private set; default = false; }

public LspClient (LspConfig config) {
    this.capabilities = new ServerCapabilities ();
    this.config = config;
    this.diagnostics_service = DiagnosticsService.get_instance ();
}

public signal void progress_updated (string token, string message, int percentage, bool

public string name () { return config.command (); }

public int get_hash () {
    //
    return (int) ((void*) this);
}

public async Json.Object? send_request (string method, Json.Object params) throws Error
    int id = next_id--;
```

```
    // 1.
    var root = new Json.Object ();
    root.set_string_member ("jsonrpc", "2.0");
    root.set_int_member ("id", id);
    root.set_string_member ("method", method);
    root.set_object_member ("params", params);

    // 2.              (Promise)
    //        send_request.callback,      Vala
    //                      yield
    pending_requests.set (id, new LspPromise (send_request.callback));

    // 3.       (      send_message_async         )
    yield this.send_message_async (root);

    // 4.
    yield;

    // 5.
    var response = responses.get (id);
    responses.unset (id);

    return response;
}

private async void send_message_async (Json.Object node) throws Error {
    // 1.
    var generator = new Json.Generator ();
    var root_node = new Json.Node (Json.NodeType.OBJECT);
    root_node.set_object (node);
    generator.set_root (root_node);

    string body = generator.to_data (null);
    string message = "Content-Length: %d\r\n\r\n%s".printf ((int) body.length, body);

    // 2.
    write_queue.add (message);

    // 3.          -         ,
    if (is_writing)return;

    is_writing = true;

    try {
        while (!write_queue.is_empty) {
            string current_msg = write_queue.poll_head ();
```

```
            yield output_stream.write_all_async (current_msg.data, Priority.DEFAULT, nul

            yield output_stream.flush_async (Priority.DEFAULT, null);
        }
    } finally {
        is_writing = false;
    }
}

private async void run_read_loop () {
    try {
        while (true) {
            // 1.           Content-Length
            string? line = yield input_stream.read_line_async (Priority.DEFAULT, null);

            if (line == null)break; //

            if (line.has_prefix ("Content-Length: ")) {
                int length = int.parse (line.substring (16).strip ());

                // 2.                                (\r\n\r\n)
                while (line != "" && line != null) {
                    line = yield input_stream.read_line_async (Priority.DEFAULT, null);

                    if (line != null)line = line.strip ();
                }

                // 3.        JSON
                uint8[] buffer = new uint8[length + 1];
                size_t bytes_read;
                yield input_stream.read_all_async (buffer[0 : length], Priority.DEFAULT,

                buffer[length] = '\0'; //

                // 4.
                this.handle_payload ((string) buffer);
            }
        }
    } catch (Error e) {
        if (!(e is IOError.CANCELLED)) {
            debug ("LSP Read Loop Error: %s", e.message);
        }
    }
}

public async void send_response_async (Json.Node id, Json.Node? result = null) throws Er
```

```
    var root = new Json.Object ();
    root.set_string_member ("jsonrpc", "2.0");
    root.set_member ("id", id);

    //                  'result'     'method'   'params'
    if (result != null) {
        root.set_member ("result", result);
    } else {
        root.set_member ("result", new Json.Node (Json.NodeType.NULL));
    }

    var generator = new Json.Generator ();
    var root_node = new Json.Node (Json.NodeType.OBJECT);
    root_node.set_object (root);
    generator.set_root (root_node);

    yield this.send_message_async (root);
}

private void handle_payload (string payload) {
    try {
        var parser = new Json.Parser ();
        parser.load_from_data (payload);
        var root = parser.get_root ().get_object ();

        if (root.has_member ("method")) {
            string method = root.get_string_member ("method");
            if (method == "window/workDoneProgress/create" && root.has_member ("id")) {
                //              ,
                var node_id = root.get_member ("id");
                send_response_async.begin (node_id, new Json.Node (Json.NodeType.NULL));
                return;
            }
        }

        //                (   'id')
        if (root.has_member ("id")) {
            this.handle_response (root);
        }
        //                        (   'method',    'id')
        else if (root.has_member ("method")) {
            this.handle_incoming_notification (root);
        }
    } catch (Error e) {
        warning ("Failed to parse LSP payload: %s", e.message);
    }
```

```
}

private void handle_response (Json.Object response) {
    int id = (int) response.get_int_member ("id");

    if (pending_requests.has_key (id)) {
        var promise = pending_requests.get (id);
        pending_requests.unset (id);

        //
        responses.set (id, response);

        //
        //       Idle.add,            MainContext (UI    )
        Idle.add ((owned) promise.callback);
        return;
    }

    if (response.has_member ("method")) {
        var node_result = config.server_response_result (response);
        var node_id = response.get_member ("id");
        send_response_async.begin (node_id, node_result);
    }

    // if (response.has_member ("method")) {
    // string method = response.get_string_member ("method");
    // switch (method) {
    // case "client/registerCapability" : {
    // message ("!!!handle_response - client/registerCapability - " + json_object_to_str
    // var node_id = response.get_member ("id");
    // send_response_async.begin (node_id, new Json.Node (Json.NodeType.NULL));
    // } return;
    // case "workspace/configuration" : {
    // message ("!!!handle_response - workspace/configuration - " + json_object_to_strin
    // handle_response_workspace_configuration (response);
    // } return;
    // }
    // }
}

private void handle_incoming_notification (Json.Object root) {
    string method = root.get_string_member ("method");

    //                        (params)
    Json.Object? params = null;
    if (root.has_member ("params")) {
```

```
    params = root.get_object_member ("params");
}

switch (method) {
case "textDocument/publishDiagnostics" :
    if (params != null) {
        string uri = params.get_string_member ("uri");
        var json_array = params.get_array_member ("diagnostics");

        //     JSON-                   IdeLspDiagnostic
        var diag_list = this.parse_diagnostics (json_array);

        //                   UI
        Idle.add (() => {
            //
            diagnostics_service.update_diagnostics (this.get_hash (), uri, diag_list

            this.diagnostics_received (uri, diag_list);
            return Source.REMOVE;
        });
    }
    break;

case "window/workDoneProgress/create" :
    message ("!!!handle_incoming_notification - window/workDoneProgress/create" + js
    break;

case "$/progress" :
    // message ("!!!handle_incoming_notification - $/progress" + json_object_to_stri
    if (params != null) {
        // Token                   , Tree-sitter   LSP
        string token = "";
        var token_node = params.get_member ("token");
        if (token_node.get_node_type () == Json.NodeType.VALUE) {
            token = token_node.get_string ();
        } else {
            token = token_node.get_int ().to_string ();
        }

        var value = params.get_object_member ("value");
        string kind = value.get_string_member ("kind");

        //
        bool active = (kind != "end");

        //             (  BasedPyright         'message')
```

```
            string msg = "";
            if (value.has_member ("title")) {
                msg = value.get_string_member ("title");
            }
            if (value.has_member ("message")) {
                string details = value.get_string_member ("message");
                msg = (msg != "") ? @"$msg: $details" : details;
            }

            //            (     )
            int perc = value.has_member ("percentage") ? (int) value.get_int_member ("pe

            //                          UI
            Idle.add (() => {
                this.progress_updated (token, msg, perc, active);
                return Source.REMOVE;
            });
        }
        break;
    case "window/logMessage":
        if (params != null)handle_log_message (params);
        break;

    case "window/showMessage":
        //                                  GNOME
        if (params != null)debug ("LSP Show Message: %s", params.get_string_member ("mes
        break;

    default:
        debug ("Unhandled LSP notification: %s", method);
        break;
    }
}

private void handle_log_message (Json.Object params) {
    int type = (int) params.get_int_member ("type");
    string message = params.get_string_member ("message");

    Idle.add (() => {
        this.log_message (type, message);
        return Source.REMOVE;
    });
}

public async bool start_server_async (string? workspace_root) {
    try {
```

```
        // 1.
        string[] argv = { config.command () };
        foreach (var arg in config.args ())argv += arg;

        // 2.
        var launcher = new SubprocessLauncher (SubprocessFlags.STDOUT_PIPE | SubprocessF
        this.process = launcher.spawnv (argv);

        // 3.
        this.output_stream = this.process.get_stdin_pipe ();
        this.input_stream = new DataInputStream (this.process.get_stdout_pipe ());

        // 4.             (                  )
        this.run_read_loop.begin ();

        // 5.    INITIALIZE
        var init_params = config.initialize_params (workspace_root, null);
        var response = yield this.send_request ("initialize", init_params.get_object ()

        if (response != null && response.has_member ("result")) {
            var result = response.get_object_member ("result");
            //
            this.parse_capabilities (result);
        }

        //       Idle.add,
        is_initialized = true;
        Idle.add (() => {
            this.initialized_with_capabilities (this.capabilities);
            IdeLspService.get_instance ().register_client (this);
            return Source.REMOVE; //
        });

        // 6.    INITIALIZED (              )
        yield this.send_notification_async ("initialized", new Json.Object ());

        debug ("LSP Server started and initialized for: %s", workspace_root ?? "unknown"
        return true;
    } catch (Error e) {
        warning ("Failed to start LSP server: %s", e.message);
        return false;
    }
}

private void parse_capabilities (Json.Object result) {
    if (!result.has_member ("capabilities"))return;
```

```
    var caps = result.get_object_member ("capabilities");

    //
    if (caps.has_member ("textDocumentSync")) {
        var sync = caps.get_member ("textDocumentSync");
        if (sync.get_node_type () == Json.NodeType.OBJECT) {
            var sync_obj = sync.get_object ();
            if (sync_obj.has_member ("change"))
                this.capabilities.sync_kind = (TextDocumentSyncKind) sync_obj.get_int_me
        } else if (sync.get_node_type () == Json.NodeType.VALUE) {
            this.capabilities.sync_kind = (TextDocumentSyncKind) sync.get_int ();
        }
    }

    //
    if (caps.has_member ("completionProvider")) {
        var comp = caps.get_object_member ("completionProvider");
        if (comp.has_member ("triggerCharacters")) {
            var triggers = comp.get_array_member ("triggerCharacters");
            this.capabilities.completion_triggers.clear ();
            foreach (var node in triggers.get_elements ()) {
                this.capabilities.completion_triggers.add (node.get_string ());
            }
        }
    }

    // Hover & Definition
    this.capabilities.hover_provider = caps.has_member ("hoverProvider");
    this.capabilities.definition_provider = caps.has_member ("definitionProvider");
}

public async void send_notification_async (string method, Json.Object params) throws Err
    var root = new Json.Object ();
    root.set_string_member ("jsonrpc", "2.0");
    root.set_string_member ("method", method);
    root.set_object_member ("params", params);

    //
    yield this.send_message_async (root);
}

private Gee.ArrayList<IdeLspDiagnostic> parse_diagnostics (Json.Array diagnostics_array)
    var result = new Gee.ArrayList<IdeLspDiagnostic> ();

    foreach (var diag_node in diagnostics_array.get_elements ()) {
        var diag_obj = diag_node.get_object ();
```

```
        var d = new IdeLspDiagnostic ();

        //
        d.message = diag_obj.get_string_member ("message");
        if (diag_obj.has_member ("severity")) {
            d.severity = (int) diag_obj.get_int_member ("severity");
        }

        //       (Range   LSP      start   end     )
        var range = diag_obj.get_object_member ("range");
        var start = range.get_object_member ("start");
        var end = range.get_object_member ("end");

        d.start_line = (int) start.get_int_member ("line");
        d.start_column = (int) start.get_int_member ("character"); // character -> start
        d.end_line = (int) end.get_int_member ("line");
        d.end_column = (int) end.get_int_member ("character"); // character -> end_colum

        result.add (d);
    }

    return result;
}

public async void text_document_did_open (string uri, string language_id, int version, s
    var params = new Json.Object ();

    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    doc.set_string_member ("languageId", language_id);
    doc.set_int_member ("version", version);
    doc.set_string_member ("text", content);

    params.set_object_member ("textDocument", doc);

    //           (notification),           ID
    yield this.send_notification_async ("textDocument/didOpen", params);

    debug ("LSP: Sent didOpen for %s", uri);
}

public async void text_document_did_change (string uri, int version, string content) thr
    var params = new Json.Object ();

    // 1.
    var doc = new Json.Object ();
```

```
    doc.set_string_member ("uri", uri);
    doc.set_int_member ("version", version);
    params.set_object_member ("textDocument", doc);

    // 2.            (      range)
    var change = new Json.Object ();
    change.set_string_member ("text", content);

    var changes = new Json.Array ();
    changes.add_object_element (change);
    params.set_array_member ("contentChanges", changes);

    yield this.send_notification_async ("textDocument/didChange", params);
}

public async void send_did_change (string uri, int version, Gee.ArrayList<PendingChange>
    var params = new Json.Object ();

    // 1.
    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    doc.set_int_member ("version", version);
    params.set_object_member ("textDocument", doc);

    // 2.
    var json_changes = new Json.Array ();
    foreach (var c in changes) {
        var obj = new Json.Object ();

        //            (range)
        var range = new Json.Object ();

        var start = new Json.Object ();
        start.set_int_member ("line", c.start_line);
        start.set_int_member ("character", c.start_char);

        var end = new Json.Object ();
        end.set_int_member ("line", c.end_line);
        end.set_int_member ("character", c.end_char);

        range.set_object_member ("start", start);
        range.set_object_member ("end", end);

        obj.set_object_member ("range", range);
        obj.set_string_member ("text", c.text);
```

```
        json_changes.add_object_element (obj);
    }
    params.set_array_member ("contentChanges", json_changes);

    yield this.send_notification_async ("textDocument/didChange", params);
}

public async void text_document_did_close (string uri) throws Error {
    var params = new Json.Object ();
    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    params.set_object_member ("textDocument", doc);

    yield this.send_notification_async ("textDocument/didClose", params);
}

private IdeLspCompletionResult parse_completion_result (Json.Node node) {
    var res = new IdeLspCompletionResult ();
    res.items = new Gee.ArrayList<IdeLspCompletionItem> ();

    Json.Array? items_array = null;

    //     1:              CompletionList { isIncomplete, items }
    if (node.get_node_type () == Json.NodeType.OBJECT) {
        var obj = node.get_object ();
        if (obj.has_member ("isIncomplete")) {
            res.is_incomplete = obj.get_boolean_member ("isIncomplete");
        }
        if (obj.has_member ("items")) {
            items_array = obj.get_array_member ("items");
        }
    }
    //     2:                  CompletionItem[]
    else if (node.get_node_type () == Json.NodeType.ARRAY) {
        items_array = node.get_array ();
    }

    if (items_array != null) {
        foreach (var item_node in items_array.get_elements ()) {
            var item_obj = item_node.get_object ();
            var item = new IdeLspCompletionItem ();

            item.label = item_obj.get_string_member ("label");

            //
            if (item_obj.has_member ("insertText"))
```

```
                item.insert_text = item_obj.get_string_member ("insertText");
            else
                item.insert_text = item.label;

            if (item_obj.has_member ("detail"))
                item.detail = item_obj.get_string_member ("detail");

            if (item_obj.has_member ("kind"))
                item.kind = (int) item_obj.get_int_member ("kind");

            //              (                    MarkupContent)
            if (item_obj.has_member ("documentation")) {
                var doc_node = item_obj.get_member ("documentation");
                if (doc_node.get_node_type () == Json.NodeType.OBJECT)
                    item.documentation = doc_node.get_object ().get_string_member ("valu
                else
                    item.documentation = doc_node.get_string ();
            }

            res.items.add (item);
        }
    }

    return res;
}

public async IdeLspCompletionResult ? request_completion (string uri, int line, int char
    var params = new Json.Object ();

    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    params.set_object_member ("textDocument", doc);

    var pos = new Json.Object ();
    pos.set_int_member ("line", line);
    pos.set_int_member ("character", character);
    params.set_object_member ("position", pos);

    var context = new Json.Object ();
    context.set_int_member ("triggerKind", (int) trigger_kind);
    if (trigger_char != null) {
        context.set_string_member ("triggerCharacter", trigger_char);
    }
    params.set_object_member ("context", context);

    var response = yield this.send_request ("textDocument/completion", params);
```

```
    if (response == null || !response.has_member ("result"))return null;

    var result_node = response.get_member ("result");
    if (result_node.get_node_type () == Json.NodeType.NULL)return null;

    return parse_completion_result (result_node);
}

private string ? parse_hover_result (Json.Node node) {
    if (node.get_node_type () != Json.NodeType.OBJECT)
        return null;

    var result = node.get_object ();

    //     1:            'contents'
    if (!result.has_member ("contents"))
        return null;

    var contents = result.get_member ("contents");

    //           (MarkupContent)
    if (contents.get_node_type () == Json.NodeType.OBJECT) {
        var obj = contents.get_object ();
        if (obj.has_member ("value")) {
            return obj.get_string_member ("value");
        }
    }
    //
    else {
        return contents.get_string ();
    }

    return null;
}

public async string ? request_hover (string uri, int line, int character) throws Error {
    var params = new Json.Object ();

    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    params.set_object_member ("textDocument", doc);

    var pos = new Json.Object ();
    pos.set_int_member ("line", line);
    pos.set_int_member ("character", character);
```

```
    params.set_object_member ("position", pos);

    var response = yield this.send_request ("textDocument/hover", params);

    if (response == null || !response.has_member ("result"))return null;

    return parse_hover_result (response.get_member ("result"));
}

public async Gee.ArrayList<IdeLspLocation>? request_definition (string uri, int line, in
    var params = new Json.Object ();

    var doc = new Json.Object ();
    doc.set_string_member ("uri", uri);
    params.set_object_member ("textDocument", doc);

    var pos = new Json.Object ();
    pos.set_int_member ("line", line);
    pos.set_int_member ("character", character);
    params.set_object_member ("position", pos);

    var response = yield this.send_request ("textDocument/definition", params);

    if (response == null || !response.has_member ("result"))return null;

    var result_node = response.get_member ("result");
    if (result_node.get_node_type () == Json.NodeType.NULL)return null;

    return parse_definition_result (result_node);
}

private Gee.ArrayList<IdeLspLocation> parse_definition_result (Json.Node node) {
    var locations = new Gee.ArrayList<IdeLspLocation> ();

    //     1:           Location {}
    if (node.get_node_type () == Json.NodeType.OBJECT) {
        var loc = parse_single_location (node.get_object ());
        if (loc != null)locations.add (loc);
    }
    //     2:          Location[]
    else if (node.get_node_type () == Json.NodeType.ARRAY) {
        var array = node.get_array ();
        foreach (var element in array.get_elements ()) {
            var loc = parse_single_location (element.get_object ());
            if (loc != null)locations.add (loc);
        }
```

```
        }

        return locations;
    }

    private IdeLspLocation ? parse_single_location (Json.Object obj) {
        var loc = new IdeLspLocation ();
        loc.uri = obj.get_string_member ("uri");

        var range = obj.get_object_member ("range");
        var start = range.get_object_member ("start");
        var end = range.get_object_member ("end");

        loc.start_line = (int) start.get_int_member ("line");
        loc.start_column = (int) start.get_int_member ("character");
        loc.end_line = (int) end.get_int_member ("line");
        loc.end_column = (int) end.get_int_member ("character");

        return loc;
    }
}
```

File: src/Services/TreeSitter/python/PythonTSHighlighter.vala

```
[CCode (cname = "tree_sitter_python")]
extern unowned TreeSitter.Language ? get_lang_python ();

public class Iide.PythonHighlighter : BaseTreeSitterHighlighter {
    public PythonHighlighter (SourceView view) {
        base (view);
    }

    protected override unowned TreeSitter.Language get_ts_language () {
        return get_lang_python ();
    }

    protected override string get_query_filename () {
        return "python/highlights.scm";
    }

    protected override string query_source () {
        return """
        ; Identifier naming conventions

        (identifier) @variable
```

```
((identifier) @constructor
 (#match? @constructor "^[A-Z]"))

((identifier) @constant
 (#match? @constant "^[A-Z][A-Z_]*$"))

; Function calls

(decorator) @function
(decorator
  (identifier) @function)

(call
  function: (attribute attribute: (identifier) @function.method))
(call
  function: (identifier) @function)

; Builtin functions

((call
  function: (identifier) @function.builtin)
 (#match?
   @function.builtin
   "^(abs|all|any|ascii|bin|bool|breakpoint|bytearray|bytes|callable|chr|classmetho

; Function definitions

(function_definition
  name: (identifier) @function)

(attribute attribute: (identifier) @property)
(type (identifier) @type)

; Literals

[
  (none)
  (true)
  (false)
] @constant.builtin

[
  (integer)
  (float)
] @number
```

```
(comment) @comment
(string) @string
(escape_sequence) @escape

(interpolation
  "{" @punctuation.special
  "}" @punctuation.special) @embedded

[
  "-"
  "-="
  "!="
  "*"
  "**"
  "**="
  "*="
  "/"
  "//"
  "//="
  "/="
  "&"
  "&="
  "%"
  "%="
  "^"
  "^="
  "+"
  "->"
  "+="
  "<"
  "<<"
  "<<="
  "<="
  "<>"
  "="
  ":="
  "=="
  ">"
  ">="
  ">>"
  ">>="
  "|"
  "|="
  "~"
  "@="
  "and"
```

```
      "in"
      "is"
      "not"
      "or"
      "is not"
      "not in"
    ] @operators

    ["(" ")" "[" "]" "{" "}"] @punctuation.bracket

    [
      "as"
      "assert"
      "async"
      "await"
      "break"
      "class"
      "continue"
      "def"
      "del"
      "elif"
      "else"
      "except"
      "exec"
      "finally"
      "for"
      "from"
      "global"
      "if"
      "import"
      "lambda"
      "nonlocal"
      "pass"
      "print"
      "raise"
      "return"
      "try"
      "while"
      "with"
      "yield"
      "match"
      "case"
    ] @keyword
    """;
}
```

```
    protected override bool is_container_node (string node_type) {
        return node_type in new string[] {
                    "function_definition", "class_definition", "module_definition"
        };
    }
}
```

File: data/org.github.kai66673.iide.gschema.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<schemalist gettext-domain="iide">
    <schema id="org.github.kai66673.iide" path="/org/github/kai66673/iide/">
        <key name="color-scheme" type="s">
            <default>"system"</default>
            <summary>Color scheme</summary>
            <description>Color scheme: light, dark, or system</description>
        </key>

        <key name="editor-font-size" type="d">
            <default>6</default>
            <summary>Editor zoom level</summary>
            <description>Editor zoom level (1-15)</description>
        </key>

        <key name="show-minimap" type="b">
            <default>true</default>
            <summary>Show minimap</summary>
            <description>Whether to show the minimap in text editors</description>
        </key>

        <key name="show-line-numbers" type="b">
            <default>true</default>
            <summary>Show line numbers</summary>
            <description>Whether to show line numbers in text editors</description>
        </key>

        <key name="highlight-current-line" type="b">
            <default>true</default>
            <summary>Highlight current line</summary>
            <description>Whether to highlight the current line in text editors</description>
        </key>

        <key name="auto-indent" type="b">
            <default>true</default>
            <summary>Auto indent</summary>
            <description>Whether to enable auto indentation</description>
```

```
</key>

<key name="panel-start-width" type="d">
    <default>250</default>
    <summary>Start panel width</summary>
    <description>Width of the start panel (file tree)</description>
</key>

<key name="panel-end-width" type="d">
    <default>250</default>
    <summary>End panel width</summary>
    <description>Width of the end panel</description>
</key>

<key name="panel-bottom-height" type="d">
    <default>200</default>
    <summary>Bottom panel height</summary>
    <description>Height of the bottom panel (terminal)</description>
</key>

<key name="panel-bottom-width" type="d">
    <default>500</default>
    <summary>Bottom panel width</summary>
    <description>Width of the bottom panel split between terminal and log</descripti
</key>

<key name="reveal-start-panel" type="b">
    <default>true</default>
    <summary>Reveal start panel</summary>
    <description>Whether the start panel is visible</description>
</key>

<key name="reveal-end-panel" type="b">
    <default>false</default>
    <summary>Reveal end panel</summary>
    <description>Whether the end panel is visible</description>
</key>

<key name="reveal-bottom-panel" type="b">
    <default>false</default>
    <summary>Reveal bottom panel</summary>
    <description>Whether the bottom panel is visible</description>
</key>

<key name="recent-projects" type="as">
    <default>[]</default>
```

```
    <summary>Recent projects</summary>
    <description>List of recently opened project paths</description>
</key>

<key name="max-recent-projects" type="d">
    <default>10</default>
    <summary>Maximum recent projects</summary>
    <description>Maximum number of recent projects to remember</description>
</key>

<key name="last-open-directory" type="s">
    <default>""</default>
    <summary>Last open directory</summary>
    <description>Last directory used for opening files or projects</description>
</key>

<key name="current-project-path" type="s">
    <default>""</default>
    <summary>Current project path</summary>
    <description>Path to the currently open project</description>
</key>

<key name="open-documents" type="as">
    <default>[]</default>
    <summary>Open documents</summary>
    <description>List of currently open document URIs</description>
</key>

<key name="panel-layout" type="s">
    <default>""</default>
    <summary>Panel layout</summary>
    <description>JSON string describing panel dock layout</description>
</key>

<key name="grid-layout" type="s">
    <default>""</default>
    <summary>Grid layout</summary>
    <description>JSON string describing grid layout</description>
</key>

<key name="window-width" type="d">
    <default>1200</default>
    <summary>Window width</summary>
    <description>Width of the main window</description>
</key>
```

```
        <key name="window-height" type="d">
            <default>800</default>
            <summary>Window height</summary>
            <description>Height of the main window</description>
        </key>

        <key name="window-maximized" type="b">
            <default>false</default>
            <summary>Window maximized</summary>
            <description>Whether the window is maximized</description>
        </key>

        <key name="shortcuts" type="s">
            <default>""</default>
            <summary>Custom shortcuts</summary>
            <description>JSON string mapping action IDs to keyboard shortcuts</description>
        </key>
    </schema>
</schemalist>
```

File: src/Services/ProjectManager.vala

```
using Gtk;
using GLib;
using Gee;

public class Iide.LanguageConfig : GLib.Object {
    public string language_id { get; construct set; }
    public string[] server_command { get; construct set; }
    public string[] file_patterns { get; construct set; }

    public LanguageConfig (string language_id, string[] server_command, string[] file_patter
        Object (
            language_id: language_id,
            server_command: server_command,
            file_patterns: file_patterns
        );
    }
}

public class Iide.ProjectManager : Object {
    private static ProjectManager? _instance;
    private GLib.File? current_project_root;
    private string? current_project_name;
    private Iide.SettingsService settings;
    private Gee.HashMap<string, LanguageConfig> language_configs;
```

```
public signal void project_opened (GLib.File project_root);
public signal void project_closed ();

public static unowned ProjectManager get_instance () {
    if (_instance == null) {
        _instance = new ProjectManager ();
    }
    return _instance;
}

public ProjectManager () {
    current_project_root = null;
    current_project_name = null;
    settings = Iide.SettingsService.get_instance ();
    language_configs = new Gee.HashMap<string, LanguageConfig> ();
    init_default_language_configs ();
}

private void init_default_language_configs () {
    language_configs.set ("c", new LanguageConfig ("c", { "clangd" }, { "*.c", "*.h" }))
    language_configs.set ("cpp", new LanguageConfig ("cpp", { "clangd" }, { "*.cpp", "*.
    language_configs.set ("python", new LanguageConfig ("python", { "basedpyright-langse
    language_configs.set ("rust", new LanguageConfig ("rust", { "rust-analyzer" }, { "*.
    language_configs.set ("go", new LanguageConfig ("go", { "gopls" }, { "*.go" }));
    language_configs.set ("typescript", new LanguageConfig ("typescript", { "typescript-
    language_configs.set ("javascript", new LanguageConfig ("javascript", { "typescript-
    language_configs.set ("json", new LanguageConfig ("json", { "vscode-json-languageser
    language_configs.set ("html", new LanguageConfig ("html", { "vscode-html-language-se
    language_configs.set ("css", new LanguageConfig ("css", { "vscode-css-language-serve
}

private string? last_loaded_config_path;

public Gee.Collection<LanguageConfig> get_language_configs () {
    return language_configs.values;
}

public LanguageConfig? get_language_config (string language_id) {
    return language_configs.get (language_id);
}

public void load_lsp_config (GLib.File project_root) {
    string? config_path = project_root.get_path ();
    if (config_path == null) {
        config_path = project_root.get_uri ();
```

```
}

if (config_path != null && config_path == last_loaded_config_path) {
    return;
}

init_default_language_configs ();

var config_dir = project_root.get_child (".iide");
var lsp_file = config_dir.get_child ("lsp.json");

if (!lsp_file.query_exists (null)) {
    message ("No .iide/lsp.json found at %s", lsp_file.get_path ());
    last_loaded_config_path = config_path;
    return;
}

try {
    var parser = new Json.Parser ();
    parser.load_from_file (lsp_file.get_path ());
    var root = parser.get_root ();
    var obj = (Json.Object) root;

    if (obj.has_member ("languages")) {
        var languages = obj.get_array_member ("languages");
        int lang_count = (int) languages.get_length ();
        for (int j = 0; j < lang_count; j++) {
            var node = languages.get_element (j);
            var lang_obj = node.get_object ();
            if (lang_obj == null) continue;

            string? lang_id = null;
            string[]? server_cmd = null;
            string[]? patterns = null;

            if (lang_obj.has_member ("id")) {
                lang_id = lang_obj.get_string_member ("id");
            }
            if (lang_obj.has_member ("server")) {
                var server_arr = lang_obj.get_array_member ("server");
                int server_count = (int) server_arr.get_length ();
                server_cmd = new string[server_count];
                for (int i = 0; i < server_count; i++) {
                    server_cmd[i] = server_arr.get_string_element (i);
                }
            }
```

```
                if (lang_obj.has_member ("patterns")) {
                    var patterns_arr = lang_obj.get_array_member ("patterns");
                    int patterns_count = (int) patterns_arr.get_length ();
                    patterns = new string[patterns_count];
                    for (int i = 0; i < patterns_count; i++) {
                        patterns[i] = patterns_arr.get_string_element (i);
                    }
                }

                if (lang_id != null) {
                    merge_language_config (lang_id, server_cmd, patterns);
                }
            }
        }

        message ("Loaded LSP config from %s", lsp_file.get_path ());
        last_loaded_config_path = config_path;
    } catch (Error e) {
        warning ("Failed to load LSP config: %s", e.message);
    }
}

private void merge_language_config (string lang_id, string[]? server_cmd, string[]? patt
    var config = language_configs.get (lang_id);
    if (config != null) {
        if (server_cmd != null && server_cmd.length > 0) {
            config.server_command = server_cmd;
        }
        if (patterns != null && patterns.length > 0) {
            config.file_patterns = patterns;
        }
        message ("Merged LSP config for '%s'", lang_id);
        return;
    }

    if (server_cmd != null && patterns != null) {
        language_configs.set (lang_id, new LanguageConfig (lang_id, server_cmd, patterns
        message ("Added new LSP config for '%s'", lang_id);
    }
}

public async void open_project_async (GLib.File project_root) {
    if (!project_root.query_exists (null)) {
        stderr.printf ("Project directory does not exist: %s\n", project_root.get_path (
        return;
    }
```

```
    if (current_project_root != null) {
        close_project ();
    }

    init_default_language_configs ();

    current_project_root = project_root;
    current_project_name = project_root.get_basename ();

    settings.current_project_path = project_root.get_path ();
    settings.add_recent_project (project_root.get_path ());
    settings.last_open_directory = project_root.get_parent ().get_path ();

    load_lsp_config (project_root);

    project_opened (project_root);
}

public void close_project () {
    if (current_project_root != null) {
        current_project_root = null;
        current_project_name = null;
        settings.current_project_path = "";
        project_closed ();
    }
}

public string? get_workspace_root_path () {
    if (current_project_root != null) {
        return current_project_root.get_path ();
    }
    return null;
}

public void open_project_by_path (string path) {
    if (path != null && path != "") {
        var file = GLib.File.new_for_path (path);
        if (file.query_exists (null)) {
            open_project_async.begin (file);
        }
    }
}

public GLib.File? get_current_project_root () {
    return current_project_root;
```

```
    }

    public string? get_current_project_name () {
        return current_project_name;
    }

    public bool has_open_project () {
        return current_project_root != null;
    }

    public async void open_project_dialog (Window parent_window) {
        var dialog = new FileDialog () {
            title = _("Open Project"),
            modal = true
        };

        var last_dir = settings.last_open_directory;
        if (last_dir != null && last_dir != "") {
            dialog.initial_folder = GLib.File.new_for_path (last_dir);
        }

        try {
            var file = yield dialog.select_folder (parent_window, null);
            if (file != null) {
                yield open_project_async (file);
            }
        } catch {
            // User dismissed dialog or other error - silently ignore
        }
    }
}
```

File: src/Services/TreeSitter/TreeSitterManager.vala

```
// [CCode (cname = "tree_sitter_rust")]
// extern unowned TreeSitter.Language ? get_language_rust ();
// [CCode (cname = "tree_sitter_c")]
// extern unowned TreeSitter.Language ? get_language_c ();
// [CCode (cname = "tree_sitter_javascript")]
// extern unowned TreeSitter.Language ? get_language_javascript ();
// [CCode (cname = "tree_sitter_python")]
// extern unowned TreeSitter.Language ? get_language_python ();
// [CCode (cname = "tree_sitter_ruby")]
// extern unowned TreeSitter.Language ? get_language_ruby ();
// [CCode(cname = "tree_sitter_cpp")]
// extern unowned TreeSitter.Language ? get_language_cpp();
```

```
// [CCode(cname = "tree_sitter_vala")]
// extern unowned TreeSitter.Language ? get_language_vala();
// [CCode (cname = "tree_sitter_go")]
// extern unowned TreeSitter.Language ? get_language_go ();
// [CCode (cname = "tree_sitter_bash")]
// extern unowned TreeSitter.Language ? get_language_bash ();
// [CCode (cname = "tree_sitter_json")]
// extern unowned TreeSitter.Language ? get_language_json ();
// [CCode (cname = "tree_sitter_php")]
// extern unowned TreeSitter.Language ? get_language_php ();
// [CCode (cname = "tree_sitter_html")]
// extern unowned TreeSitter.Language ? get_language_html ();
// [CCode (cname = "tree_sitter_xml")]
// extern unowned TreeSitter.Language ? get_language_xml ();
// [CCode (cname = "tree_sitter_typescript")]
// extern unowned TreeSitter.Language ? get_language_typescript ();
// [CCode (cname = "tree_sitter_yaml")]
// extern unowned TreeSitter.Language ? get_language_yaml ();



class Iide.TreeSitterManager : GLib.Object {
    public BaseTreeSitterHighlighter ? get_ts_highlighter(SourceView view) {
        var language_name = ((GtkSource.Buffer) view.buffer).language.name.down();
        message("LANG Detected: " + language_name);
        switch (language_name) {
        case "cpp" :
            return new CppHighlighter(view);
        case "python":
            return new PythonHighlighter(view);
        case "rust":
            return new RustHighlighter(view);
        case "vala":
            return new ValaHighlighter(view);
        }
        return null;
    }

    // public unowned TreeSitter.Language? get_ts_language (GtkSource.Buffer buffer) {
    // var language_name = buffer.language.name.down ();
    // unowned TreeSitter.Language? language = null;
    // switch (language_name) {
    // case "bash" :
    // language = get_language_bash ();
    // break;
    // case "c" :
```

```
    // language = get_language_c ();
    // break;
    // case "cpp" :
    // language = get_language_cpp ();
    // break;
    // case "go" :
    // language = get_language_go ();
    // break;
    // case "javascript" :
    // language = get_language_javascript ();
    // break;
    // case "json" :
    // language = get_language_json ();
    // break;
    // case "html" :
    // language = get_language_html ();
    // break;
    // case "php" :
    // language = get_language_php ();
    // break;
    // case "python" :
    // language = get_language_python ();
    // break;
    // case "ruby" :
    // language = get_language_ruby ();
    // break;
    // case "rust" :
    // language = get_language_rust ();
    // break;
    // case "typescript" :
    // language = get_language_typescript ();
    // break;
    // case "xml" :
    // language = get_language_xml ();
    // break;
    // case "yaml" :
    // language = get_language_yaml ();
    // break;
    // }
    // return language;
    // }
}
```

File: src/Services/SettingsService.vala

```
/*
```

```
 * settingsservice.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

public enum ColorScheme {
    SYSTEM,
    LIGHT,
    DARK;

    public static ColorScheme from_string (string value) {
        switch (value) {
        case "light":
            return LIGHT;
        case "dark":
            return DARK;
        default:
            return SYSTEM;
        }
    }

    public string to_string () {
        switch (this) {
        case LIGHT:
            return "light";
        case DARK:
            return "dark";
        default:
            return "system";
        }
    }
```

```
    public Adw.ColorScheme to_adw_color_scheme () {
        switch (this) {
        case LIGHT:
            return Adw.ColorScheme.FORCE_LIGHT;
        case DARK:
            return Adw.ColorScheme.FORCE_DARK;
        default:
            return Adw.ColorScheme.DEFAULT;
        }
    }
}

public class FontSizeHelper : Object {
    public const int DEFAULT_ZOOM_LEVEL = 6;
    public const int MIN_ZOOM_LEVEL = 1;
    public const int MAX_ZOOM_LEVEL = 15;
    private const int[] FONT_SIZES = { 4, 6, 8, 10, 12, 14, 16, 18, 20, 24, 28, 32, 36, 40,

    public static int get_size_for_zoom_level (int zoom_level) {
        if (zoom_level >= MIN_ZOOM_LEVEL && zoom_level <= MAX_ZOOM_LEVEL) {
            return FONT_SIZES[zoom_level - 1];
        }
        return FONT_SIZES[DEFAULT_ZOOM_LEVEL - 1];
    }

    public static int[] get_available_sizes () {
        return FONT_SIZES;
    }
}

public class Iide.SettingsService : Object {
    private static SettingsService? _instance;
    private Settings settings;

    public signal void editor_setting_changed (string key);

    public static SettingsService get_instance () {
        if (_instance == null) {
            _instance = new SettingsService ();
        }
        return _instance;
    }

    private SettingsService () {
        settings = new Settings ("org.github.kai66673.iide");
```

```
}

public ColorScheme color_scheme {
    get {
        return ColorScheme.from_string (settings.get_string ("color-scheme"));
    }
    set {
        settings.set_string ("color-scheme", value.to_string ());
        editor_setting_changed ("color-scheme");
    }
}

public int editor_font_size {
    get {
        return (int) settings.get_double ("editor-font-size");
    }
    set {
        settings.set_double ("editor-font-size", (double) value);
        editor_setting_changed ("editor-font-size");
    }
}

public bool show_minimap {
    get {
        return settings.get_boolean ("show-minimap");
    }
    set {
        settings.set_boolean ("show-minimap", value);
        editor_setting_changed ("show-minimap");
    }
}

public bool show_line_numbers {
    get {
        return settings.get_boolean ("show-line-numbers");
    }
    set {
        settings.set_boolean ("show-line-numbers", value);
        editor_setting_changed ("show-line-numbers");
    }
}

public bool highlight_current_line {
    get {
        return settings.get_boolean ("highlight-current-line");
    }
```

```
    set {
        settings.set_boolean ("highlight-current-line", value);
        editor_setting_changed ("highlight-current-line");
    }
}

public bool auto_indent {
    get {
        return settings.get_boolean ("auto-indent");
    }
    set {
        settings.set_boolean ("auto-indent", value);
        editor_setting_changed ("auto-indent");
    }
}

public int panel_start_width {
    get {
        return (int) settings.get_double ("panel-start-width");
    }
    set {
        settings.set_double ("panel-start-width", (double) value);
    }
}

public int panel_end_width {
    get {
        return (int) settings.get_double ("panel-end-width");
    }
    set {
        settings.set_double ("panel-end-width", (double) value);
    }
}

public int panel_bottom_height {
    get {
        return (int) settings.get_double ("panel-bottom-height");
    }
    set {
        settings.set_double ("panel-bottom-height", (double) value);
    }
}

public int panel_bottom_width {
    get {
        return (int) settings.get_double ("panel-bottom-width");
```

```
    }
    set {
        settings.set_double ("panel-bottom-width", (double) value);
    }
}

public bool reveal_start_panel {
    get {
        return settings.get_boolean ("reveal-start-panel");
    }
    set {
        settings.set_boolean ("reveal-start-panel", value);
    }
}

public bool reveal_end_panel {
    get {
        return settings.get_boolean ("reveal-end-panel");
    }
    set {
        settings.set_boolean ("reveal-end-panel", value);
    }
}

public bool reveal_bottom_panel {
    get {
        return settings.get_boolean ("reveal-bottom-panel");
    }
    set {
        settings.set_boolean ("reveal-bottom-panel", value);
    }
}

public string[] recent_projects {
    owned get {
        return settings.get_strv ("recent-projects");
    }
}

public int max_recent_projects {
    get {
        return (int) settings.get_double ("max-recent-projects");
    }
    set {
        settings.set_double ("max-recent-projects", (double) value);
    }
```

```
}

public string last_open_directory {
    owned get {
        return settings.get_string ("last-open-directory");
    }
    set {
        settings.set_string ("last-open-directory", value);
    }
}

public string current_project_path {
    owned get {
        return settings.get_string ("current-project-path");
    }
    set {
        settings.set_string ("current-project-path", value);
    }
}

public Gee.ArrayList<string> open_documents {
    owned get {
        var arr = settings.get_strv ("open-documents");
        var list = new Gee.ArrayList<string> ();
        foreach (var s in arr) {
            list.add (s);
        }
        return (owned) list;
    }
    set {
        var arr = new string[value.size];
        int i = 0;
        foreach (var s in value) {
            arr[i++] = s;
        }
        settings.set_strv ("open-documents", arr);
    }
}

public string panel_layout {
    owned get {
        return settings.get_string ("panel-layout");
    }
    set {
        settings.set_string ("panel-layout", value);
    }
```

```
}

public string grid_layout {
    owned get {
        return settings.get_string ("grid-layout");
    }
    set {
        settings.set_string ("grid-layout", value);
        Settings.sync ();
    }
}

public int window_width {
    get {
        return (int) settings.get_double ("window-width");
    }
    set {
        settings.set_double ("window-width", (double) value);
    }
}

public int window_height {
    get {
        return (int) settings.get_double ("window-height");
    }
    set {
        settings.set_double ("window-height", (double) value);
    }
}

public bool window_maximized {
    get {
        return settings.get_boolean ("window-maximized");
    }
    set {
        settings.set_boolean ("window-maximized", value);
    }
}

public void add_recent_project (string path) {
    var projects = new List<string> ();
    projects.prepend (path);

    foreach (var p in recent_projects) {
        if (p != path) {
            bool found = false;
```

```
                foreach (var existing in projects) {
                    if (existing == path) {
                        found = true;
                        break;
                    }
                }
                if (!found) {
                    projects.append (p);
                }
            }
        }

        var arr = new string[projects.length ()];
        int i = 0;
        foreach (var p in projects) {
            arr[i++] = p;
        }
        settings.set_strv ("recent-projects", arr);
    }

    public void remove_recent_project (string path) {
        var projects = new List<string> ();
        foreach (var p in recent_projects) {
            if (p != path) {
                projects.append (p);
            }
        }

        var arr = new string[projects.length ()];
        int i = 0;
        foreach (var p in projects) {
            arr[i++] = p;
        }
        settings.set_strv ("recent-projects", arr);
    }

    public void clear_recent_projects () {
        settings.set_strv ("recent-projects", {});
    }

    public signal void setting_changed (string key);
}
```

File: vapi/libtreesitter.vapi

```
[CCode (cheader_filename = "tree_sitter/api.h")]
```

```
namespace TreeSitter {

    [CCode (cname = "TREE_SITTER_LANGUAGE_VERSION")]
    public const int LANGUAGE_VERSION;

    [CCode (cname = "TREE_SITTER_MIN_COMPATIBLE_LANGUAGE_VERSION")]
    public const int MIN_COMPATIBLE_LANGUAGE_VERSION;

    [CCode (cname = "TSInputEncoding", cprefix = "TSInputEncoding", has_type_id = false)]
    public enum InputEncoding {
        UTF8,
        UTF16
    }

    [CCode (cname = "TSSymbolType", cprefix = "TSSymbolType", has_type_id = false)]
    public enum SymbolType {
        Regular,
        Anonymous,
        Auxiliary
    }

    [CCode (cname = "TSLogType", cprefix = "TSLogType", has_type_id = false)]
    public enum LogType {
        Parse,
        Lex
    }

    [SimpleType]
    [CCode (cname = "TSSymbol", has_type_id = false)]
    public struct Symbol : uint16 {
    }

    [CCode (cname = "TSLanguage")]
    [Compact]
    public class Language {
        uint32 version;
        uint32 symbol_count;
        uint32 alias_count;
        uint32 token_count;
        uint32 external_token_count;
        // const char **symbol_names;
        // const TSSymbolMetadata *symbol_metadata;
        // const uint16 *parse_table;
        // const TSParseActionEntry *parse_actions;
        // const TSLexMode *lex_modes;
        // const TSSymbol *alias_sequences;
```

```
        uint16 max_alias_sequence_length;
        // bool (*lex_fn)(TSLexer *, TSStateId);
        // bool (*keyword_lex_fn)(TSLexer *, TSStateId);
        // Symbol keyword_capture_token;
/*
   struct {
    const bool *states;
    const TSSymbol *symbol_map;
    void *(*create)();
    void (*destroy)(void *);
    bool (*scan)(void *, TSLexer *, const bool *symbol_whitelist);
    unsigned (*serialize)(void *, char *);
    void (*deserialize)(void *, const char *, unsigned);
   } external_scanner;
 */
        [CCode (cname = "ts_language_symbol_count")]
        public uint32 get_symbol_count ();

        [CCode (cname = "ts_language_symbol_name")]
        public unowned string get_symbol_name (Symbol symbol);

        [CCode (cname = "ts_language_symbol_for_name")]
        public Symbol symbol_for_name (string name);

        [CCode (cname = "ts_language_symbol_type")]
        public SymbolType get_symbol_type (Symbol symbol);

        [CCode (cname = "ts_language_abi_version")]
        public uint32 get_version ();
    }

    [CCode (cname = "TSParser", free_function = "ts_parser_delete")]
    [Compact]
    public class Parser {
        [CCode (cname = "ts_parser_new")]
        public Parser ();

        [CCode (cname = "ts_parser_language")]
        public unowned Language get_language ();

        [CCode (cname = "ts_parser_set_language")]
        public bool set_language (Language language);

        [CCode (cname = "ts_parser_parse_string", array_length_type = "uint32_t")]
        public Tree parse_string (Tree? tree, uint8[] source);
    }
```

```
[CCode (cname = "TSTree", free_function = "ts_tree_delete")]
[Compact]
public class Tree {
    [CCode (cname = "ts_tree_root_node")]
    public Node root_node ();

    [CCode (cname = "ts_tree_edit")]
    public void edit (InputEdit edit);

    [CCode (cname = "ts_tree_get_changed_ranges")]
    public Range[] get_changed_ranges (Tree new_tree);
}

[SimpleType]
[CCode (cname = "TSPoint", has_type_id = false)]
public struct Point {
    uint32 row;
    uint32 column;
}

[CCode (cname = "TSRange", has_type_id = false)]
public struct Range {
    Point start_point;
    Point end_point;
    uint32 start_byte;
    uint32 end_byte;
}

[CCode (cname = "TSInput", has_type_id = false)]
public struct Input {
    void* payload;
    // const char *(*read)(void *payload, uint32 byte_index, Point position, uint32 *byt
    InputEncoding encoding;
}

[CCode (cname = "TSLogger", has_type_id = false)]
public struct Logger {
    void* payload;
    // void (*log)(void *payload, TSLogType, const char *);
}

[CCode (cname = "TSInputEdit", has_type_id = false)]
public struct InputEdit {
    uint32 start_byte;
    uint32 old_end_byte;
```

```
    uint32 new_end_byte;
    Point start_point;
    Point old_end_point;
    Point new_end_point;
}

[SimpleType]
[CCode (cname = "TSNode", has_type_id = false)]
public struct Node {

    [CCode (cname = "ts_node_type")]
    public unowned string type ();

    [CCode (cname = "ts_node_string")]
    public string to_str ();

    [CCode (cname = "ts_node_start_point")]
    public Point start_point ();

    [CCode (cname = "ts_node_end_point")]
    public Point end_point ();

    [CCode (cname = "ts_node_start_byte")]
    public uint32 start_byte ();

    [CCode (cname = "ts_node_end_byte")]
    public uint32 end_byte ();

    [CCode (cname = "ts_node_child_count")]
    public uint32 child_count ();

    [CCode (cname = "ts_node_child")]
    public unowned Node child (uint32 index);

    [CCode (cname = "ts_node_descendant_for_byte_range")]
    public Node descendant_for_byte_range (uint32 start, uint32 end);

    [CCode (cname = "ts_node_named_descendant_for_byte_range")]
    public Node named_descendant_for_byte_range (uint32 start, uint32 end);

    [CCode (cname = "ts_node_parent")]
    public Node parent ();

    [CCode (cname = "ts_node_is_null")]
    public bool is_null ();
```

```
    [CCode (cname = "ts_node_child_by_field_name")]
    public Node child_by_field_name (string field_name, uint32 name_len = 0);
}

[CCode (cname = "TSTreeCursor", free_function = "ts_tree_cursor_delete", has_type_id = f
[Compact]
public class TreeCursor {
    [CCode (cname = "ts_tree_cursor_goto_next_sibling")]
    public bool goto_next_sibling ();

    [CCode (cname = "ts_tree_cursor_goto_first_child")]
    public bool goto_first_child ();

    [CCode (cname = "ts_tree_cursor_current_node")]
    public Node current_node ();

    [CCode (cname = "ts_tree_cursor_goto_parent")]
    public bool goto_parent ();
}

[CCode (cname = "TSQueryError", cprefix = "TSQueryError", has_type_id = false)]
public enum QueryError {
    None,
    Syntax,
    NodeType,
    Field,
    Capture,
    Structure,
    Language
}

[CCode (cname = "TSQuery", free_function = "ts_query_delete")]
[Compact]
public class Query {
    [CCode (cname = "ts_query_new")]
    public Query (Language language, string source, uint32 source_len, out uint32 error_

    [CCode (cname = "ts_query_capture_count")]
    public uint32 capture_count ();

    [CCode (cname = "ts_query_capture_name_for_id", array_length = false)]
    public unowned string capture_name_for_id (uint32 id, out uint32 length);

    [CCode (cname = "ts_query_string_count")]
    public uint32 string_count ();
}
```

```
    [CCode (cname = "TSQueryMatch", has_type_id = false, destroy_function = "")]
    public struct QueryMatch {
        public uint32 id;
        public uint16 pattern_index;
        public uint16 capture_count;
        [CCode (array_length_cname = "capture_count")]
        public QueryCapture[] captures;
    }

    [CCode (cname = "TSQueryCapture", has_type_id = false)]
    public struct QueryCapture {
        public Node node;
        public uint32 index;
    }

    [CCode (cname = "TSQueryCursor", free_function = "ts_query_cursor_delete")]
    [Compact]
    public class QueryCursor {
        [CCode (cname = "ts_query_cursor_new")]
        public QueryCursor ();

        [CCode (cname = "ts_query_cursor_exec")]
        public void exec (Query query, Node node);

        [CCode (cname = "ts_query_cursor_next_match")]
        public bool next_match (out QueryMatch match);

        [CCode (cname = "ts_query_cursor_set_byte_range")]
        public void set_byte_range (uint32 start, uint32 end);
    }
}

/*

   TSLogger ts_parser_logger(const TSParser *);
   void ts_parser_set_logger(TSParser *, TSLogger);
   void ts_parser_print_dot_graphs(TSParser *, FILE *);
   void ts_parser_halt_on_error(TSParser *, bool);
   TSTree *ts_parser_parse(TSParser *, const TSTree *, TSInput);
   bool ts_parser_enabled(const TSParser *);
   void ts_parser_set_enabled(TSParser *, bool);
   size_t ts_parser_operation_limit(const TSParser *);
   void ts_parser_set_operation_limit(TSParser *, size_t);
   void ts_parser_reset(TSParser *);
   void ts_parser_set_included_ranges(TSParser *, const TSRange *, uint32_t);
```

```
  const TSRange *ts_parser_included_ranges(const TSParser *, uint32_t *);

  TSTree *ts_tree_copy(const TSTree *);
  void ts_tree_edit(TSTree *, const TSInputEdit *);
  TSRange *ts_tree_get_changed_ranges(const TSTree *, const TSTree *, uint32_t *);
  void ts_tree_print_dot_graph(const TSTree *, FILE *);
  const TSLanguage *ts_tree_language(const TSTree *);

  TSSymbol ts_node_symbol(TSNode);
  char *ts_node_string(TSNode);
  bool ts_node_eq(TSNode, TSNode);
  bool ts_node_is_null(TSNode);
  bool ts_node_is_named(TSNode);
  bool ts_node_is_missing(TSNode);
  bool ts_node_has_changes(TSNode);
  bool ts_node_has_error(TSNode);
  TSNode ts_node_parent(TSNode);
  TSNode ts_node_child(TSNode, uint32_t);
  TSNode ts_node_named_child(TSNode, uint32_t);
  uint32_t ts_node_child_count(TSNode);
  uint32_t ts_node_named_child_count(TSNode);
  TSNode ts_node_next_sibling(TSNode);
  TSNode ts_node_next_named_sibling(TSNode);
  TSNode ts_node_prev_sibling(TSNode);
  TSNode ts_node_prev_named_sibling(TSNode);
  TSNode ts_node_first_child_for_byte(TSNode, uint32_t);
  TSNode ts_node_first_named_child_for_byte(TSNode, uint32_t);
  TSNode ts_node_descendant_for_byte_range(TSNode, uint32_t, uint32_t);
  TSNode ts_node_named_descendant_for_byte_range(TSNode, uint32_t, uint32_t);
  TSNode ts_node_descendant_for_point_range(TSNode, TSPoint, TSPoint);
  TSNode ts_node_named_descendant_for_point_range(TSNode, TSPoint, TSPoint);
  void ts_node_edit(TSNode *, const TSInputEdit *);

  void ts_tree_cursor_delete(TSTreeCursor *);
  void ts_tree_cursor_reset(TSTreeCursor *, TSNode);
  int64_t ts_tree_cursor_goto_first_child_for_byte(TSTreeCursor *, uint32_t);

 */
```

File: meson.build

```
project(
    'iide',
    ['c', 'vala'],
    version: '0.1.0',
    meson_version: '>= 1.0.0',
```

```
    default_options: [
        'warning_level=2',
        'werror=false',
    ],
)

add_project_arguments('-DGETTEXT_PACKAGE="' + meson.project_name() + '"', language: 'c')

tree_sitter_sources = [
    'vendor/tree-sitter/lib/src/lib.c',
    'parsers/ts_helper.c',
    'parsers/bash/src/parser.c',
    'parsers/bash/src/scanner.c',
    'parsers/json/src/parser.c',
    'parsers/cpp/src/parser.c',
    'parsers/cpp/src/scanner.c',
    'parsers/go/src/parser.c',
    'parsers/html/src/parser.c',
    'parsers/html/src/scanner.c',
    'parsers/javascript/src/parser.c',
    'parsers/javascript/src/scanner.c',
    'parsers/python/src/parser.c',
    'parsers/python/src/scanner.c',
    'parsers/ruby/src/parser.c',
    'parsers/ruby/src/scanner.c',
    'parsers/rust/src/parser.c',
    'parsers/rust/src/scanner.c',
    'parsers/yaml/src/parser.c',
    'parsers/yaml/src/scanner.c',
    'parsers/vala/src/parser.c',
]

tree_sitter_lib = static_library(
    'tree-sitter-runtime',
    tree_sitter_sources,
    include_directories: [
        include_directories('vendor/tree-sitter/lib/src'),
        include_directories('vendor/tree-sitter/lib/include'),
        include_directories('parsers/vala/src'),
    ],
)

tree_sitter_vapi = meson.get_compiler('vala').find_library('libtreesitter', dirs: join_paths

tree_sitter_xml_sources = [
    'parsers/xml/xml/src/parser.c',
```

```
    'parsers/xml/xml/src/scanner.c',
]

tree_sitter_xml = static_library(
    'tree-sitter-xml',
    tree_sitter_xml_sources,
    include_directories: [
        include_directories('parsers/xml/xml/src'),
    ],
)

tree_sitter_c_sources = [
    'parsers/c/src/parser.c',
]

tree_sitter_c = static_library(
    'tree-sitter-c',
    tree_sitter_c_sources,
    include_directories: [
        include_directories('parsers/c/src'),
    ],
)

tree_sitter_typescript_sources = [
    'parsers/typescript/typescript/src/parser.c',
    'parsers/typescript/typescript/src/scanner.c',
]

tree_sitter_typescript = static_library(
    'tree-sitter-typescript',
    tree_sitter_typescript_sources,
    include_directories: [
        include_directories('parsers/typescript/typescript/src'),
    ],
)

tree_sitter_php_sources = [
    'parsers/php/php/src/parser.c',
    'parsers/php/php/src/scanner.c',
]

tree_sitter_php = static_library(
    'tree-sitter-php',
    tree_sitter_php_sources,
    include_directories: [
        include_directories('parsers/php/php/src'),
```

```
    ],
)

tree_sitter_dep = declare_dependency(
    link_with: [
        tree_sitter_lib,
        tree_sitter_xml,
        tree_sitter_c,
        tree_sitter_typescript,
        tree_sitter_php,
    ],
    include_directories: include_directories(join_paths('vendor', 'tree-sitter', 'lib', 'inc
    dependencies: tree_sitter_vapi,
)

i18n = import('i18n')
gnome = import('gnome')
valac = meson.get_compiler('vala')

srcdir = meson.project_source_root() / 'src'

config_h = configuration_data()
config_h.set_quoted('PACKAGE_VERSION', meson.project_version())
config_h.set_quoted('GETTEXT_PACKAGE', 'iide')
config_h.set_quoted('LOCALEDIR', get_option('prefix') / get_option('localedir'))
configure_file(output: 'config.h', configuration: config_h)

config_dep = valac.find_library('config', dirs: srcdir)
config_inc = include_directories('.')

subdir('data')
subdir('src')
subdir('po')

gnome.post_install(
    glib_compile_schemas: true,
    gtk_update_icon_cache: true,
    update_desktop_database: true,
)

local_settings_conf = configuration_data()
local_settings_conf.set('PREFIX', get_option('prefix'))
local_settings_sh = configure_file(
    input: 'scripts/install-local-settings.sh.in',
    output: 'install-local-settings.sh',
    configuration: local_settings_conf,
```

```
)
meson.add_install_script(local_settings_sh)
```

File: src/Widgets/TextView/FontZoomer.vala

```
using Gtk;
using GtkSource;
using GLib;

public class FontZoomer : Object {
    private View src_view;
    private int zoom_level;
    private Iide.SettingsService settings;
    public signal void zoom_changed(int level);

    public FontZoomer(View src_view) {
        this.src_view = src_view;
        this.settings = Iide.SettingsService.get_instance();

        if (!this.src_view.has_css_class("text-view")) {
            this.src_view.add_css_class("text-view");
        }

        this.zoom_level = settings.editor_font_size;
        if (this.zoom_level < FontSizeHelper.MIN_ZOOM_LEVEL || this.zoom_level > FontSizeHel
            this.zoom_level = FontSizeHelper.DEFAULT_ZOOM_LEVEL;
        }
        this.src_view.add_css_class("zoom-" + zoom_level.to_string());

        var scroll_controller = new EventControllerScroll(EventControllerScrollFlags.VERTICA
        scroll_controller.scroll.connect((dx, dy) => {
            var event = scroll_controller.get_current_event();
            if (event == null)return false;

            var modifiers = event.get_modifier_state();
            if ((modifiers & Gdk.ModifierType.CONTROL_MASK) != 0) {
                if (dy < 0) {
                    zoom_in();
                } else if (dy > 0) {
                    zoom_out();
                }
                return true;
            }
            return false;
        });
```

```
    this.src_view.add_controller(scroll_controller);
}

public void zoom_in() {
    if (zoom_level < FontSizeHelper.MAX_ZOOM_LEVEL) {
        src_view.remove_css_class("zoom-" + zoom_level.to_string());
        zoom_level++;
        src_view.add_css_class("zoom-" + zoom_level.to_string());
        settings.editor_font_size = zoom_level;
        zoom_changed(zoom_level);
    }
}

public void zoom_out() {
    if (zoom_level > FontSizeHelper.MIN_ZOOM_LEVEL) {
        src_view.remove_css_class("zoom-" + zoom_level.to_string());
        zoom_level--;
        src_view.add_css_class("zoom-" + zoom_level.to_string());
        settings.editor_font_size = zoom_level;
        zoom_changed(zoom_level);
    }
}

public void zoom_reset() {
    src_view.remove_css_class("zoom-" + zoom_level.to_string());
    zoom_level = FontSizeHelper.DEFAULT_ZOOM_LEVEL;
    src_view.add_css_class("zoom-" + zoom_level.to_string());
    settings.editor_font_size = zoom_level;
    zoom_changed(zoom_level);
}

public void set_zoom_level(int level) {
    if (level < FontSizeHelper.MIN_ZOOM_LEVEL || level > FontSizeHelper.MAX_ZOOM_LEVEL)
        return;
    }
    if (zoom_level == level) {
        return;
    }
    src_view.remove_css_class("zoom-" + zoom_level.to_string());
    zoom_level = level;
    src_view.add_css_class("zoom-" + zoom_level.to_string());
    zoom_changed(zoom_level);
}

public int get_zoom_level() {
    return zoom_level;
```

```
    }
}
```

File: src/Widgets/TextView/SourceView.vala

```
public class Iide.WordRange {
    public int line;
    public int start_column;
    public int end_column;

    public WordRange (int line, int start_column, int end_column) {
        this.line = line;
        this.start_column = start_column;
        this.end_column = end_column;
    }

    public bool is_equal (WordRange? other) {
        if (other == null) {
            return false;
        }
        return line == other.line && start_column == other.start_column && end_column == oth
    }
}

public class Iide.LspTooltipWidget : Gtk.Box {
    private Gtk.Label label;
    private Gtk.Spinner spinner;

    public LspTooltipWidget () {
        Object (
                orientation: Gtk.Orientation.HORIZONTAL,
                spacing: 6,
                margin_top: 8,
                margin_bottom: 8,
                margin_start: 8,
                margin_end: 8
        );

        spinner = new Gtk.Spinner ();
        label = new Gtk.Label ("    ...");
        label.use_markup = true;
        label.wrap = true;
        label.max_width_chars = 60;

        append (spinner);
        append (label);
```

```
        spinner.start ();
    }

    public void update_text (string? text, bool show_spinner) {
        if (show_spinner) {
            spinner.start ();
            spinner.show ();
        } else {
            spinner.stop ();
            spinner.hide ();
        }

        if (text != null && text != "") {
            label.set_markup (text);
        } else {
            label.set_text ("        ");
        }
    }
}

public class Iide.SourceView : GtkSource.View {
    public Window window;
    public string uri { get; private set; }
    private Iide.TreeSitterManager ts_manager;
    public BaseTreeSitterHighlighter? ts_highlighter;
    public GutterMarkRenderer mark_renderer;
    public string icon_name = "text-x-generic";
    private Gtk.TextIter? pending_scroll_iter = null;

    private WordRange? last_hover_range = null;
    private LspTooltipWidget tooltip_widget;
    private string tooltip_separator = "                      ";

    // Incremental didChange...
    private Gee.ArrayList<PendingChange> pending_queue = new Gee.ArrayList<PendingChange> ()
    private uint debounce_id = 0;
    private uint debounce_full_id = 0;
    private int document_version = 0;

    private ulong insert_handler_id = 0;
    private ulong delete_handler_id = 0;
    private int lsp_sync_kind = 0; // 0 -           , 1 - incremental, 2 - full sync

    // private LoggerService logger = LoggerService.get_instance ();

    public SourceView (Window window, string uri, GtkSource.Buffer buffer) {
```

```
Object (buffer : buffer);
this.window = window;
this.uri = uri;
this.ts_manager = new TreeSitterManager ();
this.ts_highlighter = null;

this.tooltip_widget = new LspTooltipWidget ();

// LSP-complete
var completion = get_completion ();
completion.select_on_show = true;
completion.show_icons = true;
completion.page_size = 18;
completion.remember_info_visibility = false;
var provider = new LspCompletionProvider (this);
completion.add_provider (provider);

// Build extra menu for context menu
var zoom_section = new GLib.Menu ();
zoom_section.append (_("Zoom In"), "app.zoom_in");
zoom_section.append (_("Zoom Out"), "app.zoom_out");
zoom_section.append (_("Reset Zoom"), "app.zoom_reset");

var view_section = new GLib.Menu ();
view_section.append (_("Minimap"), "app.toggle_minimap");

var extra_menu = new GLib.Menu ();
extra_menu.append_section (null, zoom_section);
extra_menu.append_section (null, view_section);
this.extra_menu = extra_menu;

var settings = Iide.SettingsService.get_instance ();

show_line_numbers = settings.show_line_numbers;
highlight_current_line = settings.highlight_current_line;
auto_indent = settings.auto_indent;
indent_on_tab = true;

// Marks gutter
set_show_line_marks (false);
var left_gutter = get_gutter (Gtk.TextWindowType.LEFT);
left_gutter.visible = true;

mark_renderer = new GutterMarkRenderer ();
mark_renderer.set_icons_size (FontSizeHelper.get_size_for_zoom_level (settings.edito
left_gutter.insert (mark_renderer, 0);
```

```
LspDiagnosticsMark.set_mark_attributes (this);

// Connect to settings changes to apply to all open documents
settings.editor_setting_changed.connect ((key) => {
    switch (key) {
        case "show-line-numbers" :
            show_line_numbers = settings.show_line_numbers;
            break;
        case "highlight-current-line" :
            highlight_current_line = settings.highlight_current_line;
            break;
        case "auto-indent":
            auto_indent = settings.auto_indent;
            break;
    }
});

// SpaceDrawer
var space_drawer = get_space_drawer ();

// 2.
space_drawer.set_enable_matrix (true);
space_drawer.set_types_for_locations (
                                      GtkSource.SpaceLocationFlags.ALL,
                                      GtkSource.SpaceTypeFlags.NONE
);
space_drawer.set_types_for_locations (
                                      GtkSource.SpaceLocationFlags.LEADING | GtkSour
                                      GtkSource.SpaceTypeFlags.SPACE | GtkSource.Spa
);

// Tree-sitter
detect_language ();
ts_highlighter = ts_manager.get_ts_highlighter (this);
if (ts_highlighter != null) {
    ((GtkSource.Buffer) (buffer)).highlight_syntax = false;
}

// LSP-tooltips
has_tooltip = true;
query_tooltip.connect (on_query_tooltip);

// Control-click controller (goto definition)
var click_gest = new Gtk.GestureClick ();
click_gest.set_button (1);
```

```
    click_gest.pressed.connect (on_click_pressed);
    add_controller (click_gest);

    // setup_buffer_signals ();
    // setup_buffer_signals_test ();
    this.connect_incremental_signals ();

    buffer.set_modified (false);
}

//
public void setup_lsp_sync (LspClient client) {
    this.apply_sync_strategy (client.capabilities, client);
}

public void setup_no_lsp_sync () {
    lsp_sync_kind = 0;
    if (insert_handler_id > 0)buffer.disconnect (insert_handler_id);
    if (delete_handler_id > 0)buffer.disconnect (delete_handler_id);

    //                 (        )
    this.pending_queue.clear ();
}

private void connect_incremental_signals () {
    insert_handler_id = buffer.insert_text.connect ((ref location, text, len) => {
        var change = new PendingChange (text, location);
        this.add_change (change);
    });

    delete_handler_id = buffer.delete_range.connect ((start, end) => {
        var change = new PendingChange ("", start, end);
        this.add_change (change);
    });
}

private void apply_sync_strategy (ServerCapabilities caps, LspClient client) {
    //                      (Full Sync)
    if (caps.sync_kind != TextDocumentSyncKind.INCREMENTAL) {
        //       'before'
        if (insert_handler_id > 0)buffer.disconnect (insert_handler_id);
        if (delete_handler_id > 0)buffer.disconnect (delete_handler_id);

        //                 (           Full Sync)
        this.pending_queue.clear ();
```

```
        //         Full Sync (                  )
        buffer.changed.connect_after (() => {
            this.start_debounce_full_timer ();
        });

        //                          (              )
        client.text_document_did_change.begin (this.uri, this.document_version, this.buf
        lsp_sync_kind = 2;
    } else {
        lsp_sync_kind = 1;
        reset_timer ();
    }
}

private void start_debounce_full_timer () {
    // 1.               ,
    if (debounce_full_id > 0) {
        Source.remove (debounce_full_id);
    }

    // 2.               (    , 500       )
    debounce_full_id = Timeout.add (500, () => {
        this.flush_changes_full_async.begin (); //
        debounce_full_id = 0;
        return Source.REMOVE;
    });
}

private async void flush_changes_full_async () {
    this.document_version++;

    //                (         ,          Main Loop       )
    string current_text = this.buffer.text;

    var client = IdeLspService.get_instance ().get_client_for_uri (this.uri);
    if (client != null) {
        try {
            //
            yield client.text_document_did_change (this.uri, this.document_version, curr

            debug ("LSP: Full sync sent for %s (v%d)", this.uri, this.document_version);
        } catch (Error e) {
            warning ("LSP: Full sync error: %s", e.message);
        }
    }
}
```

```
private void add_change (PendingChange nc) {
    if (!pending_queue.is_empty) {
        // var last = pending_queue.get (pending_queue.size - 1);

        //
        //    :        char_count()
        // if (nc.text != "" && last.text != "" && nc.start_offset == last.end_offset) {

        // last.text += nc.text;

        ////
        // last.end_offset = nc.end_offset;
        // last.end_line = nc.end_line;
        // last.end_char = nc.end_char;

        // reset_timer ();
        // return;
        // }

        // TODO: merge mergeable delete_range changes...
        //           (Backspace)
        //          -                ,
        // if (nc.text == "" && last.text == "" && nc.end_offset == last.start_offset) {
        // last.start_offset = nc.start_offset;
        // last.start_line = nc.start_line;
        // last.start_char = nc.start_char;
        // reset_timer ();
        // return;
        // }
    }

    pending_queue.add (nc);
    reset_timer ();
}

private void reset_timer () {
    if (debounce_id > 0)Source.remove (debounce_id);
    debounce_id = Timeout.add (400, () => {
        flush_changes ();
        return Source.REMOVE;
    });
}

public void flush_changes () {
    if (lsp_sync_kind != 1)return;
```

```
        if (pending_queue.is_empty)return;

        var changes = pending_queue;
        pending_queue = new Gee.ArrayList<PendingChange> ();
        if (debounce_id > 0) {
            Source.remove (debounce_id);
            debounce_id = 0;
        }

        this.document_version++;

        //
        var lsp_service = IdeLspService.get_instance ();
        lsp_service.send_did_change.begin (this.uri, this.document_version, changes);
    }

    private async void flush_changes_async () {
        if (pending_queue.is_empty)return;

        var changes = pending_queue;
        pending_queue = new Gee.ArrayList<PendingChange> ();
        if (debounce_id > 0) {
            Source.remove (debounce_id);
            debounce_id = 0;
        }

        this.document_version++;

        //
        var lsp_service = IdeLspService.get_instance ();
        yield lsp_service.send_did_change (this.uri, this.document_version, changes);
    }

    public async void sync_changes_async () {
        switch (lsp_sync_kind) {
        case 1:
            yield flush_changes_async ();

            break;
        case 2:
            yield flush_changes_full_async ();

            break;
        }
    }
```

```
public GtkSource.Language? language {
    set {
        ((GtkSource.Buffer) buffer).language = value;
    }
    get {
        return ((GtkSource.Buffer) buffer).language;
    }
}

public void detect_language () {
    var manager = GtkSource.LanguageManager.get_default ();
    var file = GLib.File.new_for_uri (uri);

    string mime_type = mime_type_for_file (file);
    message ("MIME: _ " + mime_type);

    icon_name = IconProvider.get_mime_type_icon_name (mime_type);
    language = manager.guess_language (file.get_path (), mime_type);

    // Fake file type detection
    // "Not all files are equal"
    if (file.get_basename () == "CMakeLists.txt") {
        language = manager.get_language ("cmake");
        icon_name = "text-x-cmake"; // Specific icon for CMake
    }
}

public WordRange ? get_word_range_under_cursor (Gtk.TextIter cursor_iter) {
    // 2.    :                          ,
    unichar c = cursor_iter.get_char ();
    if (c.isspace () || c == '\0') {
        return null;
    }

    // 3.
    Gtk.TextIter end_iter = cursor_iter;

    // 4.
    //                   , backward_word_start     false                .
    //          ,                  .
    if (!cursor_iter.starts_word ()) {
        cursor_iter.backward_word_start ();
    }

    // 5.
    if (!end_iter.ends_word ()) {
```

```
        end_iter.forward_word_end ();
    }

    return new WordRange (cursor_iter.get_line (), cursor_iter.get_line_offset (), end_i
}

private bool on_lsp_diagnostics_tooltip (GLib.SList<weak GtkSource.Mark> marks, Gtk.Tool
    var sb = new StringBuilder ();

    foreach (var mark in marks) {
        var lsp_mark = mark as LspDiagnosticsMark;
        if (lsp_mark == null) {
            continue;
        }

        string icon;
        string header_color;

        switch (lsp_mark.severity) {
        case 1 :
            icon = " ";
            header_color = "#F44336"; //
            break;
        case 2 :
            icon = " ";
            header_color = "#FF9800"; //
            break;
        case 3:
        case 4:
            icon = " ";
            header_color = "#2196F3"; //
            break;
        default:
            icon = " ";
            header_color = "#F44336"; //
            break;
        }

        if (sb.len > 0) {
            sb.append ("\n" + tooltip_separator + "\n");
        }

        //
        sb.append_printf ("%s <span font_weight='bold' foreground='%s'>%s</span>\n",
                          icon, header_color, lsp_mark.category.up ());
        sb.append_printf ("<span>%s</span>", GLib.Markup.escape_text (lsp_mark.diagnosti
```

```
    }

    if (sb.len > 0) {
        tooltip_widget.update_text (sb.str, false);
        tooltip.set_custom (tooltip_widget);
        return true;
    }

    return false;
}

private bool on_lsp_hover_tooltip (Gtk.TextIter iter, Gtk.Tooltip tooltip) {
    WordRange? word_range = get_word_range_under_cursor (iter);
    if (word_range == null) {
        last_hover_range = null;
        return false;
    }

    tooltip.set_custom (tooltip_widget);

    //      ,             100
    if (word_range.is_equal (last_hover_range)) {
        return true;
    }
    tooltip_widget.update_text ("Loading...", true);
    last_hover_range = word_range;

    flush_changes ();
    fetch_lsp_hover_async.begin (word_range.line, word_range.start_column + 1);

    return true;
}

private bool on_query_tooltip (int x, int y, bool keyboard_mode, Gtk.Tooltip tooltip) {
    Gtk.TextIter iter;

    //
    int buffer_x, buffer_y;
    window_to_buffer_coords (Gtk.TextWindowType.WIDGET, x, y, out buffer_x, out buffer_y

    //
    if (!get_iter_at_location (out iter, buffer_x, buffer_y)) {
        return false;
    }

    //                  (        "error")
```

```
    var buffer = (GtkSource.Buffer) buffer;
    var marks = buffer.get_source_marks_at_line (iter.get_line (), null);

    if (marks.length () > 0) {
        return on_lsp_diagnostics_tooltip (marks, tooltip);
    }

    return on_lsp_hover_tooltip (iter, tooltip);
}

private async void fetch_lsp_hover_async (int line, int col) {
    var lsp_service = IdeLspService.get_instance ();
    string? markdown = yield lsp_service.request_hover (uri, line, col);

    tooltip_widget.update_text (markdown, false);
}

private void on_click_pressed (Gtk.GestureClick gesture, int n_press, double x, double y
    //
    var modifiers = gesture.get_current_event_state ();

    if ((modifiers & Gdk.ModifierType.CONTROL_MASK) != 0) {
        Gtk.TextIter iter;
        //   GTK4
        //                      (           )
        int buf_x, buf_y;
        window_to_buffer_coords (Gtk.TextWindowType.WIDGET, (int) x, (int) y, out buf_x,

        if (get_iter_at_location (out iter, buf_x, buf_y)) {
            get_buffer ().place_cursor (iter);

            //
            flush_changes ();
            handle_ctrl_click_async.begin (iter.get_line (), iter.get_line_offset ());
        }
    }
}

private async void handle_ctrl_click_async (int line, int col) {
    var lsp_service = IdeLspService.get_instance ();
    var locations = yield lsp_service.goto_definition (uri, line, col);

    if (locations == null || locations.size == 0) {
        LoggerService.get_instance ().warning ("LSP", "No locations found for goto defin
        return;
    }
```

```
        var loc = locations.get (0);
        window.get_document_manager ().open_document_with_selection (File.new_for_uri (loc.u
    }

    public void select_and_scroll (int line, int start_col, int end_col, bool is_new) {
        if (line >= buffer.get_line_count ()) {
            return;
        }

        Gtk.TextIter start_iter;
        buffer.get_iter_at_line_offset (out start_iter, line, start_col);

        Gtk.TextIter end_iter;
        buffer.get_iter_at_line_offset (out end_iter, line, end_col);

        buffer.select_range (start_iter, end_iter);
        scroll_to_iter (start_iter, 0.0, true, 0.5, 0.5);
        pending_scroll_iter = null;
        if (is_new) {
            pending_scroll_iter = start_iter;
        }
    }

    public override void size_allocate (int width, int height, int baseline) {
        base.size_allocate (width, height, baseline);
        if (pending_scroll_iter != null) {
            scroll_to_iter (pending_scroll_iter, 0.0, true, 0.5, 0.5);
            pending_scroll_iter = null;
        }
    }
}
```

File: src/style.css

```
textview {
 ----
.log-view text {
 ----
.text-view.zoom-1 {
 ----
.lsp_error_line {
 ----
.lsp_warning_line {
 ----
.lsp_info_line {
```

```
----
.text-view.zoom-2 {
----
.text-view.zoom-3 {
----
.text-view.zoom-4 {
----
.text-view.zoom-5 {
----
.text-view.zoom-6 {
----
.text-view.zoom-7 {
----
.text-view.zoom-8 {
----
.text-view.zoom-9 {
----
.text-view.zoom-10 {
----
.text-view.zoom-11 {
----
.text-view.zoom-12 {
----
.text-view.zoom-13 {
----
.text-view.zoom-14 {
----
.text-view.zoom-15 {
----
.textview-map {
----
.breadcrumps-btn {
----
.editor-status-bar .diagnostic-box {
----
opacity: 0.8; /*                        */
----
.editor-status-bar .diagnostic-box.hover {
----
); /*                  */
opacity: 1; /*                   */
----
/*                     ,                        */
.editor-status-bar .diagnostic-box.hover image {
----
button.error label {
```

```
 ----
color: @error_fg_color;
 ----
button.error image {
```

File: src/Widgets/TextView/LspCompletionProvider.vala

```
/*
 * LspCompletionProvider.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

using GLib;
using Gee;
using Gtk;
using GtkSource;

namespace Iide {

    public class LspCompletionProposal : GLib.Object, CompletionProposal {
        public IdeLspCompletionItem item { get; private set; }

        public LspCompletionProposal (IdeLspCompletionItem item) {
            this.item = item;
        }

        public string get_label () {
            return item.label;
        }
```

```
    public string ? get_markup () {
        return item.label;
    }

    public string get_text () {
        return item.insert_text;
    }

    public Icon ? get_icon () {
        return null;
    }

    public string ? get_info () {
        return item.documentation;
    }
}

public class LspCompletionProvider : GLib.Object, CompletionProvider {
    private weak SourceView source_view;
    private IdeLspService lsp_service;
    private GLib.ListStore base_store;
    private Gtk.FilterListModel filter_model;
    private string current_word = "";

    private bool filter_proposals (Object item) {
        var proposal = item as LspCompletionProposal;
        if (proposal == null)return false;

        //          filter_text   LSP,         ,     label
        string haystack = proposal.get_label ();
        string needle = this.current_word;

        if (needle == "")return true;

        //                  GtkSource.Completion
        uint priority;
        return GtkSource.Completion.fuzzy_match (haystack, needle, out priority);
    }

    public LspCompletionProvider (SourceView view) {
        this.source_view = view;
        this.lsp_service = IdeLspService.get_instance ();

        // 1.
        base_store = new GLib.ListStore (typeof (LspCompletionProposal));
```

```
    // 2.         ,                 filter_proposals
    var custom_filter = new Gtk.CustomFilter (filter_proposals);

    // 3.         -
    filter_model = new Gtk.FilterListModel (base_store, custom_filter);
}

public virtual bool is_trigger (Gtk.TextIter iter, unichar ch) {
    // 1.
    var client = IdeLspService.get_instance ().get_client_for_uri (this.source_view.

    if (client == null || !client.is_initialized) {
        //                ,                  (          )
        return ch == '.';
    }

    // 2.      ,
    string s = ch.to_string ();
    if (client.capabilities.completion_triggers.contains (s)) {
        return true;
    }

    return false;
}

public virtual CompletionActivation get_activation (CompletionContext context) {
    //        :   Ctrl+Space
    return CompletionActivation.INTERACTIVE | CompletionActivation.USER_REQUESTED;
}

public virtual async GLib.ListModel populate_async (CompletionContext context, GLib.
    base_store.remove_all ();
    current_word = context.get_word ();

    if (source_view == null) {
        return filter_model;
    }

    yield source_view.sync_changes_async ();

    var buffer = source_view.buffer;
    var insert_mark = buffer.get_insert ();
    TextIter iter;
    buffer.get_iter_at_mark (out iter, insert_mark);

    int line = iter.get_line ();
```

```
    int character = iter.get_line_offset ();
    string uri = source_view.uri;

    string? trigger_char = null;
    var trigger_kind = CompletionTriggerKind.INVOKED; // Invoked

    TextIter prev = iter;
    if (prev.backward_char ()) {
        unichar ch = prev.get_char ();
        string s = ch.to_string ();

        var client = IdeLspService.get_instance ().get_client_for_uri (this.source_v
        if (client != null && client.capabilities.completion_triggers.contains (s))
            trigger_char = s;
            trigger_kind = CompletionTriggerKind.TRIGGER_CHARACTER;
        }
    }

    var result = yield lsp_service.request_completion (uri, line, character, trigger

    if (result == null || result.items.size == 0) {
        return filter_model;
    }

    foreach (var item in result.items) {
        var proposal = new LspCompletionProposal (item);
        base_store.append (proposal);
    }

    return filter_model;
}

private string get_icon_name_for_kind (int kind) {
    switch (kind) {
    case 2 : case 3: return "code-function-symbolic"; // Method, Function
    case 4: return "code-class-symbolic"; // Constructor
    case 5: return "code-variable-symbolic"; // Field
    case 6: return "code-variable-symbolic"; // Variable
    case 7: return "code-class-symbolic"; // Class
    case 8: return "code-class-symbolic"; // Interface
    case 9: return "code-context-menu-symbolic"; // Module
    case 10: return "code-variable-symbolic"; // Property
    case 11: return "code-variable-symbolic"; // Unit
    case 12: return "code-variable-symbolic"; // Value
    case 13: return "code-class-symbolic"; // Enum
    case 14: return "code-variable-symbolic"; // Keyword
```

```
    case 15: return "code-context-menu-symbolic"; // Snippet
    case 16: return "code-variable-symbolic"; // Color
    case 17: return "code-variable-symbolic"; // File
    case 18: return "code-variable-symbolic"; // Reference
    default: return "code-variable-symbolic";
    }
}

public virtual void display (CompletionContext context, CompletionProposal proposal,
    var p = (LspCompletionProposal) proposal;

    switch (cell.column) {
    case CompletionColumn.ICON:
        cell.set_icon_name (get_icon_name_for_kind (p.item.kind));
        break;
    case CompletionColumn.TYPED_TEXT:
        cell.text = p.get_label ();
        break;
    case CompletionColumn.DETAILS: {
        string? doc = p.item.documentation;
        if (doc != null && doc != "") {
            cell.set_markup (doc);
        }
        break;
    }
    case CompletionColumn.COMMENT: {
        string? doc = p.item.detail;
        if (doc != null && doc != "") {
            cell.set_markup (doc);
        }
        break;
    }
    default:
        break;
    }
}

// 2.            (   )
public virtual void activate (CompletionContext context, CompletionProposal proposal
    var view = context.get_view ();
    var buffer = view.get_buffer ();

    TextIter start, end;

    //                ,      GSV            "           "
    if (context.get_bounds (out start, out end)) {
```

```
                //             ,
                buffer.delete (ref start, ref end);
            } else {
                //           (       ),              ,
                buffer.get_iter_at_mark (out start, buffer.get_insert ());
                end = start;
                string? word = context.get_word ();
                if (word != null && word != "") {
                    start.backward_chars (word.char_count ());
                    buffer.delete (ref start, ref end);
                }
            }

            //
            var p = (LspCompletionProposal) proposal;
            buffer.insert (ref start, p.get_label (), -1);
        }

        //            -
        public virtual void refilter (CompletionContext context, GLib.ListModel model) {
            current_word = context.get_word ();

            // 6.
            var filter = filter_model.get_filter () as Gtk.CustomFilter;
            filter.changed (Gtk.FilterChange.DIFFERENT);

            if (model.get_n_items () == 0) {
                context.get_completion ().hide ();
            }
        }

        public virtual string ? get_title () { return "Simple"; }
        public virtual int get_priority (CompletionContext context) { return 100; }
        public virtual bool is_running (CompletionContext context) { return true; }
    }
}
```

File: src/Services/LSP/IdeLspService.vala

```
using GLib;
using Gee;

public class Iide.IdeLspService : GLib.Object {
    private static IdeLspService? _instance;
    private Gee.HashMap<string, LspClient> clients;
    private Gee.HashMap<string, string> uri_to_client_key;
```

```
private Gee.HashMap<string, int> document_versions;
private Gee.HashMap<string, bool> client_starting;
private Gee.ArrayList<PendingOpen> pending_opens;

private LoggerService logger = LoggerService.get_instance ();

public signal void diagnostics_updated (string uri, ArrayList<IdeLspDiagnostic> diagnost

public class PendingOpen {
    public string uri;
    public string language_id;
    public string content;
    public string? workspace_root;
    public SourceView view;

    public PendingOpen (string uri, string language_id, string content, string? workspac
        this.uri = uri;
        this.language_id = language_id;
        this.content = content;
        this.workspace_root = workspace_root;
        this.view = view;
    }
}

// [ClientHash] -> [Token] -> LspTaskInfo
private Gee.HashMap<int, Gee.HashMap<string, LspTaskInfo?>> progress_map =
    new Gee.HashMap<int, Gee.HashMap<string, LspTaskInfo?>> ();

public signal void tasks_changed (Gee.List<LspTaskInfo?> active_tasks);

construct {
    clients = new Gee.HashMap<string, LspClient> ();
    uri_to_client_key = new Gee.HashMap<string, string> ();
    document_versions = new Gee.HashMap<string, int> ();
    client_starting = new Gee.HashMap<string, bool> ();
    pending_opens = new Gee.ArrayList<PendingOpen> ();
}

public static unowned IdeLspService get_instance () {
    if (_instance == null) {
        _instance = new IdeLspService ();
    }
    return _instance;
}

public LspClient ? get_client_by_hash (int client_id) {
```

```
    foreach (var client in clients.values) {
        if (client.get_hash () == client_id) {
            return client;
        }
    }
    return null;
}

public void register_client (LspClient client) {
    int id = client.get_hash ();

    client.progress_updated.connect ((token, msg, perc, active) => {
        if (!progress_map.has_key (id))
            progress_map.set (id, new Gee.HashMap<string, LspTaskInfo?> ());

        var client_tasks = progress_map.get (id);

        if (active) {
            var info = LspTaskInfo () {
                server_name = client.name (),
                message = msg,
                percentage = perc
            };
            client_tasks.set (token, info);
        } else {
            client_tasks.unset (token);
        }

        emit_tasks_changed ();
    });
}

private void emit_tasks_changed () {
    var all_tasks = new Gee.ArrayList<LspTaskInfo?> ();
    foreach (var client_map in progress_map.values) {
        foreach (var task in client_map.values) {
            all_tasks.add (task);
        }
    }
    tasks_changed (all_tasks);
}

public async void open_document (string uri, string language_id, string content, string?
    var server_key = LspRegistry.get_lsp_id (language_id);
    if (server_key == null) {
        debug ("IdeLspService: No LSP server configured for language: %s", language_id);
```

```
    Idle.add (() => {
        view.setup_no_lsp_sync ();
        return Source.REMOVE;
    });
    return;
}

debug ("IdeLspService: Opening document %s (lang=%s)", uri, language_id);

if (clients.has_key (server_key)) {
    var client = clients.get (server_key);
    uri_to_client_key.set (uri, server_key);
    document_versions.set (uri, 1);
    try {
        yield client.text_document_did_open (uri, language_id, 1, content);
    } catch (Error e) {
        logger.error ("LSP", "Failed to open document %s: %s".printf (uri, e.message
    }

    Idle.add (() => {
        view.setup_lsp_sync (client);
        return Source.REMOVE;
    });

    return;
}

if (client_starting.get (server_key) == true) {
    pending_opens.add (new PendingOpen (uri, language_id, content, workspace_root, v
    return;
}

client_starting.set (server_key, true);

var config = LspRegistry.get_config (server_key);
if (config == null) {
    client_starting.set (server_key, false);
    return;
}

var client = new LspClient (config);
client.diagnostics_received.connect ((uri, diagnostics) => {
    diagnostics_updated (uri, diagnostics);
});

bool started = yield client.start_server_async (workspace_root);
```

```
    logger.info ("LSP", "Started server for %s: %b (%s)".printf (server_key, started, wo

    if (started) {
        clients.set (server_key, client);
        uri_to_client_key.set (uri, server_key);
        document_versions.set (uri, 1);
        try {
            yield client.text_document_did_open (uri, language_id, 1, content);
        } catch (Error e) {
            logger.error ("LSP", "Failed to open document %s: %s".printf (uri, e.message
        }

        Idle.add (() => {
            view.setup_lsp_sync (client);
            return Source.REMOVE;
        });

        yield process_pending_opens ();
    } else {
        // TODO: restart logic...
        Idle.add (() => {
            view.setup_no_lsp_sync ();
            return Source.REMOVE;
        });

        warning ("IdeLspService: Failed to start LSP server for %s", server_key);
    }

    client_starting.set (server_key, false);
}

private async void process_pending_opens () {
    var opens_to_process = new ArrayList<PendingOpen> ();
    foreach (var open in pending_opens) {
        opens_to_process.add (open);
    }
    pending_opens.clear ();

    foreach (var open in opens_to_process) {
        yield open_document (open.uri, open.language_id, open.content, open.workspace_ro
    }
}

public async void change_document (string uri, string content, int? change_start = null,
    var server_key = uri_to_client_key.get (uri);
```

```
    logger.debug ("LSP", "Changed doc: " + uri + " / server_key: " + server_key);
    if (server_key == null || !clients.has_key (server_key)) {
        return;
    }

    var client = clients.get (server_key);
    var version = document_versions.get (uri);
    document_versions.set (uri, version + 1);

    try {
        yield client.text_document_did_change (uri, version + 1, content);
    } catch (Error e) {
        logger.error ("LSP", "Failed to change document (FULL sync) %s: %s".printf (uri,
    }
}

public async void send_did_change (string uri, int version, Gee.ArrayList<PendingChange>
    var server_key = uri_to_client_key.get (uri);
    if (server_key == null || !clients.has_key (server_key)) {
        return;
    }

    var client = clients.get (server_key);
    try {
        yield client.send_did_change (uri, version, changes);
    } catch (Error e) {
        logger.error ("LSP", "Failed to change document (INCREMENTAL sync) %s: %s".print
    }
}

public async void close_document (string uri) {
    var server_key = uri_to_client_key.get (uri);
    if (server_key == null || !clients.has_key (server_key)) {
        return;
    }

    var client = clients.get (server_key);
    try {
        yield client.text_document_did_close (uri);
    } catch (Error e) {
        logger.error ("LSP", "Failed to close document %s: %s".printf (uri, e.message));
    }

    uri_to_client_key.unset (uri);
    document_versions.unset (uri);
```

```
}

public LspClient ? get_client_for_uri (string uri) {
    var server_key = uri_to_client_key.get (uri);
    if (server_key == null) {
        return null;
    }
    return clients.get (server_key);
}

public async IdeLspCompletionResult ? request_completion (string uri,
                                                          int line,
                                                          int character,
                                                          string? trigger_character = nu
                                                          CompletionTriggerKind trigger_
    var client = get_client_for_uri (uri);
    if (client == null) {
        return null;
    }
    try {
        return yield client.request_completion (uri, line, character, trigger_character,
    } catch (Error e) {
        logger.error ("LSP", "Failed to request completion for %s: %s".printf (uri, e.me
        return null;
    }
}

public async string ? request_hover (string uri, int line, int character) {
    var client = get_client_for_uri (uri);
    if (client == null) {
        logger.debug ("HOVER", "client is null!");
        return null;
    }
    try {
        return yield client.request_hover (uri, line, character);
    } catch (Error e) {
        logger.error ("LSP", "Failed to request hover for %s: %s".printf (uri, e.message
        return null;
    }
}

public async Gee.ArrayList<IdeLspLocation>? goto_definition (string uri, int line, int c
    var client = get_client_for_uri (uri);
    if (client == null)return null;
    try {
        return yield client.request_definition (uri, line, character);
```

```
        } catch (Error e) {
            logger.error ("LSP", "Failed to request definition for %s: %s".printf (uri, e.me
            return null;
        }
    }
}
```

File: src/application.vala

```
/* application.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

[CCode (cname = "gtk_style_context_add_provider_for_display", cheader_filename = "gtk/gtk.h'
extern void add_provider_to_display (Gdk.Display display, Gtk.StyleProvider provider, uint p

public class Iide.Application : Adw.Application {
    private Iide.SettingsService settings;
    private Iide.ActionManager action_manager;
    private SimpleActionGroup simple_action_group = new SimpleActionGroup ();

    public signal void zoom_changed (int zoom_level);
    public signal void minimap_changed (bool visible);

    public Application () {
        Object (
                application_id: "org.github.kai66673.iide",
                flags: ApplicationFlags.DEFAULT_FLAGS,
                resource_base_path: "/org/github/kai66673/iide"
        );
```

```
}

construct {
    settings = Iide.SettingsService.get_instance ();
    action_manager = Iide.ActionManager.get_instance ();

    register_builtin_actions ();
    apply_shortcuts ();
}

private void register_builtin_actions () {
    action_manager.register_action (new SaveAllAction (this));
    action_manager.register_action (new OpenProjectAction (this));
    action_manager.register_action (new PreferencesAction (this));
    action_manager.register_action (new ToggleMinimapAction (this));
    action_manager.register_action (new FuzzyFinderAction (this));
    action_manager.register_action (new SearchInFilesAction (this));
    action_manager.register_action (new ZoomInAction ());
    action_manager.register_action (new ZoomOutAction ());
    action_manager.register_action (new ZoomResetAction ());
    action_manager.register_action (new ExpandSelectionAction ());
    action_manager.register_action (new ShrinkSelectionAction ());
    action_manager.register_action (new QuitAction ());

    // Ins/Ovr toggle
    //
    var toggle_overwrite = new SimpleAction ("toggle-overwrite", null);
    toggle_overwrite.activate.connect (() => {
        //                    SourceView
        var win = active_window as Iide.Window;
        if (win != null) {
            var view = win.get_active_source_view ();
            if (view != null) {
                view.overwrite = !view.overwrite;
            }
        }
    });
    simple_action_group.add_action (toggle_overwrite);
    set_accels_for_action ("editor.toggle-overwrite", { "Insert" });
}

private void apply_shortcuts () {
    action_manager.apply_shortcuts_to_application (this);

    action_manager.get_all_actions ().foreach ((action) => {
        action.shortcut_changed.connect ((new_shortcut) => {
```

```
                if (new_shortcut != null && new_shortcut != "") {
                    this.set_accels_for_action ("app." + action.id, { new_shortcut });
                } else {
                    this.set_accels_for_action ("app." + action.id, {});
                }
            });
            return true;
        });
    }

    public Iide.ActionManager get_action_manager () {
        return action_manager;
    }

    public Iide.SettingsService get_settings () {
        return settings;
    }

    public override void activate () {
        base.activate ();

        var icon_theme = Gtk.IconTheme.get_for_display (Gdk.Display.get_default ());
        icon_theme.add_resource_path ("/org/github/kai66673/iide/icons");

        var css_provider = new Gtk.CssProvider ();
        css_provider.load_from_resource ("/org/github/kai66673/iide/style.css");
        add_provider_to_display (
                                 Gdk.Display.get_default (),
                                 css_provider,
                                 Gtk.STYLE_PROVIDER_PRIORITY_APPLICATION
        );

        var win = this.active_window ?? new Iide.Window (this);
        win.present ();
    }
}

private class SaveAllAction : Iide.Action {
    private weak Iide.Application app;

    public SaveAllAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "save"; } }
    public override string name { get { return _("Save All"); } }
```

```
    public override string? description { get { return _("Save all open documents"); } }
    public override string? icon_name { get { return "document-save-symbolic"; } }
    public override string? category { get { return "File"; } }

    public override bool can_execute () {
        return app ? .active_window is Iide.Window;
    }

    public override void execute () {
        var win = app ? .active_window as Iide.Window;
        win?.save_modified ();
    }
}

private class OpenProjectAction : Iide.Action {
    private weak Iide.Application app;

    public OpenProjectAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "open_project"; } }
    public override string name { get { return _("Open Project"); } }
    public override string? description { get { return _("Open a project folder"); } }
    public override string? icon_name { get { return "folder-open-symbolic"; } }
    public override string? category { get { return "File"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var win = app ? .active_window as Iide.Window;
        win?.open_project_dialog ();
    }
}

private class PreferencesAction : Iide.Action {
    private weak Iide.Application app;

    public PreferencesAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "preferences"; } }
    public override string name { get { return _("Preferences"); } }
```

```
    public override string? description { get { return _("Open preferences dialog"); } }
    public override string? icon_name { get { return "preferences-system-symbolic"; } }
    public override string? category { get { return "Application"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var dialog = new Iide.PreferencesDialog ();
        dialog.set_transient_for (app ? .active_window);
        dialog.present ();
    }
}

private class ToggleMinimapAction : Iide.Action {
    private weak Iide.Application app;

    public ToggleMinimapAction (Iide.Application app) {
        this.app = app;
        this.state = Iide.SettingsService.get_instance ().show_minimap;
    }

    public override string id { get { return "toggle_minimap"; } }
    public override string name { get { return _("Toggle Minimap"); } }
    public override string? description { get { return _("Show or hide the minimap"); } }
    public override string? icon_name { get { return "view-fullscreen-symbolic"; } }
    public override string? category { get { return "View"; } }
    public override bool is_toggle { get { return true; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var settings = Iide.SettingsService.get_instance ();
        state = !state;
        settings.show_minimap = state;
        app?.minimap_changed (state);

        state_changed (state);
        Iide.ActionManager.get_instance ().set_toggle_state (id, state);
    }
}

private class ZoomInAction : Iide.Action {
```

```
    private Iide.SettingsService settings;

    public override string id { get { return "zoom_in"; } }
    public override string name { get { return _("Zoom In"); } }
    public override string? description { get { return _("Increase editor font size"); } }
    public override string? icon_name { get { return "zoom-in-symbolic"; } }
    public override string? category { get { return "View"; } }

    public override bool can_execute () {
        settings = Iide.SettingsService.get_instance ();
        return settings.editor_font_size < FontSizeHelper.MAX_ZOOM_LEVEL;
    }

    public override void execute () {
        settings = Iide.SettingsService.get_instance ();
        var app = GLib.Application.get_default () as Iide.Application;
        settings.editor_font_size++;
        app?.zoom_changed (settings.editor_font_size);
    }
}

private class ZoomOutAction : Iide.Action {
    private Iide.SettingsService settings;

    public override string id { get { return "zoom_out"; } }
    public override string name { get { return _("Zoom Out"); } }
    public override string? description { get { return _("Decrease editor font size"); } }
    public override string? icon_name { get { return "zoom-out-symbolic"; } }
    public override string? category { get { return "View"; } }

    public override bool can_execute () {
        settings = Iide.SettingsService.get_instance ();
        return settings.editor_font_size > FontSizeHelper.MIN_ZOOM_LEVEL;
    }

    public override void execute () {
        settings = Iide.SettingsService.get_instance ();
        var app = GLib.Application.get_default () as Iide.Application;
        settings.editor_font_size--;
        app?.zoom_changed (settings.editor_font_size);
    }
}

private class ZoomResetAction : Iide.Action {
    private Iide.SettingsService settings;
```

```
    public override string id { get { return "zoom_reset"; } }
    public override string name { get { return _("Zoom Reset"); } }
    public override string? description { get { return _("Reset editor font size to default'
    public override string? icon_name { get { return "zoom-original-symbolic"; } }
    public override string? category { get { return "View"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        settings = Iide.SettingsService.get_instance ();
        var app = GLib.Application.get_default () as Iide.Application;
        settings.editor_font_size = FontSizeHelper.DEFAULT_ZOOM_LEVEL;
        app?.zoom_changed (settings.editor_font_size);
    }
}

private class ExpandSelectionAction : Iide.Action {
    public override string id { get { return "expand_selection"; } }
    public override string name { get { return _("Expand Selection"); } }
    public override string? description { get { return _("Expand the current selection"); }
    public override string? icon_name { get { return "zoom-original-symbolic"; } }
    public override string? category { get { return "View"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var app = GLib.Application.get_default () as Iide.Application;
        var win = app ? .active_window as Iide.Window;
        if (win != null) {
            win.get_active_source_view () ? .ts_highlighter ? .expand_selection ();
        }
    }
}

private class ShrinkSelectionAction : Iide.Action {
    public override string id { get { return "shrink_selection"; } }
    public override string name { get { return _("Shrink Selection"); } }
    public override string? description { get { return _("Shrink the current selection"); }
    public override string? icon_name { get { return "zoom-original-symbolic"; } }
    public override string? category { get { return "View"; } }

    public override bool can_execute () {
```

```
        return true;
    }

    public override void execute () {
        var app = GLib.Application.get_default () as Iide.Application;
        var win = app ? .active_window as Iide.Window;
        if (win != null) {
            win.get_active_source_view () ? .ts_highlighter ? .shrink_selection ();
        }
    }
}

private class QuitAction : Iide.Action {
    public override string id { get { return "quit"; } }
    public override string name { get { return _("Quit"); } }
    public override string? description { get { return _("Quit the application"); } }
    public override string? icon_name { get { return "application-exit-symbolic"; } }
    public override string? category { get { return "Application"; } }

    public override bool can_execute () {
        return true;
    }

    public override void execute () {
        var app = GLib.Application.get_default () as Iide.Application;
        app?.quit ();
    }
}

private class FuzzyFinderAction : Iide.Action {
    private weak Iide.Application app;

    public FuzzyFinderAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "fuzzy_finder"; } }
    public override string name { get { return _("Quick Open"); } }
    public override string? description { get { return _("Open a file quickly by name"); } }
    public override string? icon_name { get { return "system-search-symbolic"; } }
    public override string? category { get { return "File"; } }

    public override bool can_execute () {
        return app ? .active_window is Iide.Window;
    }
```

```
    public override void execute () {
        var win = app ? .active_window as Iide.Window;
        if (win != null) {
            var dialog = new Iide.FuzzyFinderDialog (win, win.get_document_manager ());
            dialog.set_transient_for (win);
            dialog.present ();
        }
    }
}

private class SearchInFilesAction : Iide.Action {
    private weak Iide.Application app;

    public SearchInFilesAction (Iide.Application app) {
        this.app = app;
    }

    public override string id { get { return "search_in_files"; } }
    public override string name { get { return _("Search in Files"); } }
    public override string? description { get { return _("Search for text in all project fil
    public override string? icon_name { get { return "edit-find-symbolic"; } }
    public override string? category { get { return "Edit"; } }

    public override bool can_execute () {
        return app ? .active_window is Iide.Window;
    }

    public override void execute () {
        var win = app ? .active_window as Iide.Window;
        if (win != null) {
            var dialog = new Iide.SearchInFilesDialog (win, win.get_document_manager ());
            dialog.set_transient_for (win);
            dialog.present ();
        }
    }
}
```

File: src/Services/TreeSitter/BaseTreeSitterHighlighter.vala

```
using TreeSitter;

public struct BreadcrumbItem {
    public string name;
    public TreeSitter.Point start_point;
}
```

```
public abstract class Iide.BaseTreeSitterHighlighter : Object {
    //            Tree-sitter
    protected Parser parser;
    protected TreeSitter.Tree? tree;
    protected Query? query;
    protected QueryCursor cursor;

    // GTK
    protected GtkSource.Buffer buffer;
    protected SourceView view;

    //            : [capture_index, theme_index]
    // theme_index: 1 - Light, 0 - Dark
    private Gtk.TextTag ? [, ] capture_tags;

    //                       (            DocumentManager)
    private int current_theme_index = 0;

    //        Debounce
    private uint highlight_timeout_id = 0;
    private const uint DEBOUNCE_MS = 150;

    //           Range   Tree-sitter
    private Gee.ArrayQueue<TreeSitter.Range?> selection_stack = new Gee.ArrayQueue<TreeSitte
    private bool is_internal_selection_change = false;

    private Gee.List<BreadcrumbItem?> last_crumbs = null;
    public signal void breadcrumbs_changed (Gee.List<BreadcrumbItem?> crumbs);

    private void set_color_theme () {
        var color_scheme = SettingsService.get_instance ().color_scheme;
        current_theme_index = color_scheme != ColorScheme.LIGHT ? 1 : 0;
        if (highlight_timeout_id > 0) {
            Source.remove (highlight_timeout_id);
        }
        initial_rehighlight ();
    }

    protected BaseTreeSitterHighlighter (SourceView view) {
        view.set_insert_spaces_instead_of_tabs (true);
        view.set_tab_width (4);
        view.set_smart_backspace (true);

        this.view = view;
        this.buffer = (GtkSource.Buffer) view.get_buffer ();
```

```
    this.parser = new Parser ();
    this.cursor = new QueryCursor ();

    //
    unowned Language lang = get_ts_language ();
    this.parser.set_language (lang);
    //       Query
    load_query (lang);
    //                                Query
    prepare_capture_mapping ();

    //
    set_color_theme ();
    this.buffer.notify["style-scheme"].connect_after (set_color_theme);

    //
    buffer.insert_text.connect_after (on_insert_text);
    buffer.delete_range.connect (on_delete_range);

    buffer.notify["cursor-position"].connect (() => {
        //                                  expand/shrink -

        if (!is_internal_selection_change) {
            selection_stack.clear ();
        }
    });

    buffer.notify["cursor-position"].connect (update_breadcrumbs);
}

//                              (Vala, Cpp   . .)
protected abstract unowned Language get_ts_language ();
protected abstract string get_query_filename ();
protected abstract string query_source ();

//                         Breadcrumbs
protected abstract bool is_container_node (string node_type);

private void load_query (Language lang) {
    string source = query_source ();

    uint32 error_offset;
    QueryError error_type;
    this.query = new Query (lang, source, (uint32) source.length, out error_offset, out

    if (error_type != QueryError.None) {
```

```
        warning ("TreeSitter Query Error at %u: %s", error_offset, error_type.to_string
    }
}

private void prepare_capture_mapping () {
    if (query == null)return;

    uint32 count = query.capture_count ();
    capture_tags = new Gtk.TextTag ? [count, 2];
    var style_service = StyleService.get_instance ();

    for (uint32 i = 0; i < count; i++) {
        uint32 name_len;
        string name = query.capture_name_for_id (i, out name_len);

        //
        capture_tags[i, 0] = style_service.get_tag (name, 0);
        capture_tags[i, 1] = style_service.get_tag (name, 1);

        //                 (     , @function.method -> @function)
        if (capture_tags[i, 0] == null && name.contains (".")) {
            string base_name = name.split (".")[0];
            capture_tags[i, 0] = style_service.get_tag (base_name, 0);
            capture_tags[i, 1] = style_service.get_tag (base_name, 1);
        }
    }
}

private void on_buffer_changed () {
    if (highlight_timeout_id > 0) {
        Source.remove (highlight_timeout_id);
    }

    highlight_timeout_id = Timeout.add (DEBOUNCE_MS, () => {
        // run_highlighting ();
        sync_and_render ();
        highlight_timeout_id = 0;
        return Source.REMOVE;
    });
}

private void initial_rehighlight () {
    Gtk.TextIter start, end;
    buffer.get_bounds (out start, out end);

    //
```

```
    string content = buffer.get_text (start, end, false);
    this.tree = parser.parse_string (null, content.data);

    update_breadcrumbs ();

    //
    buffer.remove_all_tags (start, end);

    this.cursor = new QueryCursor ();
    cursor.exec (query, tree.root_node ());
    render_matches ();
}

private void sync_and_render () {
    Gtk.TextIter start, end;
    buffer.get_bounds (out start, out end);
    //
    string content = buffer.get_text (start, end, false);

    // old_tree               on_insert_text / on_delete_range
    unowned TreeSitter.Tree? old_tree = this.tree;
    var new_tree = parser.parse_string (old_tree, content.data);

    update_breadcrumbs ();

    TreeSitter.Range[] changed_ranges = old_tree.get_changed_ranges (new_tree);
    // uint32 length = (uint32) changed_ranges.length;
    this.tree = (owned) new_tree;
    this.cursor = new QueryCursor ();
    foreach (var range in changed_ranges) {
        Gtk.TextIter s, e;
        //
        get_iters_from_ts_node_coords (range.start_point, range.end_point, out s, out e)

        //
        buffer.remove_all_tags (s, e);

        cursor.set_byte_range (range.start_byte, range.end_byte);
        cursor.exec (query, tree.root_node ());
        render_matches ();
    }
}

private void render_matches () {
    QueryMatch match;
    while (cursor.next_match (out match)) {
```

```
        foreach (var capture in match.captures) {
            uint32 name_len;
            string capture_name = query.capture_name_for_id (capture.index, out name_len

            Gtk.TextTag? tag = null;

            if (capture_name == "punctuation.bracket") {
                int lvl = (get_nesting_level (capture.node) % 5) + 1; //      5
                tag = StyleService.get_instance ().get_tag ("bracket.lvl" + lvl.to_strin
            } else {
                tag = capture_tags[capture.index, current_theme_index];
            }                if (tag != null) {
                apply_tag_fast (capture.node, tag);
            }
        }
    }
}

private int get_nesting_level (TreeSitter.Node node) {
    int level = 0;
    TreeSitter.Node? parent = node.parent ();
    while (parent != null && !parent.is_null ()) {
        string type = parent.type ();
        //                      ,              . .
        if (type == "block" || type == "argument_list" || type == "parameters" || type =
            level++;
        }
        parent = parent.parent ();
    }
    return level;
}

private void apply_tag_fast (TreeSitter.Node node, Gtk.TextTag tag) {
    Gtk.TextIter start, end;

    //
    get_iters_from_ts_node (buffer, node, out start, out end);

    buffer.apply_tag (tag, start, end);
}

public static void get_iters_from_ts_node (Gtk.TextBuffer buffer, TreeSitter.Node node,
                                           out Gtk.TextIter start_iter, out Gtk.TextIter
    buffer.get_iter_at_line (out start_iter, (int) node.start_point ().row);
    start_iter.set_line_index ((int) node.start_point ().column);
```

```
    buffer.get_iter_at_line (out end_iter, (int) node.end_point ().row);
    end_iter.set_line_index ((int) node.end_point ().column);
}

//
private void get_iters_from_ts_node_coords (TreeSitter.Point start_p, TreeSitter.Point e
                                            out Gtk.TextIter s, out Gtk.TextIter e) {
    buffer.get_iter_at_line (out s, (int) start_p.row);
    s.set_line_index ((int) start_p.column);

    buffer.get_iter_at_line (out e, (int) end_p.row);
    e.set_line_index ((int) end_p.column);
}

private void on_insert_text (Gtk.TextIter iter, string text, int len_bytes) {
    InputEdit edit = {};

    //             (            )
    Gtk.TextIter start_buf;
    buffer.get_start_iter (out start_buf);
    edit.start_byte = (uint32) buffer.get_slice (start_buf, iter, false).length;

    edit.start_point = TreeSitter.Point () {
        row = (uint32) iter.get_line (),
        column = (uint32) iter.get_line_index ()
    };

    //       Tree-sitter -
    edit.old_end_byte = edit.start_byte;
    edit.old_end_point = edit.start_point;

    //      ,
    uint32 lines_added;
    uint32 last_line_bytes;
    calculate_text_stats (text, out lines_added, out last_line_bytes);

    edit.new_end_byte = edit.start_byte + (uint32) text.length;

    edit.new_end_point = TreeSitter.Point () {
        row = edit.start_point.row + lines_added,
        column = lines_added > 0 ? last_line_bytes : edit.start_point.column + last_line
    };

    tree.edit (edit);

    //                                (debounce)
```

```
    on_buffer_changed ();
}

private void calculate_text_stats (string text, out uint32 lines, out uint32 last_column
    lines = 0;
    last_column = 0;
    int i = 0;
    unichar c;

    while (text.get_next_char (ref i, out c)) {
        if (c == '\n') {
            lines++;
            last_column = 0;
        } else {
            // Tree-sitter
            //
            last_column += (uint32) c.to_utf8 (null);
        }
    }
}

private void on_delete_range (Gtk.TextIter start, Gtk.TextIter end) {
    InputEdit edit = {};

    Gtk.TextIter start_buf;
    buffer.get_start_iter (out start_buf);
    edit.start_byte = (uint32) buffer.get_slice (start_buf, start, false).length;
    edit.old_end_byte = (uint32) buffer.get_slice (start_buf, end, false).length;
    edit.new_end_byte = edit.start_byte;

    edit.start_point = TreeSitter.Point () {
        row = (uint32) start.get_line (),
        column = (uint32) start.get_line_index ()
    };
    edit.old_end_point = TreeSitter.Point () {
        row = (uint32) end.get_line (),
        column = (uint32) end.get_line_index ()
    };
    edit.new_end_point = edit.start_point;

    tree.edit (edit);

    //                                (debounce)
    on_buffer_changed ();
}
```

```
public void expand_selection () {
    if (tree == null)return;

    Gtk.TextIter start_sel, end_sel;
    //
    buffer.get_selection_bounds (out start_sel, out end_sel);

    //
    Gtk.TextIter start_buf;
    buffer.get_start_iter (out start_buf);

    //
    uint32 start_byte = (uint32) buffer.get_slice (start_buf, start_sel, false).length;
    uint32 end_byte = (uint32) buffer.get_slice (start_buf, end_sel, false).length;

    //      ,
    TreeSitter.Node root = tree.root_node ();
    TreeSitter.Node node = root.named_descendant_for_byte_range (start_byte, end_byte);

    if (node.is_null ())return;

    if (node.start_byte () == start_byte && node.end_byte () == end_byte) {
        var parent = node.parent ();
        if (!parent.is_null ())node = parent;
    }

    //
    TreeSitter.Range current_range = {
        { (uint32) start_sel.get_line (), (uint32) start_sel.get_line_index () },
        { (uint32) end_sel.get_line (), (uint32) end_sel.get_line_index () },
        start_byte,
        end_byte
    };

    selection_stack.offer_head (current_range);

    //
    Gtk.TextIter new_start, new_end;
    get_iters_from_ts_node (buffer, node, out new_start, out new_end);
    is_internal_selection_change = true;
    buffer.select_range (new_start, new_end);
    is_internal_selection_change = false;
}

public void shrink_selection () {
    if (selection_stack.is_empty)return;
```

```
    //
    var last_range = selection_stack.poll ();

    if (last_range != null) {
        Gtk.TextIter s_iter, e_iter;

        //
        get_iters_from_ts_node_coords (last_range.start_point, last_range.end_point, out

        //
        is_internal_selection_change = true;
        buffer.select_range (s_iter, e_iter);
        is_internal_selection_change = false;
    }
}

public Gee.List<BreadcrumbItem?> get_breadcrumbs_at_cursor () {
    var result = new Gee.ArrayList<BreadcrumbItem?> ();
    if (tree == null)return result;

    Gtk.TextIter insert_iter;
    buffer.get_iter_at_mark (out insert_iter, buffer.get_insert ());

    //                        (             )
    Gtk.TextIter start_buf;
    buffer.get_start_iter (out start_buf);
    uint32 byte_offset = (uint32) buffer.get_slice (start_buf, insert_iter, false).lengt

    //
    TreeSitter.Node node = tree.root_node ().named_descendant_for_byte_range (byte_offse

    //
    while (!node.is_null ()) {
        //
        if (is_container_node (node.type ())) {
            TreeSitter.Node? name_node = find_name_node (node);
            if (name_node != null && !name_node.is_null ()) {
                Gtk.TextIter s, e;
                get_iters_from_ts_node (buffer, name_node, out s, out e);
                result.insert (0, BreadcrumbItem () {
                    name = buffer.get_text (s, e, false),
                    start_point = node.start_point () //                   (fn/class)
                });
            }
        }
```

```
            node = node.parent ();
        }
        return result;
    }

    private TreeSitter.Node? find_name_node (TreeSitter.Node node) {
        string f_name = "name";
        TreeSitter.Node name_node = node.child_by_field_name (f_name, (uint32) f_name.length
        if (!name_node.is_null ())return name_node;

        //    :
        for (uint32 i = 0; i < uint32.min (node.child_count (), 5); i++) {
            TreeSitter.Node child = node.child (i);
            if (child.type ().contains ("identifier"))return child;
        }
        return null;
    }

    private void update_breadcrumbs () {
        var new_crumbs = get_breadcrumbs_at_cursor ();
        if (last_crumbs != new_crumbs) {
            last_crumbs = new_crumbs;
            breadcrumbs_changed (last_crumbs);
        }
    }

    ~BaseTreeSitterHighlighter () {
        //                free_functions   VAPI
        this.parser = null;
        this.tree = null;
        this.query = null;
        this.cursor = null;
    }
}
```

File: src/Services/DocumentManager.vala

```
/*
 * documentmanager.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
```

```
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

using Gee;
using GLib;
using Gtk;
using Panel;

public class Iide.DocumentManager : GLib.Object {
    public Window window;
    private Iide.IdeLspManager lsp_manager;
    private string? current_workspace_root;

    private LoggerService logger = LoggerService.get_instance ();
    private static DocumentManager? _instance;

    public static DocumentManager get_instance () {
        return _instance;
    }

    public Gee.HashMap<string, TextView> _documents;
    public Gee.HashMap<string, TextView> documents {
        get {
            _documents = new Gee.HashMap<string, TextView> ();
            window.grid.foreach_frame ((frame) => {
                var pages = frame.get_pages ();

                for (uint i = 0; i < pages.get_n_items (); i++) {
                    var item = pages.get_item (i) as Adw.TabPage;
                    if (item == null) {
                        continue;
                    }
                    var child = item.get_child ();
                    if (child == null) {
                        continue;
                    }
                    var text_view = child as TextView;
```

```
                if (text_view != null) {
                    _documents.set (text_view.uri, text_view);
                }
            }
        });
        return _documents;
    }
}

public DocumentManager (Window window) {
    this.window = window;
    DocumentManager._instance = this;
    // documents = new Gee.HashMap<string, TextView> ();
    lsp_manager = Iide.IdeLspManager.get_instance ();

    lsp_manager.connect_diagnostics ((uri, diagnostics) => {
        var doc = documents.get (uri);
        if (doc != null) {
            var lsp_diagnostics = new Gee.ArrayList<IdeLspDiagnostic> ();
            foreach (var diag in diagnostics) {
                var d = new IdeLspDiagnostic ();
                d.severity = diag.severity;
                d.message = diag.message;
                d.start_line = diag.start_line;
                d.start_column = diag.start_column;
                d.end_line = diag.end_line;
                d.end_column = diag.end_column;
                lsp_diagnostics.add (d);
            }

            Idle.add (() => {
                doc.update_diagnostics (lsp_diagnostics);
                return false;
            });
        }
    });
}

public void set_workspace_root (string? root) {
    current_workspace_root = root;
}

public signal void document_opened (TextView document);

public Panel.Widget? open_document (GLib.File file, Panel.Position ? pos) {
    return open_document_with_selection (file, -1, -1, -1, pos);
```

```
}

public Panel.Widget? open_document_with_selection (GLib.File file, int line, int start_c
    string uri = file.get_uri ();

    var docs = documents;
    if (docs.has_key (uri)) {
        var widget = docs.get (uri);
        widget.raise ();
        widget.view_grab_focus ();
        if (line >= 0) {
            widget.select_and_scroll (line, start_col, end_col, false);
        }
        return widget;
    } else {
        var shared_table = Iide.StyleService.get_instance ().shared_table;
        var buffer = new GtkSource.Buffer (shared_table);
        var source_file = new GtkSource.File ();
        source_file.location = file;
        var file_loader = new GtkSource.FileLoader (buffer, source_file);
        Iide.TextView? panel_widget = null;
        file_loader.load_async.begin (Priority.DEFAULT, null, null, (obj, res) => {
            try {
                file_loader.load_async.end (res);
                panel_widget = new Iide.TextView (file, buffer, window);

                panel_widget.buffer_saved.connect (() => {
                    string content = ((GtkSource.Buffer) panel_widget.text_view.buffer).
                    lsp_manager.change_document.begin (uri, content);
                });

                if (pos == null) {
                    window.grid.add (panel_widget);
                } else {
                    window.add_widget (panel_widget, pos);
                }
                panel_widget.raise ();
                panel_widget.view_grab_focus ();
                logger.debug ("Doc", "Document panel widget created: " + uri);

                if (line >= 0) {
                    // TODO: with timeout...
                    panel_widget.select_and_scroll (line, start_col, end_col, true);
                }

                string content = buffer.text;
```

```
                    string? lang_id = lsp_manager.get_language_id_for_file (file);
                    if (lang_id != null) {
                        lsp_manager.open_document.begin (uri, lang_id, content, current_work
                    }
                } catch (Error e) {
                    logger.error ("Doc", "Error Opening File", "Failed to read file %s: %s"
                }
            });
            return panel_widget;
        }
    }

    public Panel.Widget? get_document_for_file (GLib.File file) {
        string uri = file.get_uri ();
        return documents.get (uri);
    }

    public bool is_file_open (GLib.File file) {
        return documents.has_key (file.get_uri ());
    }

    public Gee.ArrayList<string> get_open_document_uris () {
        var uris = new Gee.ArrayList<string> ();
        foreach (var uri in documents.keys) {
            uris.add (uri);
        }
        return uris;
    }

    public void open_document_by_uri (string uri) {
        var file = GLib.File.new_for_uri (uri);
        if (file.query_exists (null)) {
            open_document (file, null);
        }
    }
}
```

File: src/window.vala

```
/* window.vala
 *
 * Copyright 2026 kai
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
```

```
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */

using Gtk;
using Adw;
using Panel;

public class Iide.Window : Panel.DocumentWorkspace {

    private Iide.DocumentManager document_manager;
    private Iide.ProjectManager project_manager;
    private Iide.SettingsService settings;

    private Gtk.Button lsp_btn;
    private Gtk.Spinner lsp_spin;
    private Gtk.Label lsp_count;
    private Gtk.Popover lsp_popover;
    private Gtk.Box lsp_list_box;

    private Gtk.Button global_diag_btn;
    private Gtk.Label global_diag_label;
    private Gtk.Image global_diag_icon;
    private DiagnosticsPanel panel_widget_diagnostics;

    private BasePanel[] panel_widgets;

    public Window (Gtk.Application app) {
        Object (application: app);
        GtkSource.init ();
    }

    construct {
        settings = Iide.SettingsService.get_instance ();
        document_manager = new Iide.DocumentManager (this);
        project_manager = Iide.ProjectManager.get_instance ();
        document_manager.document_opened.connect ((widget) => {
```

```
    grid.add (widget);
    widget.raise ();
    widget.view_grab_focus ();
});

project_manager.project_opened.connect ((project_root) => {
    document_manager.set_workspace_root (project_root.get_uri ());
});

project_manager.project_closed.connect (() => {
    document_manager.set_workspace_root (null);
});

// Header
var header = new Adw.HeaderBar ();
var menu_button = new Gtk.MenuButton ();
menu_button.icon_name = "open-menu-symbolic";

var menu = new GLib.Menu ();
menu.append (_("Open Project"), "app.open_project");
menu.append (_("Quick Open"), "app.fuzzy_finder");
menu.append (_("Search in Files"), "app.search_in_files");
menu.append (_("Save All"), "app.save");
menu.append (_("Preferences"), "app.preferences");
menu.append (_("About"), "app.about");
menu.append (_("Quit"), "app.quit");
menu_button.set_menu_model (menu);

header.pack_end (menu_button);

var panel_layout = settings.panel_layout;
if (panel_layout != null && panel_layout != "") {
    Iide.PanelLayoutHelper.deserialize_dock (panel_layout, dock);
} else {
    dock.reveal_start = settings.reveal_start_panel;
    dock.start_width = settings.panel_start_width;
    dock.reveal_end = settings.reveal_end_panel;
    dock.end_width = settings.panel_end_width;
    dock.reveal_bottom = settings.reveal_bottom_panel;
    dock.bottom_height = settings.panel_bottom_height;
}
var start_toggle_btn = new Panel.ToggleButton (dock, Panel.Area.START);
header.pack_start (start_toggle_btn);

var end_toggle_btn = new Panel.ToggleButton (dock, Panel.Area.END);
header.pack_end (end_toggle_btn);
```

```
set_titlebar (header);

// Theme switcher
var style_manager = Adw.StyleManager.get_default ();
style_manager.color_scheme = settings.color_scheme.to_adw_color_scheme ();

var theme_list = new Gtk.StringList ({ "System", "Light", "Dark" });
var expr = new Gtk.PropertyExpression (typeof (Gtk.StringObject), null, "string");
var theme_dropdown = new Gtk.DropDown (theme_list, expr) {
    selected = (uint) settings.color_scheme,
    tooltip_text = _("Color Scheme"),
    show_arrow = false
};

var factory = new Gtk.SignalListItemFactory ();
factory.setup.connect ((item) => {
    var list_item = item as Gtk.ListItem;
    var box = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
    var icon = new Gtk.Image ();
    var label = new Gtk.Label (null);
    label.xalign = 0;
    box.append (icon);
    box.append (label);
    list_item.set_child (box);
});
factory.bind.connect ((item) => {
    var list_item = item as Gtk.ListItem;
    var box = list_item.get_child () as Gtk.Box;
    var icon = box.get_first_child () as Gtk.Image;
    var label = icon.get_next_sibling () as Gtk.Label;
    var obj = list_item.get_item () as Gtk.StringObject;
    var text = obj.get_string ();
    label.set_label (text);

    string icon_name;
    switch (text) {
        case "System":
            icon_name = "weather-overcast-symbolic";
            break;
        case "Light":
            icon_name = "weather-clear-symbolic";
            break;
        case "Dark":
            icon_name = "weather-clear-night-symbolic";
            break;
```

```
        default:
            icon_name = "image-missing-symbolic";
            break;
    }
    icon.icon_name = icon_name;
});
theme_dropdown.set_factory (factory);

theme_dropdown.notify["selected"].connect (() => {
    var scheme = (ColorScheme) theme_dropdown.selected;
    settings.color_scheme = scheme;
    style_manager.color_scheme = scheme.to_adw_color_scheme ();
});
header.pack_end (theme_dropdown);

// statusbar (                  layout)

// create_frames ();
create_panels ();

//                        layout
restore_panels_layout ();

//      toggle button    BOTTOM             layout
var bottom_toggle_btn = new Panel.ToggleButton (dock, Panel.Area.BOTTOM);
statusbar.add_suffix (1, bottom_toggle_btn);

setup_lsp_status ();
setup_global_diag_widget ();

Timeout.add (100, () => {
    var last_project_path = settings.current_project_path;

    var grid_data = settings.grid_layout;
    bool has_grid_docs = grid_data != null && grid_data != "";

    var open_docs = settings.open_documents;
    bool has_open_docs = open_docs.size > 0;

    if (last_project_path != null && last_project_path != "") {
        bool project_opened_handled = false;

        project_manager.project_opened.connect (() => {
            if (!project_opened_handled) {
                project_opened_handled = true;
                if (has_grid_docs) {
```

```
                    restore_grid_documents (grid_data);
                } else if (has_open_docs) {
                    foreach (var uri in open_docs) {
                        document_manager.open_document_by_uri (uri);
                    }
                }
            }
        });

        project_manager.open_project_by_path (last_project_path);

        Timeout.add (500, () => {
            if (!project_opened_handled) {
                project_opened_handled = true;
                if (has_grid_docs) {
                    restore_grid_documents (grid_data);
                } else if (has_open_docs) {
                    foreach (var uri in open_docs) {
                        document_manager.open_document_by_uri (uri);
                    }
                }
            }
            return Source.REMOVE;
        });
    } else {
        if (has_grid_docs) {
            restore_grid_documents (grid_data);
        } else if (has_open_docs) {
            foreach (var uri in open_docs) {
                document_manager.open_document_by_uri (uri);
            }
        }
    }

    return Source.REMOVE;
});

// Handle window close
this.close_request.connect (() => {
    save_window_settings ();
    settings.open_documents = document_manager.get_open_document_uris ();
    bool has_unsaved = false;
    foreach (var entry in document_manager.documents.entries) {
        if (entry.value is Iide.TextView) {
            var tv = (Iide.TextView) entry.value;
            if (tv.is_modified) {
```

```
                    has_unsaved = true;
                    break;
                }
            }
        }
        if (has_unsaved) {
            var dialog = new Adw.AlertDialog (_("Unsaved Changes"), _("You have unsaved
            dialog.add_response ("cancel", _("Cancel"));
            dialog.add_response ("discard", _("Discard"));
            dialog.add_response ("save", _("Save"));
            dialog.set_response_appearance ("save", Adw.ResponseAppearance.SUGGESTED);
            dialog.response.connect ((response) => {
                if (response == "save") {
                    foreach (var entry in document_manager.documents.entries) {
                        if (entry.value is Iide.TextView) {
                            var tv = (Iide.TextView) entry.value;
                            if (tv.is_modified) {
                                tv.save ();
                            }
                        }
                    }
                    settings.open_documents = document_manager.get_open_document_uris ()
                    this.destroy ();
                } else if (response == "discard") {
                    settings.open_documents = document_manager.get_open_document_uris ()
                    this.destroy ();
                }
            });
            dialog.present (this);
            return true;
        }
        return false;
    });

    repair_empty_areas ();
}

private void repair_empty_areas () {
    bool empty_start = true;
    bool empty_bottom = true;
    bool empty_end = true;
    dock.foreach_frame ((frame) => {
        var position = frame.get_position ();
        switch (position.area) {
            case Panel.Area.START:
                empty_start = false;
```

```
                break;
            case Panel.Area.BOTTOM:
                empty_bottom = false;
                break;
            case Panel.Area.END:
                empty_end = false;
                break;
            default:
                break;
        }
    });

    if (empty_start) {
        var position = new Panel.Position () {
            area = Panel.Area.START
        };
        var tmp_panel = new Panel.Widget ();
        add_widget (tmp_panel, position);
        dock.remove (tmp_panel);
    }

    if (empty_bottom) {
        var position = new Panel.Position () {
            area = Panel.Area.BOTTOM
        };
        var tmp_panel = new Panel.Widget ();
        add_widget (tmp_panel, position);
        dock.remove (tmp_panel);
    }

    if (empty_end) {
        var position = new Panel.Position () {
            area = Panel.Area.END
        };
        var tmp_panel = new Panel.Widget ();
        add_widget (tmp_panel, position);
        dock.remove (tmp_panel);
    }
}

private void create_panels () {
    panel_widget_diagnostics = new DiagnosticsPanel ();

    panel_widgets = {
        new ProjectPanel (),
        new TerminalPanel (),
```

```
        new LogPanel (),
        panel_widget_diagnostics
    };
}

private void initialize_panels () {
    foreach (var panel in panel_widgets) {
        add_widget (panel, panel.initial_pos ());
    }
}

private void restore_panels_layout () {
    var dock_layout = settings.panel_layout;
    if (dock_layout == null || dock_layout == "") {
        initialize_panels ();
        return;
    }

    var widget_layouts = Iide.PanelLayoutHelper.parse_widgets (dock_layout);

    foreach (var panel_widget in panel_widgets) {
        var widget_layout = widget_layouts.get (panel_widget.panel_id ());
        var pos = widget_layout != null? widget_layout.to_pos () : panel_widget.initial_

        add_widget (panel_widget, pos);
    }
}

private void save_window_settings () {
    settings.panel_layout = Iide.PanelLayoutHelper.serialize_dock (dock);
    var grid_json = Iide.PanelLayoutHelper.serialize_grid (grid);
    settings.grid_layout = grid_json;

    bool maximized = false;
    var surface = this.get_surface ();
    if (surface != null) {
        var toplevel = surface as Gdk.Toplevel;
        if (toplevel != null) {
            var state = toplevel.get_state ();
            maximized = (state & Gdk.ToplevelState.MAXIMIZED) != 0;
        }
    }

    if (!maximized) {
        settings.window_width = (int) this.get_width ();
        settings.window_height = (int) this.get_height ();
```

```
    }
    settings.window_maximized = maximized;
}

private async void restore_grid_documents_async (Gee.ArrayList<Iide.PanelLayoutHelper.Do
    uint last_col = 0;
    foreach (var doc_info in sorted_docs) {
        if (doc_info.column > last_col) {
            last_col = doc_info.column;
        }
    }

    for (uint i = 1; i <= last_col; i++) {
        grid.insert_column (i);
    }

    foreach (var doc_info in sorted_docs) {
        var file = GLib.File.new_for_uri (doc_info.uri);
        var pos = new Panel.Position ();
        pos.area = Panel.Area.CENTER;
        pos.column = doc_info.column;
        pos.row = doc_info.row;
        document_manager.open_document (file, pos);
    }
}

private void restore_grid_documents (string grid_data) {
    var docs = Iide.PanelLayoutHelper.parse_grid_documents (grid_data);
    if (docs.size == 0) {
        return;
    }

    var sorted_docs = new Gee.ArrayList<Iide.PanelLayoutHelper.DocumentInfo> ();
    foreach (var doc in docs) {
        sorted_docs.add (doc);
    }
    sorted_docs.sort ((a, b) => {
        int col_cmp = (int) (a.column - b.column);
        if (col_cmp != 0)return col_cmp;
        return (int) (a.row - b.row);
    });

    restore_grid_documents_async.begin (sorted_docs);
}

public void save_modified () {
```

```
    foreach (var entry in document_manager.documents.entries) {
        var widget = entry.value;
        if (widget is Iide.TextView) {
            var tv = widget as Iide.TextView;
            if (tv.is_modified) {
                tv.save ();
            }
        }
    }
}

public void open_project_dialog () {
    project_manager.open_project_dialog.begin (this);
}

public Iide.DocumentManager get_document_manager () {
    return document_manager;
}

public SourceView ? get_active_source_view () {
    // 1.
    Panel.Widget? active_widget = this.get_grid ().get_most_recent_frame ().get_visible_

    if (active_widget == null)return null;

    return (active_widget as TextView) ? .source_view;
}

private void setup_lsp_status () {
    // 1.             Statusbar
    var btn_content = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
    lsp_spin = new Gtk.Spinner ();
    lsp_count = new Gtk.Label ("0");
    btn_content.append (lsp_spin);
    btn_content.append (lsp_count);

    lsp_btn = new Gtk.Button () { child = btn_content, visible = false };
    lsp_btn.add_css_class ("flat");
    this.statusbar.add_prefix (100, lsp_btn);

    // 2.      Popover
    lsp_popover = new Gtk.Popover ();
    lsp_popover.set_parent (lsp_btn);
    var scroll = new Gtk.ScrolledWindow () {
        max_content_height = 300,
        propagate_natural_height = true,
```

```
        width_request = 350 //
    };
    lsp_list_box = new Gtk.Box (Gtk.Orientation.VERTICAL, 0);
    lsp_list_box.add_css_class ("boxed-list"); //     Adwaita

    scroll.set_child (lsp_list_box);
    lsp_popover.set_child (scroll);

    lsp_btn.clicked.connect (() => lsp_popover.popup ());

    // 3.
    IdeLspService.get_instance ().tasks_changed.connect (update_lsp_ui);
}

private void update_lsp_ui (Gee.List<LspTaskInfo?> tasks) {
    //
    Gtk.Widget? child;
    while ((child = lsp_list_box.get_first_child ()) != null)
        lsp_list_box.remove (child);

    if (tasks.size == 0) {
        lsp_btn.hide ();
        lsp_popover.popdown ();
        return;
    }

    lsp_btn.show ();
    lsp_spin.start ();
    lsp_count.label = tasks.size.to_string ();

    foreach (var task in tasks) {
        var row = new Adw.ActionRow () {
            title = task.server_name,
            subtitle = task.message
        };

        if (task.percentage >= 0) {
            var progress = new Gtk.ProgressBar () {
                fraction = task.percentage / 100.0,
                valign = Gtk.Align.CENTER
            };
            progress.add_css_class ("osd"); //
            row.add_suffix (progress);
        }

        lsp_list_box.append (row);
```

```
        }
    }

    private void setup_global_diag_widget () {
        var content = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 6);
        global_diag_icon = new Gtk.Image.from_icon_name ("emblem-ok-symbolic");
        global_diag_label = new Gtk.Label ("OK");

        content.append (global_diag_icon);
        content.append (global_diag_label);

        global_diag_btn = new Gtk.Button ();
        global_diag_btn.set_child (content);
        global_diag_btn.add_css_class ("flat");

        //
        global_diag_btn.clicked.connect (() => {
            panel_widget_diagnostics.raise ();
        });

        this.statusbar.add_prefix (50, global_diag_btn);

        //
        DiagnosticsService.get_instance ().total_count_changed.connect (update_global_diag_s
    }

    private void update_global_diag_status (int errors, int warns) {
        if (errors == 0 && warns == 0) {
            global_diag_icon.icon_name = "emblem-ok-symbolic";
            global_diag_label.label = "OK";
            global_diag_btn.remove_css_class ("error-state"); //
        } else {
            global_diag_icon.icon_name = "dialog-error-symbolic";
            global_diag_label.label = @"$errors / $warns";
            global_diag_btn.add_css_class ("error-state");
        }
    }
}
```

File: src/Widgets/TextView/TextView.vala

```
/*
 * textdocument.vala
 *
 * Copyright 2026 kai
 *
```

```
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program.  If not, see <https://www.gnu.org/licenses/>.
 *
 * SPDX-License-Identifier: GPL-3.0-or-later
 */



public class Iide.SaveDelegate : Panel.SaveDelegate {
    private Iide.TextView view;
    public SaveDelegate (Iide.TextView view) {
        Object ();
        this.view = view;

        var file = GLib.File.new_for_uri (view.uri);
        title = view.title;
        subtitle = file.get_path ();
    }

    public override bool save (GLib.Task task) {
        message ("Saving in delegate...");
        var result = view.save ();
        task.return_boolean (result);
        return result;
    }

    public override void close () {
        view.force_close ();
    }

    public override void discard () {
        view.force_close ();
    }
}

public class Iide.TextView : Panel.Widget {
```

```
public SourceView source_view;
private GtkSource.Map source_map;
private FontZoomer font_zoomer;
private Iide.SettingsService settings;
private EditorStatusBar editor_status_bar;

public Window window;
public string uri { get; private set; }
public SourceView text_view { get { return source_view; } }

public bool is_modified { get { return ((GtkSource.Buffer) source_view.buffer).get_modif

public signal void buffer_saved ();

public TextView (GLib.File file, GtkSource.Buffer buffer, Window window) {
    Object ();
    this.uri = file.get_uri ();
    this.window = window;
    this.settings = Iide.SettingsService.get_instance ();

    var adw_style_manager = Adw.StyleManager.get_default ();

    var style_manager = GtkSource.StyleSchemeManager.get_default ();
    if (adw_style_manager.color_scheme == Adw.ColorScheme.FORCE_LIGHT) {
        buffer.set_style_scheme (style_manager.get_scheme ("Adwaita"));
    } else {
        buffer.set_style_scheme (style_manager.get_scheme ("Adwaita-dark"));
    }

    adw_style_manager.notify["color-scheme"].connect (() => {
        if (adw_style_manager.color_scheme == Adw.ColorScheme.FORCE_LIGHT) {
            buffer.set_style_scheme (style_manager.get_scheme ("Adwaita"));
        } else {
            buffer.set_style_scheme (style_manager.get_scheme ("Adwaita-dark"));
        }
    });

    source_view = new SourceView (window, uri, buffer);
    icon_name = source_view.icon_name;
    font_zoomer = new FontZoomer (source_view);

    // Connect to application-level zoom and minimap changes
    var app = GLib.Application.get_default () as Iide.Application;
    if (app != null) {
        app.zoom_changed.connect ((level) => {
            font_zoomer.set_zoom_level (level);
```

```
    });
    app.minimap_changed.connect ((visible) => {
        toggle_minimap_visible (visible);
    });
}

// Connect to settings changes to apply to all open documents
settings.editor_setting_changed.connect ((key) => {
    switch (key) {
        case "editor-font-size":
            font_zoomer.set_zoom_level (settings.editor_font_size);
            break;
        case "show-minimap":
            toggle_minimap_visible (settings.show_minimap);
            break;
    }
});

source_map = new GtkSource.Map ();
source_map.set_view (source_view);
source_map.add_css_class ("textview-map");
source_map.visible = settings.show_minimap;

font_zoomer.zoom_changed.connect ((level) => {
    source_view.mark_renderer.set_icons_size (FontSizeHelper.get_size_for_zoom_level
});

var scroll = new Gtk.ScrolledWindow ();
scroll.hexpand = true;
scroll.vexpand = true;

scroll.set_child (source_view);

scroll.get_vadjustment ().bind_property ("value",
                                         source_map.get_vadjustment (), "value",
                                         BindingFlags.BIDIRECTIONAL | BindingFlags.S

var subbox = new Gtk.Box (Gtk.Orientation.HORIZONTAL, 0);
subbox.homogeneous = false;
subbox.append (scroll);
subbox.append (source_map);

var font_map = Pango.CairoFontMap.get_default ();
source_map.set_font_map (font_map);

var box = new Gtk.Box (Gtk.Orientation.VERTICAL, 0);
```

```
        box.append (subbox);

        this.editor_status_bar = new EditorStatusBar (source_view);
        box.append (this.editor_status_bar);
        if (source_view.ts_highlighter != null) {
            source_view.ts_highlighter.breadcrumbs_changed.connect (this.editor_status_bar.u
        }
        this.editor_status_bar.breadcrumb_clicked.connect ((line, column) => {
            Gtk.TextIter iter;
            buffer.get_iter_at_line (out iter, (int) line);
            iter.set_line_index ((int) column);

            buffer.place_cursor (iter);
            source_view.scroll_to_iter (iter, 0.1, false, 0, 0.5);
            source_view.grab_focus ();
        });


        child = box;

        title = file.get_basename ();

        save_delegate = new Iide.SaveDelegate (this);
        modified = false;

        buffer.modified_changed.connect_after (() => {
            modified = buffer.get_modified ();
        });
        buffer.notify["cursor-position"].connect (() => {
            Gtk.TextIter insert, selection;
            buffer.get_selection_bounds (out insert, out selection);

            // 1.
            int line = insert.get_line ();
            int col = insert.get_line_index (); //
            int sel_len = insert.get_offset () - selection.get_offset ();

            editor_status_bar.update_position (line, col, sel_len);
        });

        //          (Insert/Overwrite)
        source_view.notify["overwrite"].connect (() => {
            editor_status_bar.update_mode (source_view.overwrite);
        });
    }
```

```
public override void size_allocate (int width, int height, int baseline) {
    base.size_allocate (width, height, baseline);
    source_map.visible = settings.show_minimap && width >= 600;
}

public void toggle_minimap_visible (bool visible) {
    source_map.visible = visible;
}

public void view_grab_focus () {
    source_view.grab_focus ();
}

public bool save () {
    try {
        var text = source_view.buffer.text;
        var file = GLib.File.new_for_uri (uri);
        file.replace_contents (text.data, null, false, GLib.FileCreateFlags.NONE, null);
        ((GtkSource.Buffer) source_view.buffer).set_modified (false);
        buffer_saved ();
    } catch (Error e) {
        critical (e.message);
    }
    return true;
}

public void update_diagnostics (Gee.ArrayList<IdeLspDiagnostic> diagnostics) {
    var text_buffer = (Gtk.TextBuffer) source_view.buffer;

    text_buffer.begin_user_action ();

    LspDiagnosticsMark.clear_mark_attributes (source_view);

    int line_count = text_buffer.get_line_count ();

    int lsp_error_count = 0;
    int lsp_warning_count = 0;
    int lsp_info_count = 0;

    foreach (var diag in diagnostics) {
        if (diag.start_line >= line_count) {
            continue;
        }

        Gtk.TextIter start_iter;
```

```
            text_buffer.get_iter_at_line (out start_iter, diag.start_line);

            var mark = new LspDiagnosticsMark.from_lsp_diagnostic (diag);
            text_buffer.add_mark (mark, start_iter); //

            switch (diag.severity) {
            case 1:
                lsp_error_count++;
                break;
            case 2:
                lsp_warning_count++;
                break;
            case 3:
            case 4:
                lsp_info_count++;
                break;
            }
        }

        text_buffer.end_user_action ();

        this.editor_status_bar.update_diagnostics (lsp_error_count, lsp_warning_count, lsp_i
    }

    public void select_and_scroll (int line, int start_col, int end_col, bool is_new) {
        source_view.select_and_scroll (line, start_col, end_col, is_new);
    }
}
```

File: src/meson.build

```
iide_sources = [
    'main.vala',
    'application.vala',
    'window.vala',
    'Widgets/TextView/TextView.vala',
    'Widgets/TextView/SourceView.vala',
    'Widgets/TextView/FontZoomer.vala',
    'Widgets/TextView/GutterMarkRenderer.vala',
    'Widgets/TextView/EditorStatusBar.vala',
    'Widgets/TextView/LspCompletionProvider.vala',
    'Widgets/TextView/DiagnosticsPopover.vala',
    'Widgets/ToolViews/ProjectView.vala',
    'Widgets/ToolViews/TerminalView.vala',
    'Widgets/ToolViews/LogView.vala',
    'Widgets/PreferencesDialog.vala',
```

```
    'Widgets/FuzzyFinderDialog.vala',
    'Widgets/SearchInFilesDialog.vala',
    'Widgets/Panels/BasePanel.vala',
    'Widgets/Panels/LogPanel.vala',
    'Widgets/Panels/TerminalPanel.vala',
    'Widgets/Panels/DiagnosticsPanel.vala',
    'Widgets/Panels/ProjectPanel.vala',
    'Services/Utils.vala',
    'Services/JsonUtils.vala',
    'Services/StyleManager.vala',
    'Services/LoggerService.vala',
    'Services/DocumentManager.vala',
    'Services/IconProvider.vala',
    'Services/ProjectManager.vala',
    'Services/SettingsService.vala',
    'Services/PanelLayoutHelper.vala',
    'Services/LSP/LspTypes.vala',
    'Services/LSP/config/LspConfig.vala',
    'Services/LSP/config/PythonLspConfig.vala',
    'Services/LSP/config/LspRegistry.vala',
    'Services/LSP/LspClient.vala',
    'Services/LSP/IdeLspService.vala',
    'Services/LSP/IdeLspManager.vala',
    'Services/LSP/LspDiagnosticsMark.vala',
    'Services/LSP/DiagnosticsService.vala',
    'Services/TreeSitter/TreeSitterManager.vala',
    'Services/TreeSitter/model/AstItem.vala',
    'Services/TreeSitter/model/AstModel.vala',
    'Services/TreeSitter/BaseTreeSitterHighlighter.vala',
    'Services/TreeSitter/cpp/CppTSHighlighter.vala',
    'Services/TreeSitter/vala/ValaTSHighlighter.vala',
    'Services/TreeSitter/rust/RustHighlighter.vala',
    'Services/TreeSitter/python/PythonTSHighlighter.vala',
    'Services/Actions/Action.vala',
    'Services/Actions/ActionManager.vala',
    'Services/Actions/ShortcutSettings.vala',
]

iide_deps = [
    config_dep,
    dependency('gtk4'),
    dependency('gio-2.0', version: '>= 2.50'),
    dependency('libadwaita-1', version: '>= 1.4'),
    dependency('libpanel-1', version: '>= 1.7'),
    dependency('gtksourceview-5'),
    dependency('vte-2.91-gtk4', version: '>= 0.75.0'),
```

```
    dependency('gee-0.8'),
    dependency('json-glib-1.0'),
    dependency('jsonrpc-glib-1.0'),
]

iide_sources += gnome.compile_resources('iide-resources', 'iide.gresource.xml', c_name: 'iid

executable(
    'iide',
    iide_sources,
    dependencies: [iide_deps, tree_sitter_dep],
    include_directories: config_inc,
    install: true,
)
```